

Ducks in a Row

Certificate Lifecycle Automation & Management for ADCS

Installation & Configuration Guide

Make ADCS work like Let's Encrypt for your internal network.

Version 1.1.1
2026

Table of Contents

1. Quick Start 3

2. Intended Use 15

3. System Requirements 19

4. What to Ask Your AD and CA Team 23

5. Installation 28

6. Monitor Mode 34

7. Several Certificate Authorities 39

8. Configuration 43

9. ADCS Configuration 54

10. Verifying Your Installation 58

11. Connecting ACME Clients 63

12. Kubernetes and OpenShift 71

13. External Account Binding 81

14. Device Attestation 84

15. Revocation 89

16. Expiry Alerts 93

17. Licence and Paid Module 99

18. Hardening 104

19. Backup and Restore 113

20. Troubleshooting 119

21. Uninstalling 135

22. Glossary 138

1. Quick Start

Get from a fresh install to your first issued certificate. Setup takes about five minutes of clicking once the approvals are in place. The approvals are the rights this server's computer account needs on the CA and its templates, and they are usually the lead time, so ask for them first.

New to ACME or ADCS? The glossary explains the key terms.

Before you start

Most of what follows is granted by the people who run your Active Directory, your CA and your network. Send them what to ask your AD and CA team first: it lists every right and firewall opening the server needs, why each one is needed and where it is granted, with a change request ready to copy.

You need:

- A Windows Server that is **domain joined** to the same forest as your ADCS certificate authority.
- The **ADCS Remote Administration Tools** installed on that server. They register the COM classes Ducks in a Row uses to talk to the CA:

```
Install-WindowsFeature RSAT-ADCS-Mgmt
```

- The server's **machine account**, `DOMAIN\SERVERNAME$`, granted:
  - **Enroll** on each certificate template you want to expose,
  - **Request Certificates** on the CA itself (Certification Authority console, CA properties, Security tab). Authenticated Users hold it on a default CA, so check only that nobody removed it, and
  - **Read** on the CA, on the same tab. Read is what lets the dashboard sync the certificate inventory. Without it, enrolment still works but the dashboard stays empty.
- Network access from the server to the CA on TCP 135 and TCP 49152 to 65535, and the server's account admitted by the CA server's **Certificate Service DCOM Access** group. On a default CA that group holds Authenticated Users, so this too is usually in place already.
- Optional: your CA connection string in `CAHOST\CA Name` form. The setup wizard discovers the CAs published in Active Directory on its own; you only need the string for manual entry. Find it with:

```
certutil -config - -ping
```

1. Install

Download from the latest release and run it on the server. There are two downloads and you almost certainly want the first:

- `Ducks-in-a-Row-Setup.exe` carries the ASP.NET Core 10 runtime inside it, so it installs on a server with no internet access from the one file. It skips the runtime install when one is already present, and never removes it on uninstall.
- `Ducks-in-a-Row.msi` is the application alone, for deployment systems that manage the runtime themselves. Without the runtime the service fails before it can log anything and the installer reports a

misleading **Error 1920**.

Both installers are code signed by Haruspex Systems B.V. Check the signature and the SHA256 checksum on the release page before you run either one, as checking the download describes. Windows SmartScreen may still warn about a new release; if it does, **More info** should name Haruspex Systems B.V. as the publisher. The install wizard lets you:

- choose the destination folder (default `C:\Program Files\Ducks in a Row\`),
- choose the data folder for the database, logs, and runtime configuration (default `C:\ProgramData\Ducks in a Row\`),
- choose whether to start the service immediately,
- choose monitor mode, which watches the CA and never issues or revokes, for a read only first step on a production CA,
- open the setup page in your browser from the completion screen.

In all cases the installer:

- copies the application to the destination folder you chose,
- creates the data folder and records its location in the registry so the service finds it,
- registers the Windows service **DucksInARow** (display name "Ducks in a Row Certificate Proxy", runs as LocalSystem, set to start automatically on boot),
- opens inbound firewall rules on **TCP 5000** (HTTP) and **TCP 5001** (HTTPS),
- adds a **Ducks in a Row** Start Menu shortcut to the web UI.

Unattended install

```
msiexec /i Ducks-in-a-Row.msi /qn INSTALLFOLDER="D:\Ducks in a Row" DATAFOLDER="D:\DucksData"
START_SERVICE=0
```

- `INSTALLFOLDER`: destination folder (optional; defaults to `C:\Program Files\Ducks in a Row`).
- `DATAFOLDER`: data folder (optional; defaults to `C:\ProgramData\Ducks in a Row`). Choose it at install time; moving it on a later upgrade is not supported.
- `START_SERVICE`: `1` (default) starts the service after install; `0` registers it but leaves it stopped until the next boot or a manual start.
- `MONITOR_ONLY`: `1` installs in monitor mode. Leave it out for full mode.

`Ducks-in-a-Row-Setup.exe /quiet` takes the same properties.

The "open the setup page" prompt only appears in the interactive wizard, so a `/qn` install never launches a browser.

2. Run the setup wizard

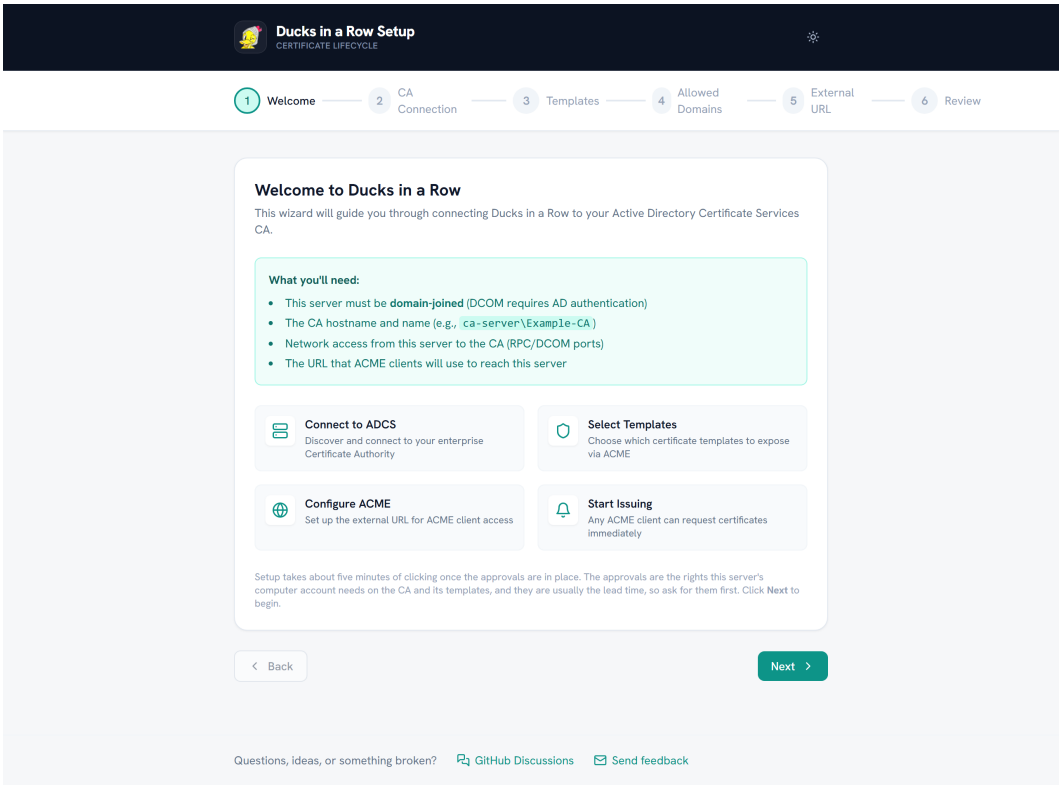
Open the dashboard in a browser on `https://your-server:5001` (or `http://your-server:5000`). You must sign in as a member of the administrator group. By default that is the server's built in Administrators group; set `Auth:AdminGroup` in `settings.json` in the data folder to delegate to a specific Windows or Active

Directory group.

> The HTTPS endpoint uses a self signed certificate out of the box (generated in the data folder as `ducks-selfsigned.bin`), so your browser will warn on first visit. For production, give Ducks in a Row a certificate your clients already trust (see Giving clients a TLS certificate they trust below).

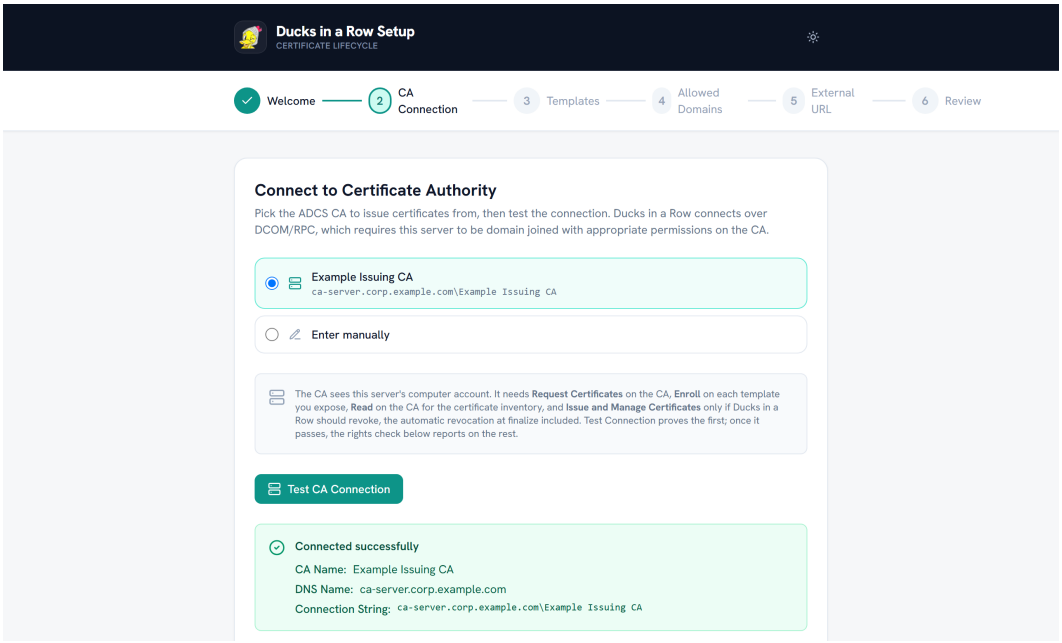
Until setup completes the service is **unconfigured**: it serves the wizard and the dashboard shell, and every CA operation answers 503. It never falls back to a fake CA on its own.

On first run you are taken to the setup wizard. It walks you through six short steps. It remembers where you were, so a service restart in the middle of it does not send you back to the beginning.



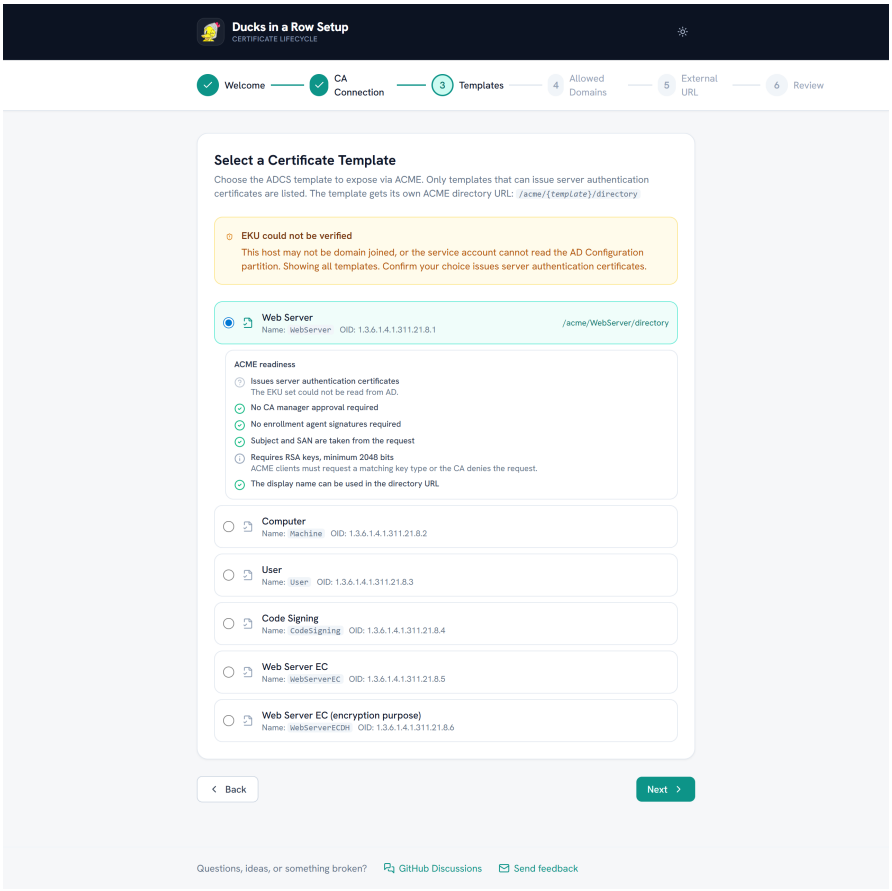
Setup wizard welcome step listing what you need before you start

1. **Welcome.** A short checklist of what you need before you start.
2. **Connect a CA.** The wizard discovers the CAs published in Active Directory and lets you pick one, or enter `CAHOST\CA Name` by hand. It then tests the connection through the same DCOM path the service uses in production. A pass proves that the service's account holds Request Certificates on the CA, and the wizard then checks the service's other rights. The Review step runs the full check, template permissions included, and says plainly what is not proven yet. See the service's rights.



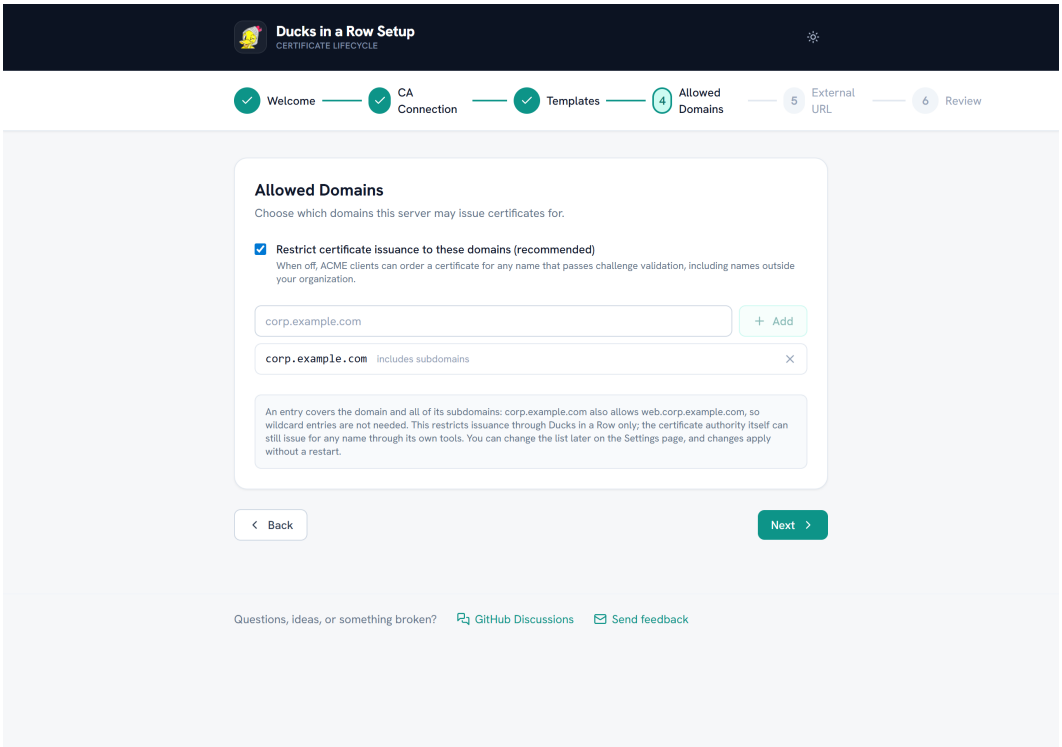
Setup wizard connection step showing a discovered CA and the test connection button

3. **Choose a template.** It lists the templates that CA publishes, checks each one for ACME readiness, and lets you choose the template to expose. No template is exposed over ACME until a selection is recorded on this step: on a fresh install, every template's directory URL answers 403 until then.



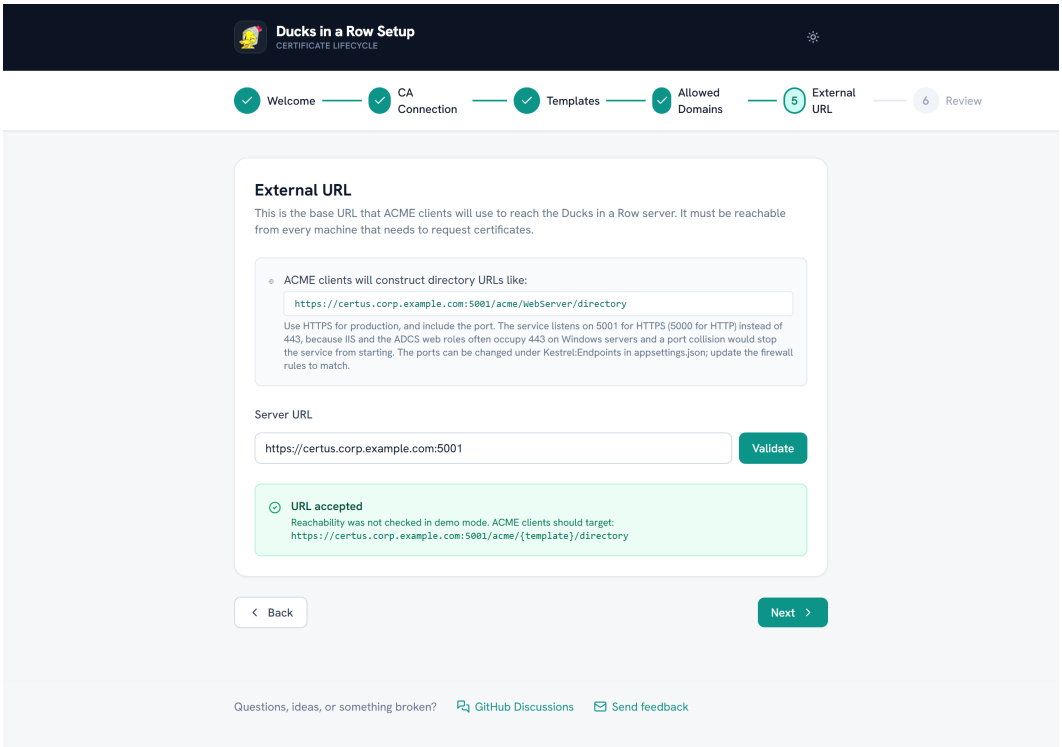
Setup wizard templates step showing the ACME readiness checklist

4. **Client access.** Two controls decide who can get a certificate, and on a new install both arrive switched on. **External account binding** is required: an ACME client can register only with a credential you create for it on the dashboard's ACME page. That is why step 3 below starts by issuing one. See external account binding. **Allowed domains** decides which names this server may issue certificates for. On a domain joined server the restriction is already on with your Active Directory domain filled in; in a workgroup there is nothing to suggest, so it starts off. An entry covers the domain and all of its subdomains, so `corp.example.com` also covers `web.corp.example.com`; wildcard entries are not needed and are not accepted. The list restricts issuance **through Ducks in a Row only**: the CA itself can still issue for any name through its own tools, so this complements CA side controls like name constraints rather than replacing them. You can switch either control off, and the step says in one line what that gives up. Leaving both on is the recommendation; the hardening guide explains why binding matters most in an Active Directory domain. Both can be changed later without a restart: binding on the ACME page, the domain list on the Settings page.



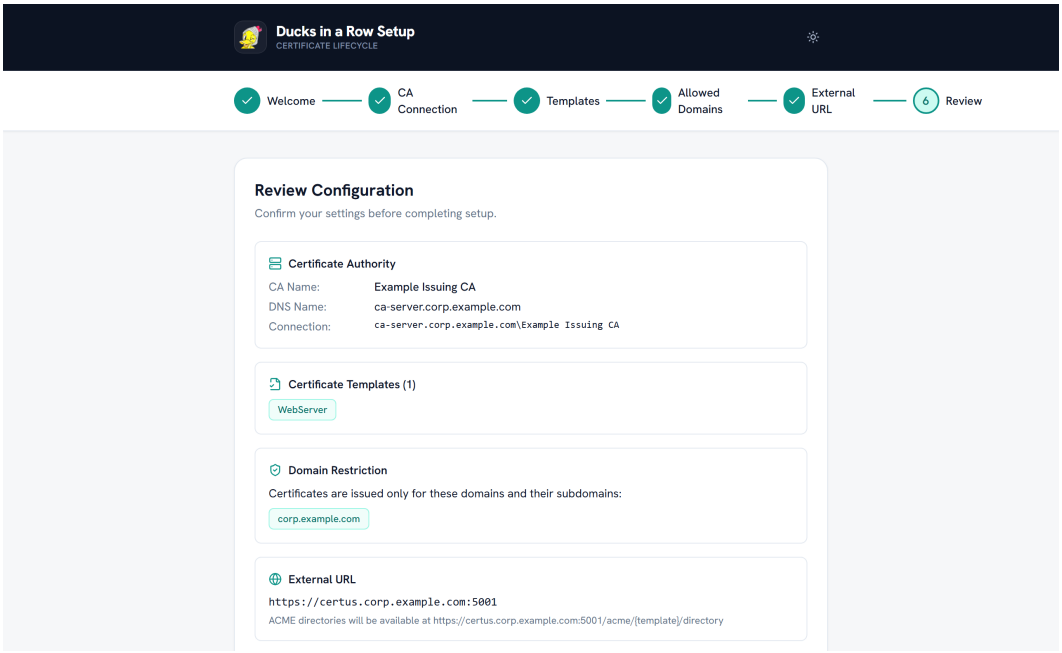
Setup wizard client access step with binding required, the domain restriction enabled and one domain listed

5. **Set the external URL.** This is the address clients will use to reach the server. It comes prefilled with a suggestion and is checked for you. On this step the wizard can also enrol an HTTPS certificate for the server from your CA in one click, so clients trust the connection (see Giving clients a TLS certificate they trust).



Setup wizard external URL step showing the server URL and the directory URL clients will use

6. **Review and apply.** The step summarises your choices, binding and the domain restriction included, and runs the rights check again, with a row for each template you chose. Unless step 5 enrolled an HTTPS certificate for the server, Complete Setup waits for you to tick a box naming the rights that are not proven yet. It never refuses because a right is missing. Your choices are then saved to `settings.json` in the data folder and the service restarts itself to load them. The wizard waits for the service to come back, then shows a ready to run certbot command, a link to sign up for security notices, and a button to open the dashboard. The Settings page keeps that link.



Setup wizard review step summarising the CA, the template, external account binding, the allowed domains and the external URL

To point an existing install at a different CA later, you have two options. Either edit `Certus:CaConnectionString` in `settings.json` in the data folder and restart the service, or clear that value and restart to reopen the wizard. Setup locks once it is complete, so the wizard will not reopen while a CA is still configured.

Before you switch, let every ACME order that is waiting at the current CA for a manager's approval finish: approve or deny each pending request there. An order still waiting after the switch cannot complete, because the new CA does not hold its request, and the old CA's request is left for you to deny by hand. See the one automatic revocation.

3. Request your first certificate

Issue a credential

External account binding is required on a new install, so before a client can register it needs a credential from you:

1. On the dashboard, open the **ACME** page and its **Credentials** tab.
2. Choose **New credential**, give it a name that says which client or team holds it, and create it.
3. Copy the **key ID** and the **HMAC key** from the panel that opens. The HMAC key is shown only now and again if you regenerate it, never afterwards.

Hand both values to the client. The panel also writes ready to paste client setup for certbot, win-acme and the others. If you switched binding off in the wizard, skip this step. See issuing a credential.

Point a client at the directory

Each certificate template is its own ACME endpoint:

```
https://your-server:5001/acme/<template>/directory
```

`<template>` is the template's programmatic name (its AD `cn`) or its display name. Display names with spaces work; the client URL encodes them. A display name carrying an invisible character, such as a soft hyphen left behind by a paste from a word processor, is refused with a 400; use the programmatic name and see Troubleshooting.

Point any standard ACME client at that directory URL and request a certificate. A quick test with certbot:

```
certbot certonly --standalone \
  --server https://your-server:5001/acme/WebServer/directory \
  --eab-kid <key ID> --eab-hmac-key <HMAC key> \
  --email you@example.com \
  -d host.corp.example.com \
  --key-type rsa --rsa-key-size 2048
```

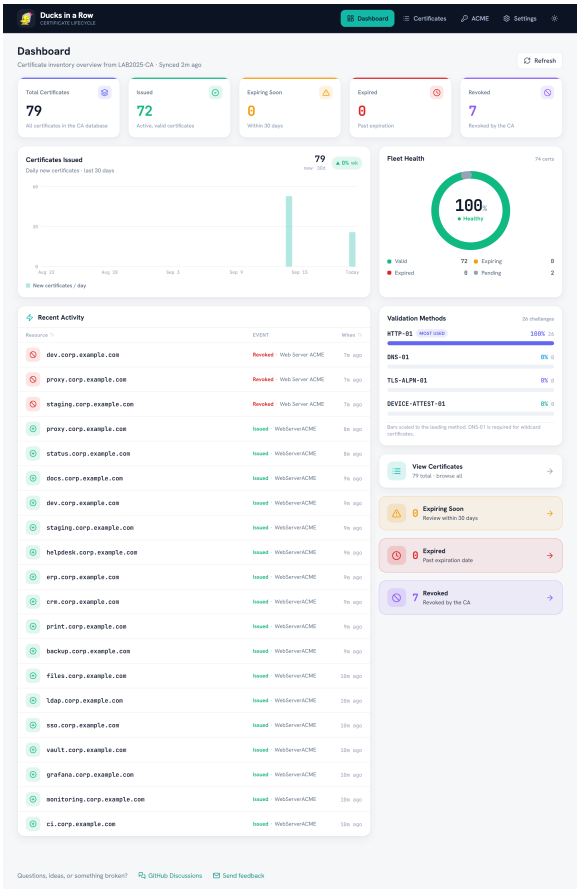
The two `--eab` flags matter only for the client's first registration; a certbot that already holds an account on this server does not need them again.

Match the key type to the template. Your client must ask for the key type the template wants, and templates differ. An RSA template needs an RSA key; an `ECDSA_P256` template needs a key on that curve. Ask for the wrong one and the CA policy module refuses at finalize with `Denied by Policy Module`. You do not have to work this out yourself. The wizard's template step reports the key algorithm it read from the template, and the **Client setup** snippets on the dashboard's ACME page arrive with the right flags already filled in. Copy them from there rather than from memory. The stock `Web Server ACME` template is RSA, which is why the command above forces RSA: most clients default to an elliptic curve key. See connecting ACME clients for the per client flags.

No account pre-registration is needed beyond the credential: the client creates an account from its own key the first time it connects and binds it to the credential then. If binding was switched off, the client needs no credential at all. See external account binding.

See Connecting ACME clients for certbot, win-acme, Caddy, Traefik, and Posh-ACME.

4. Confirm it worked



The dashboard showing the certificate inventory and expiry tiles after the first sync

- The client writes out the issued certificate and chain.
- The certificate appears in the dashboard inventory within one sync cycle (every 5 minutes by default, `Certus:SyncIntervalMinutes`).
- The health endpoint reports healthy:

```
Invoke-RestMethod https://your-server:5001/health
```

If anything misbehaves, see Troubleshooting.

Configuration reference

The settings below are the ones a first install usually touches. Configuration is the complete list, including alerting, rate limiting, challenge egress and the keys that ship as an explicit null.

Configuration is layered. Shipped defaults live in `appsettings.json` in the installation folder; the setup wizard writes instance settings (CA connection string, external URL) to `settings.json` in the **data folder**, which overrides them and survives upgrades. Environment variables override both. Restart the service after editing either file.

Setting	Default	Purpose
`Certus:CaConnectionString`	`null` (unconfigured; the wizard sets it)	ADCS CA in `CAHOST\CA Name` form
`Certus:DatabasePath`	`ducks.db` in the data folder	SQLite database file. It must stay in the data folder, or the service refuses to start (why)
`Certus:ExternalUrl`	`null`	The base URL you expect clients to use. Recorded and checked at startup; ACME URLs themselves follow the host the client connects to
`Certus:SyncIntervalMinutes`	`5`	How often the dashboard syncs from the CA
`Certus:RequestHistoryDays`	`30`	How far back the sync reaches for pending, denied, and failed requests, so their detail pages can show the CA's own explanation. Issued and revoked certificates are always synced in full. Set to `0` to skip those three passes entirely, which also empties the Pending, Denied, and Failed filters on the certificate list
`Certus:Acme:ExposeAllTemplates`	`false`	Break glass override: expose every CA published template over ACME, ignoring the wizard's template selection. Leave it `false`; the open posture is logged as a warning at startup
`Certus:EnableWalMode`	`true`	SQLite write ahead logging, for better concurrent read and write performance. Set it to `false` on a filesystem that cannot support WAL, or when a backup or replication tool needs a single database file with no `-wal` and `-shm` sidecars. Applied in both directions on every start, so changing it converts the existing database. See Troubleshooting for what each mode means. The data folder must be on a local disk
`Auth:Mode`	`Negotiate`	Windows Integrated Authentication for the dashboard
`Auth:AdminGroup`	`null` (built in Administrators)	Group allowed into the dashboard and setup

Setting	Default	Purpose
`Auth:RequireHttps`	`true`	HSTS and HTTP to HTTPS redirect outside development
`Auth:TrustedProxies`	`[]`	Reverse proxy IPs whose `X-Forwarded-*` headers are trusted

Challenge validation egress

For HTTP-01 and TLS-ALPN-01, the server itself connects out to the host being validated. Loopback, link local (including the cloud metadata address), and IPv6 unique local targets are always refused; the RFC 1918 private ranges are allowed by default, because an internal CA usually issues for exactly those addresses. If everything you validate is public, set `Certus:Acme:ChallengeValidation:BlockPrivateRanges` to `true` in `settings.json`. The hardening guide explains the tradeoff and the rest of the egress settings.

Giving clients a TLS certificate they trust

ACME clients refuse to connect to an ACME server whose own TLS certificate they do not trust. The easiest fix is built into the app: on the external URL step, the setup wizard can enrol an HTTPS certificate for this server from the CA you connected, install it, and reload the service to serve it. The **Settings** page can renew that certificate later. Try these built in options first.

If you would rather supply your own certificate (for example, one from a different issuer), point Kestrel at a PFX file instead:

```
"Kestrel": {
  "Endpoints": {
    "Https": {
      "Url": "https://0.0.0.0:5001",
      "Certificate": {
        "Path": "C:\\ProgramData\\Ducks in a Row\\ducks.pfx",
        "Password": "your-pfx-password"
      }
    }
  }
}
```

For a quick lab test only, you can set `Auth:RequireHttps` to `false` and use plain HTTP on port 5000 instead.

See the Alert Configuration section of the Installation Guide for SMTP and webhook expiry notifications, or the Troubleshooting page for common issues.

2. Intended Use

What Ducks in a Row is for, the environment it is designed to run in, what it does, and the security properties you can rely on. The page also says what not to do with it, how long it is supported, and who makes it.

What it is for

Ducks in a Row lets the machines in a Windows Active Directory estate request and renew their own TLS certificates from the organisation's existing Active Directory Certificate Services (ADCS) enterprise certificate authority, using the standard ACME protocol (RFC 8555). It also reads the CA's certificate inventory, alerts administrators before certificates and certificate revocation lists expire, and lets an administrator revoke certificates within a fixed ceiling.

In ADCS terms it is a registration authority. It authenticates the requester, checks that the requester controls the names it asks for, chooses the certificate template, and only then hands the request to the CA. The CA signs what Ducks in a Row authorises, with the CA's own key and under the CA's own policy.

It can also run in monitor mode, chosen at install time, in which it never writes to the CA: no issuance and no revocation, only the inventory, the alerts and the CRL watch.

It is not a certificate authority, not a replacement for ADCS, and not a public ACME service.

The environment it assumes

- **An internal network.** ACME clients inside your estate talk to a CA inside your estate. Nothing in the product is designed for exposure to the internet, and nothing in this documentation tells you to publish it there.
- **A domain joined Windows Server.** The service runs as LocalSystem and reaches the CA and Active Directory as the server's computer account, with no stored credential. See system requirements.
- **The CA's own permissions.** What the server may do at the CA is exactly what its computer account has been granted: Request Certificates and Read on the CA, Enroll on the templates you expose, and Issue and Manage Certificates only if you want revocation. The CA enforces those rights, not the product, and monitor mode needs fewer of them. See what to ask your AD and CA team.
- **Administrators under change control.** Settings live in files that only SYSTEM and Administrators can write, and the dashboard admits only members of the administrator group. There is no read only role: everyone who can sign in can change settings, issue credentials and revoke. The product trusts those people, so who is in that group, and who can change `settings.json`, is part of your security posture. See configuration.

What it does

- **An ACME server for your CA.** Each ADCS template you choose becomes its own ACME directory. Clients prove control of a name with HTTP-01, DNS-01, TLS-ALPN-01 or `device-attest-01`, and the issued certificate comes from your CA, on your trust chain. See connecting ACME clients.

- **Control over who may ask.** External account binding gates account registration, and an allowed domain list limits which names may be ordered. See external account binding.
- **Its own HTTPS certificate.** The server enrols and renews the certificate it serves the dashboard and the ACME endpoints with. See configuration.
- **Inventory and alerts.** The CA's certificate database, synced on a schedule, and email or webhook alerts before a certificate or a CRL expires. See expiry alerts.
- **Revocation, within limits.** From the dashboard, within a capability ceiling and a scope you choose; ACME clients may revoke their own certificates. See revocation.
- **A setup wizard and a dashboard**, behind Windows Integrated Authentication, limited to the administrator group.

Security properties

In plain words, what you can rely on:

- **It validates every request before the CA sees it.** An ACME client must prove control of each name it asks for. The name must be on the allowed domain list when one is set, and inside the credential's namespace when external account binding is on. The certificate signing request may carry only a subject alternative name, a key usage and an extended key usage, and anything else is refused before submission. After issuance the certificate the CA returned is compared with the order, and one that carries a name or a capability the order did not authorise is revoked and never delivered. See hardening.
- **It never holds the CA's signing key.** The CA signs; Ducks in a Row only submits. Taking over the server gives an attacker what the server's computer account may do at the CA, and nothing more.
- **The revocation ceiling cannot be raised.** No setting, break glass flag or dashboard control lets it revoke anything but a TLS certificate. See the ceiling you cannot raise.
- **It never contacts the manufacturer or any third party.** No telemetry, no update check, no crash reporting. The only outbound connections are to systems you configure: your CA and directory, your SMTP server and webhook endpoints, the names it validates during a challenge, and the CRL distribution points it watches. It never installs updates by itself; see upgrading.
- **Its files are protected.** The data folder is writable only by SYSTEM and Administrators, stored secrets are encrypted with a keyring kept in it, and a configuration file anyone else can write is refused rather than read. See hardening.
- **Monitor mode never writes to the CA.** Every write is refused at several layers, and the CA rights it needs do not include Enroll or Issue and Manage Certificates. See monitor mode.

What not to do

Each of these is possible, because the product does not know your estate better than you do, and each opens a risk the defaults are there to close.

Do not	The risk it opens
Publish the ACME endpoint, or the dashboard, to the internet	Anyone can register an ACME account, run challenges against your names, probe the service and load the CA with requests. The product is designed for an internal network
Leave the `Certus:Acme:ExposeAllTemplates` break glass on	Every template the CA publishes becomes an ACME endpoint, including templates whose subject rules and purposes you never reviewed for ACME. The wizard's template selection is what limits what a client can ask for. The service logs the open posture as a warning at every start
Set the administrator group, `Auth:AdminGroup`, to a broad group such as Domain Users	Every member can change settings, issue and revoke credentials, revoke certificates and read the whole inventory. The dashboard has no read only role, so the group is the whole control
Run with external account binding off and no allowed domain list on an estate where ordinary users can create DNS records	Any domain account that can create a DNS record and answer a challenge for it can obtain a certificate for that name from your CA, under a template you exposed. Keep at least one of the two on; both is better
Set `Auth:RequireHttps` to `false`, except for a short lab test	Dashboard sign in and ACME traffic run over plain HTTP on port 5000, where they can be read or altered on the network
Enable `EDITF_ATTRIBUTESUBJECTALTNAME2` on the CA	The CA then honours a subject alternative name supplied in a request attribute by any requester (ESC6), which Ducks in a Row never asks for and cannot validate. Nothing in the product needs it. See hardening the CA

The hardening guide covers the CA side in more depth, and configuration lists the two combinations the service refuses to start with at all.

Support

Ducks in a Row receives security fixes until 31 December 2031, at least five years after 1.0.0 became generally available, delivered through upgrades to the latest release, with no backports.

The kinds of support, community and commercial, are in SUPPORT.md. How to report a vulnerability, when advisories are published, and what a release may ship with are in SECURITY.md. Security fixes reach you by upgrading.

The manufacturer

- **Manufacturer:** Haruspex Systems B.V., registered with the Dutch Chamber of Commerce (KVK) under number 42142878.
- **Postal address:** Wecelostraat 3, 6901 GD Zevenaar, the Netherlands.
- **Email:** licensing@haruspex.systems for licensing and general enquiries, feedback@haruspex.systems for feedback, and security@haruspex.systems to report a vulnerability, as SECURITY.md describes.
- **Website:** https://ducksinarow.dev
- **Source code and releases:** https://github.com/haruspexsystems/Ducks-in-a-Row

This page is the one place the manufacturer's details are stated; the README and the documentation index link here rather than repeat them.

3. System Requirements

What the server needs before you install. If you only want to get going, the quickstart covers the short version; this page is the full list, including the network ports and the reason each one is open. One installation serves one CA, so an estate with more than one issuing CA needs one server per CA, as several certificate authorities describes.

Server requirements

Requirement	Details
Operating system	Windows Server 2019, Server 2022 with current updates, or Server 2025. Verified on Server 2019 and Server 2025; see the Server 2022 note below
Runtime	ASP.NET Core 10.0. `Ducks-in-a-Row-Setup.exe` installs it for you, offline. Install the Hosting Bundle by hand only if you use the bare MSI
Windows feature	`RSAT-ADCS-Mgmt`, the ADCS Remote Administration Tools
Processor	Two x64 cores minimum. The work is mostly waiting on input and output, being COM and RPC calls to the CA plus SQLite, with short bursts of certificate cryptography
Memory	1 GB minimum, 2 GB recommended
Disk	About 500 MB for the application and the .NET runtime. Plan at least 5 GB free for database growth, logs and upgrades. At roughly 1000 ACME accounts renewing over three years the database reaches the low hundreds of MB, and the log takes at most 3,000 MB: 30 files of up to 100 MB each, one a day, with a full file rolling to `ducks-<date>_001.log` and so on
Network	TCP 5000 for HTTP and TCP 5001 for HTTPS, both configurable
Domain	Must be domain joined, because the CA is reached over DCOM

None of these Windows Server versions is dropped during the support period, which runs to 31 December 2031, without a published decision on how its users keep receiving security fixes, so you can plan on the version you run today.

Server 2022 needs current updates first. The .NET 10 runtime requires Control-flow Enforcement Technology on Server 2022, and an installation that has not been serviced since early 2022 does not provide it. Build 20348.587 is confirmed too old; check yours with `winver`. The service then crashes before it can write a log line, and the installer reports the misleading **Error 1920**. Server 2019 predates the requirement and Server 2025 ships with it, so neither is affected.

Installing the prerequisites

Run these in an elevated PowerShell session on the Ducks in a Row server before you install:

```
# Install the ADCS Remote Administration Tools
Install-WindowsFeature RSAT-ADCS-Mgmt

# Verify the feature is installed
Get-WindowsFeature RSAT-ADCS-Mgmt

# Verify the .NET 10 runtime is present, after the setup bundle
# or a manual Hosting Bundle install
dotnet --list-runtimes
```

Why RSAT-ADCS-Mgmt is required. Ducks in a Row talks to the certification authority through the ADCS COM classes, and it is the Remote Server Administration Tools feature that registers those classes on the system. Without it the COM objects cannot be created at all, and the service fails with `DISP_E_MEMBERNOTFOUND` (0x80020003) during certificate sync and CA queries. See troubleshooting if you are reading that error now.

ADCS requirements

To issue certificates through ADCS, all of the following must be in place:

- An Active Directory Certificate Services enterprise certification authority.
- The ADCS Remote Administration Tools (`RSAT-ADCS-Mgmt`) installed on the Ducks in a Row server, as above.
- DCOM and RPC network access from the Ducks in a Row server to the CA server, on the ports under network ports.
- The server's machine account admitted by the CA server's **Certificate Service DCOM Access** group, which is what lets an ordinary account call the CA over DCOM from the network. On a default CA the group holds Authenticated Users, so this is usually already in place.
- The server's machine account granted **Enroll** on each certificate template you want to expose over ACME.
- The server's machine account granted **Request Certificates** on the CA itself. Authenticated Users hold it on a default CA, so this too is usually already in place.
- The server's machine account granted **Read** on the CA itself. Without it enrolment still works, but the dashboard inventory stays empty.
- Certificate templates published to Active Directory.

The service runs as LocalSystem, so the account that needs those rights is the server's machine account, written `DOMAIN\SERVERNAME$`. Revocation additionally needs **Issue and Manage Certificates** on the CA, whether it comes from the dashboard, from an ACME client, or from the automatic revocation at finalize; see revocation for what that does and does not allow. What to ask your AD and CA team lists all of these, with the firewall openings, in a form you can forward to whoever grants them.

CA keys in an HSM

Ducks in a Row never handles the CA's private key. Every call it makes goes to the certification authority service on the CA server over DCOM, through the ADCS request and administration interfaces: it submits a request, collects the certificate the CA signed, reads the CA database, downloads the CA's own certificate

chain and asks for a revocation. Where the CA keeps its key, in the software key storage provider Windows installs by default, in a hardware security module (HSM) or in any other key storage provider or cryptographic service provider, is a CA setting the service cannot see and never asks about. Nothing on the Ducks in a Row side is configured for it.

The one place an HSM can show is how long the CA takes to answer a submission, because the HSM signs each certificate. That time is spent inside the CA call the service makes while it answers an ACME client's finalize request, so the client sees a slightly slower finalize response and nothing else; the order status it reads afterwards is the same as with any other CA.

What has been verified: issuing, revoking and inventory sync against CAs whose key sits in the default software key storage provider, on Server 2019 and Server 2025. What has not: any CA whose key sits in an HSM, and any provider other than the default. Nothing in the design depends on the provider, but until a CA with such a key has been measured, the honest statement is that it has not been verified.

Network ports

Direction	Port	Protocol	Purpose
Inbound to the server	5001	TCP	ACME clients and dashboard browsers, over HTTPS
Inbound to the server	5000	TCP	Optional. With the default settings it only redirects to HTTPS on 5001
Outbound to the CA	135	TCP	DCOM and RPC endpoint mapper
Outbound to the CA	49152-65535	TCP	DCOM and RPC dynamic ports
Outbound to targets	80	TCP	HTTP-01 challenge validation
Outbound to DNS	53	TCP and UDP	DNS-01 challenge validation
Outbound to targets	443	TCP	TLS-ALPN-01 challenge validation
Outbound to CRL distribution points	80, 389, 443	TCP	Optional. CRL expiry alerts, over HTTP and LDAP, and HTTPS for any https location
Outbound to the mail server	587 by default	TCP	Optional. Email alerts over SMTP
Outbound to webhook endpoints	As configured	TCP	Optional. Webhook alerts

The installer opens 5000 and 5001 in this server's own firewall and nothing upstream of it.

The three outbound challenge rows are needed only for the challenge types you actually use. The server is the party that performs validation, so it needs to reach the name being validated, not the other way round.

The hardening guide covers narrowing that egress.

`device-attest-01` needs no outbound rule at all. The device presents its attestation in the challenge response and the server verifies it against a trust anchor it already holds, so nothing is fetched. See device attestation.

4. What to Ask Your AD and CA Team

Ducks in a Row is a Windows service that sends certificate requests to your existing Active Directory Certificate Services CA as its own computer account, the same way any domain member enrols. It holds none of the CA's keys and needs no domain administrator rights, no schema change, and no service account or password.

This page lists everything the server needs from the people who run Active Directory, the CA and the network, so you can forward it to them before you install. Setup takes about five minutes of clicking once these are in place, and the approvals are usually the lead time, so ask for them first. The change request at the end is ready to copy.

The account the CA sees

The service runs as LocalSystem. On the network LocalSystem acts as the server's computer account, so every right on this page is granted to that account, written `DOMAIN\SERVERNAME$`. For a server named `DUCKS-SERVER` in the `CORP` domain that is `CORP\DUCKS-SERVER$`. No person's account is involved, and there is no password to store, rotate or leak.

Computer accounts do not appear in the Windows object picker until you add **Computers** to the object types.

Granting to a group

You can grant each right to a group that holds the computer account rather than to the account itself. If the server is ever rebuilt under a new computer account, you then change one group membership instead of every grant. Use a group of its own for this, not Domain Computers: a right held by every computer in the domain is the kind of broad grant the hardening guide warns about.

A computer picks up a new group membership only when its Kerberos tickets are renewed, which in practice means a restart. After adding the server's account to a group, restart the server, or purge the system's tickets and restart the service:

```
klist -li 0x3e7 purge
Restart-Service DucksInARow
```

Until then the CA still sees the old membership. The setup wizard's rights check reads group membership from the directory, so a template's Enroll row can read Inferred through the new group while the CA is still refusing it, and the rows the CA answers itself keep failing until the restart.

What to ask for

Each row is one thing to ask for. The rights are named exactly as they appear on the Security tabs of the Windows consoles, so whoever grants them can find them.

What	Why it is needed	Who usually grants it and where	Needed when
Enroll on each certificate template you expose over ACME	The CA checks it on every request. The server's own HTTPS certificate comes from the first template you choose at setup and is renewed from it, so keep Enroll on that one even if you later stop exposing it	The template owner or CA administrator, in the Certificate Templates console (certtmpl.msc), the template's Properties, Security tab	Always, once per template
Request Certificates on the CA	Submitting requests, the wizard's Test Connection, the template list, downloading the CA's certificates, CRL watching, and collecting a certificate the CA held for approval	The CA administrator, in the Certification Authority console (certsrv.msc), the CA's Properties, Security tab	Always. Authenticated Users hold it on a default CA, so ask only that nobody removed it
Read on the CA	The certificate inventory and the expiry alerts built on it. Read alone opens the whole certificate database, and neither this right nor Request Certificates implies the other	The CA administrator, on the same Security tab	Always. Without it issuance works and the dashboard stays empty
Issue and Manage Certificates on the CA (optional)	Every revocation the product makes: from the dashboard, by an ACME client revoking its own certificate, and the automatic revocation of a certificate that breaks the capability ceiling. It also lets the account approve and deny requests the CA holds	The CA administrator, on the same Security tab	Only if you want revocation. See below for what is lost without it
Membership of the CA server's **Certificate Service DCOM Access** group	The CA is called over DCOM, and on the CA server this group is what lets an ordinary account activate it from the network. On a default CA, Authenticated Users is a member, which covers every domain computer	The CA server's administrator, in Computer Management, Local Users and Groups	Always. Ask only that nobody removed Authenticated Users
The ADCS Remote Administration Tools (RSAT-ADCS-Mgmt) on the Ducks in a Row server	They register the ADCS COM classes the service uses to talk to the CA	Whoever administers the Ducks in a Row server	Always, before setup

What	Why it is needed	Who usually grants it and where	Needed when
Domain membership in the CA's forest	The CA authenticates the server's computer account through Active Directory, and the service reads certificate templates from the forest's directory	A domain administrator, or an account delegated to join computers	Always
TCP 135 and TCP 49152 to 65535 from the Ducks in a Row server to the CA	DCOM and RPC: the endpoint mapper first, then a dynamic port	The network team, and the CA server's own firewall	Always
TCP 5001 from ACME clients and administrators to the Ducks in a Row server, and TCP 5000 if you want it	5001 serves ACME and the dashboard over HTTPS. With the default settings 5000 only redirects to 5001. The installer opens both in this server's own firewall, and nothing upstream	The network team	5001 always. 5000 only for people who type an http address
TCP 80 and TCP 443 from the Ducks in a Row server to the names being validated, and DNS to the server's own DNS servers	The server performs ACME validation itself: TCP 80 for HTTP-01, TCP 443 for TLS-ALPN-01, and for DNS-01 its own DNS servers on port 53, TCP and UDP, which must be able to resolve the names being validated	The network team	Only for the challenge types your clients use. None for device-attest-01
HTTP (TCP 80), LDAP (TCP 389), and HTTPS (TCP 443) for any https location, from the Ducks in a Row server to the CRL distribution points	CRL expiry alerts read every published copy of each CRL. A parent CA's CRL, such as an offline root's, is read only from these locations	The network team, and whoever runs the web server that publishes your CRLs	Optional. Without it the connected CA's own CRLs are still read from the CA
SMTP (TCP 587 by default) and your webhook endpoints, from the Ducks in a Row server	Expiry alerts by email and by webhook	The mail and network teams	Only when alerts are configured
A group for the dashboard's administrators (optional)	Auth:AdminGroup names the Windows or Active Directory group whose members may sign in to the dashboard. Left unset, it is this server's local Administrators group	An Active Directory administrator	Optional

Install the tools from an elevated PowerShell session on the Ducks in a Row server. They are needed even though this server is not the CA:

```
Install-WindowsFeature RSAT-ADCS-Mgmt
```

Without Issue and Manage Certificates, issuance, the inventory and alerts keep working, and every revocation fails with a permission error naming the right. That includes an ACME client revoking a certificate whose key it believes is compromised, and the automatic revocation at finalize: a certificate that breaks the ceiling then stays valid at the CA, and the service log names its serial number and asks for it to be revoked by hand. The client still never receives it. See what the CA has to allow before you decide.

A CA installed on a domain controller. The defaults above were checked on CAs installed on member servers. A domain controller has no local groups, so there the Certificate Service DCOM Access group is the domain's group of that name, in the Builtin container of Active Directory Users and Computers. Check its members rather than assuming.

What is not being asked for

- No **Autoenroll** on any template. Nothing here autoenrols.
- No **Write** or **Full Control** on a template, and no change to any template.
- No **Manage CA**, and no change to the CA's settings or flags. In particular, never `EDITF_ATTRIBUTESUBJECTALTNAME2`: the product does not need it, and it lets any requester add any subject alternative name on every template.
- **Issue and Manage Certificates** only if you want revocation through Ducks in a Row, the automatic revocation at finalize included.
- In monitor mode, no **Enroll** and no **Issue and Manage Certificates** at all. Read on the CA, plus the Request Certificates a default CA already grants, is the whole request, which makes it the easier first ask on a production CA.
- No rights in Active Directory. The service reads certificate templates, the CA's enrollment record and its own group memberships with the read access every domain computer already has.
- No domain administrator, no schema extension, and no service account.

The change request

Fill in the three placeholders and delete the lines you do not need.

```
Change request: rights for the Ducks in a Row certificate service

Ducks in a Row is a Windows service that sends certificate requests to
our CA as its own computer account. It holds no CA keys and needs no
domain administrator rights, no schema change and no service account.

Computer account: <COMPUTER ACCOUNT> (for example CORP\DUCKS-SERVER$)
CA: <CA>
Templates: <TEMPLATES>

Please grant, to the computer account or to a group that holds it:
```

```
1. Enroll, and not Autoenroll, on each template listed above.
   Certificate Templates console, template Properties, Security tab.
2. Read on the CA.
   Certification Authority console, CA Properties, Security tab.
3. Only if we revoke through Ducks in a Row, the automatic revocation at
   finalize included: Issue and Manage Certificates on the CA, on the same
   tab.

Please confirm these defaults are still in place:

4. Request Certificates on the CA, held by Authenticated Users or
   granted to the computer account.
5. Authenticated Users, or the computer account, in the CA server's
   Certificate Service DCOM Access group.

Network:

6. TCP 135 and TCP 49152 to 65535 from the Ducks in a Row server to
   the CA.
7. TCP 5001 from ACME clients and administrators to the Ducks in a Row
   server.
8. Only for what we use: TCP 80 and 443 to the names being validated,
   DNS to our own DNS servers, the CRL distribution points (TCP 80,
   389, and 443 for an https location), SMTP, webhooks.

Not requested: Autoenroll, Write or Full Control on any template, Manage
CA, any change to the CA's flags, any right in Active Directory.

If you grant through a group, please tell us: the server needs a
restart before the CA sees the new membership.

How we will confirm it: the setup wizard checks the service's own
rights after Test Connection and again on its Review step, and the
"Service rights on the CA" card on the Settings page repeats the check.

Rollback: remove the grants above. Nothing else on the CA or in the
directory is changed.
```

What each status in that check means, and how each right is established, is in the service's rights.

5. Installation

Two ways to install. Use the setup bundle unless you have a reason not to.

Installer	What it does	When to use it
`Ducks-in-a-Row-Setup.exe`	Installs or updates the ASP.NET Core 10 runtime when the server's is missing or older, then runs the MSI	Almost always, including on servers with no internet access
`Ducks-in-a-Row.msi`	The application only. Fails to start if the runtime is absent	Deployment systems that manage the runtime themselves

The setup bundle carries the runtime inside it, so an offline server installs from the one file. It installs that runtime, as the full ASP.NET Core Hosting Bundle with the ASP.NET Core Module for IIS, when the server has no ASP.NET Core 10 runtime or only an older update of it, which it replaces, and skips it when the server already has the same update or a later one. It never removes the runtime on uninstall, because other applications on the server may be using it.

Two things to know when the bundle installs or updates the runtime:

- **ASP.NET Core sites in IIS restart.** On a server that already has the runtime, IIS itself keeps running, but the worker processes that host ASP.NET Core sites are stopped while the ASP.NET Core Module is replaced, and start again on their next request. Run the setup in a quiet period on such a server.
- **The setup never restarts the server for the runtime.** The runtime installer can ask for a restart, for example when a file it replaces is in use. The setup then carries on, installs Ducks in a Row, and finishes as a success (exit code 0) with no restart prompt. The only sign is in the setup log: the line for the `AspNetCoreHostingBundle` package says `exited with code: 0xbc2` (3010). If you find it, restart the server when it suits you, so the runtime update is complete.

The bare MSI does not install the runtime. Without it the service fails before any managed code runs, and the installer reports **Error 1920**, which almost never means what it says. See troubleshooting.

Check the download

Both installers are signed by Haruspex Systems B.V. and timestamped. Check a download before you run it. The same commands work on the MSI.

```
$signature = Get-AuthenticodeSignature .\Ducks-in-a-Row-Setup.exe
$signature.Status # Valid
$signature.SignerCertificate.Subject # CN=Haruspex Systems B.V., O=Haruspex Systems B.V., ...
$signature.TimeStamperCertificate.Subject # a Certum timestamp authority
```

The status must be `Valid`, the signer must be Haruspex Systems B.V., and the timestamp must be there: it is what keeps the signature valid after the signing certificate itself expires. Anything else, such as `NotSigned`, `HashMismatch`, or a different signer, means the file is not the one we published. Do not run it.

Compare the checksum too, against the `SHA256SUMS` file on the release page:

```
Get-FileHash .\Ducks-in-a-Row-Setup.exe -Algorithm SHA256
```

When you run a signed installer, the Windows permission prompt names Haruspex Systems B.V. as the verified publisher. Windows SmartScreen may still warn about a new release while the signing certificate builds up a reputation with Microsoft. If it does, **More info** should name the same publisher. If it names anyone else, or an unknown publisher, do not run the file.

Every `.exe` and `.dll` the installer puts in the application folder carries the same signature, so an App Control for Business (WDAC) or AppLocker policy can allow everything it installs with one publisher rule. The setup program itself also runs a few helpers from a temporary folder while it works; those are signed by their own publishers, the WiX Toolset and Microsoft.

Releases up to and including 0.10.0-beta.1 were not signed. For those, the checksum is the only check.

What the installer does

Action	Details
Install the application	`C:\Program Files\Ducks in a Row\`, or the folder you choose
Create the data folder	`C:\ProgramData\Ducks in a Row\`, or the folder you choose. Its location is recorded in the registry so the service can find it
Create the log folder	`logs\` inside the data folder
Register the service	`DucksInARow`, display name "Ducks in a Row Certificate Proxy", starts automatically, runs as LocalSystem
Set service recovery	Restart after the first and second failures, nothing after the third, failure count reset after a day. See if the service stops
Start the service	Immediately after install, unless you clear the option
Record the instance mode	The registry value `Mode`, `monitor` or `full`, beside the data folder's location. See monitor mode
Create the modules folder	`modules\` inside the application folder, empty, with the application folder's permissions. The paid module installs into it
Record the module contract	The registry value `ModuleContractVersion`, for example `1.0`, beside the data folder's location. The paid module's installer reads it to check that the module fits this release
Open firewall rules	Inbound TCP 5000 and TCP 5001

Action	Details
Add a Start Menu shortcut	Opens the web interface

The data folder is not removed when you uninstall. See uninstalling.

The install wizard

Running either installer interactively gives you five choices:

1. The destination folder, default `C:\Program Files\Ducks in a Row\`.
2. The data folder for the database, logs and runtime configuration, default `C:\ProgramData\Ducks in a Row\`.
3. Whether to start the service immediately.
4. Whether to run in monitor mode: watch the CA, never issue or revoke. Unchecked by default. It is the low risk way to start on a production CA, and switching to issuance later needs no reinstall.
5. Whether to open the setup page in your browser from the final screen.

Choose the data folder now. Moving it during a later upgrade is not supported. It holds the database too, which cannot be moved out of it on its own, so if the data belongs on another disk, choose a folder on that disk here. Use a local disk: a network share is not supported.

An upgrade keeps the folders and the mode chosen before, and on a monitor mode server the monitor mode box is already ticked. If someone set the registry value `Mode` by hand to something the service cannot use, setup stops and names it; see troubleshooting.

If the server has the paid module installed, and the new release needs a module built for it, the wizard opens with a warning page before anything else. After such an upgrade the installed module does not load, so the paid features stay off until you install the module built for the new release. Certificate issuance and renewal are not affected. **Continue** upgrades anyway; **Cancel** leaves the installed version as it is.

Unattended install

```
msiexec /i Ducks-in-a-Row.msi /qn `
  INSTALLFOLDER="D:\Ducks in a Row" `
  DATAFOLDER="D:\DucksData" `
  START_SERVICE=0 `
  /l*v "C:\temp\ducks-install.log"
```

Property	Default	Meaning
`INSTALLFOLDER`	`C:\Program Files\Ducks in a Row`	Where the application goes

Property	Default	Meaning
`DATAFOLDER`	`C:\ProgramData\Ducks in a Row`	Where the database, logs and settings go
`START_SERVICE`	`1`	`0` registers the service but leaves it stopped until the next boot or a manual start
`MONITOR_ONLY`	not set	`1` installs in monitor mode. Leave it out for full mode; on an upgrade, leaving it out keeps the mode the server already has. Any other value is refused
`ALLOW_INCOMPATIBLE_MODULE`	not set	`1` lets an unattended upgrade go ahead although the installed paid module will not load after it. Without it such an upgrade stops and says why. It is never remembered, and any other value is refused

The prompt to open the setup page appears only in the interactive wizard, so a `/qn` install never launches a browser. The same goes for the paid module warning page: an install with no wizard, which includes the setup bundle's `/quiet` and `/passive`, stops instead of asking, unless you pass `ALLOW_INCOMPATIBLE_MODULE=1`.

The setup bundle passes the same five properties to the MSI. Give them on its own command line:

```
Ducks-in-a-Row-Setup.exe /quiet /norestart MONITOR_ONLY=1 START_SERVICE=0 /log
"C:\temp\ducks-bundle.log"
```

Keep `/norestart` on unattended runs of the setup bundle. The runtime update never restarts the server, but the Ducks in a Row installer itself can still ask for a restart if a file it replaces stays in use. With `/norestart` the setup then exits with code 3010 and you restart when it suits you; without it, `/quiet` and `/passive` restart the server for that. Exit code 0 means done.

If the service stops

The installer sets the service to recover by itself, twice:

Failure	What Windows does
First	Restarts the service after 30 seconds
Second	Restarts the service after 30 seconds
Third and later	Nothing: the service stays stopped

Windows forgets the count once a day passes with no failure. Check what is set with:

```
sc.exe qfailure DucksInARow
```

Why two restarts. The paid module runs inside the service. A fault in it can end the service's process outright, and the service notes each time that happens while the module is starting up. After the second time it sets that version of the module aside, and starts without it from then on. So two restarts bring the service back with certificate issuance and renewal working, whatever the module did. The paid features stay off until the module is fixed, and the licence page says so.

Why not more. Almost every other way this service stops is a fault it found while starting, usually a configuration it cannot work with. It writes the reason to the log and stops. A restart does not fix a configuration, so a third restart would only have it reach the same fault again, and you would be left watching it flap rather than seeing it plainly down with the reason in the log.

A configuration the service refuses at startup is not restarted at all. Windows records it as a service that failed to start (System log events 7009 and 7000 from the Service Control Manager) rather than one that ended unexpectedly (7031), and runs no recovery action for it. The service stays stopped, with the reason in its log. This was measured with a `Mode` value the service refuses, not reasoned about.

The failure flag stays off. Windows runs these actions when a service ends without reporting that it stopped. A service that does report stopped, with an error code, counts as a failure only when the failure flag is on, and the installer leaves it off.

Every install, repair and upgrade sets these actions again, and replaces any you set yourself with `sc.exe failure`. None of this is a substitute for reading the log. The reason the service stopped is in `logs\ducks-<date>.log` in the data folder.

Releases before 1.1 configured no recovery at all.

After installing

The service serves the setup wizard until you complete it, and every CA operation answers 503 until then. Open `https://your-server:5001` and work through the wizard; the quickstart walks through each step.

Then confirm the install is healthy with verifying your installation.

Upgrading

Ducks in a Row never checks for a new release and never installs one by itself. It makes no connection to us or to anyone else, so there is no automatic update setting to turn off. You learn that a release exists from the releases page, the repository's security advisories, the changelog, or the security notices list you can sign up for; SUPPORT.md says how. A security fix ships as its own patch release of the latest release where that is feasible, and upgrading to it is free of charge; see SECURITY.md.

To upgrade:

1. Take a copy of the data folder first, as backup and restore describes. An upgrade keeps it, and the copy is what you fall back on if anything goes wrong.
2. Check the download: the signature, the signer and the checksum, exactly as for a first install.

3. Run the new `Ducks-in-a-Row-Setup.exe` on the server, over the existing installation, interactively or unattended with the same command as a fresh install. The bundle first brings the ASP.NET Core runtime up to the update it carries when the server's is missing or older; the start of this page says what that means for IIS sites and a restart. The bare MSI never does, so with the MSI, install the runtime the release notes name before you run it. The wizard shows the folders chosen before and the monitor mode box as the server has it, so nothing has to be chosen again; leaving `MONITOR_ONLY` out of an unattended upgrade keeps the mode. Moving the data folder during an upgrade is not supported.
4. If the server has the paid module installed and the new release needs a module built for it, the wizard opens with a warning page first, and an unattended upgrade stops unless you pass `ALLOW_INCOMPATIBLE_MODULE=1`. See the install wizard for what the warning means; certificate issuance and renewal are not affected either way.
5. The installer stops the service, removes the previous version, installs the new one, and starts the service again. ACME clients and the dashboard lose their connection while that runs, and a client retries on its next renewal attempt. An older release cannot be installed over a newer one; the installer refuses with "A newer version of Ducks in a Row is already installed".

What an upgrade keeps:

- The data folder, with the database, the logs, `settings.json`, `ducks-setup.json` and the Data Protection keyring in `keys\`. Your CA connection, templates, policies, ACME accounts, alert settings and history all live there.
- The install folder and the data folder you chose, and the instance mode.
- The server's HTTPS certificate in the machine store, and the thumbprint in `settings.json` that names it.
- The firewall rules, the service registration and its recovery actions, which the installer sets again.
- A paid module in the `modules\` folder, which loads again only if it was built for the new release.

Afterwards, run verifying your installation: the first check confirms the installed files carry our signature, and the health check that the service is up. Then read the changelog entry for the release, and `logs\ducks-<date>.log` once: a setting the new release finds wrong is reported there as a startup warning that names it.

6. Monitor Mode

Monitor mode is Ducks in a Row with issuance switched off. The service watches your certificate authority and never writes to it: it keeps the certificate inventory, sends expiry alerts and watches the revocation lists, and it never submits a certificate request and never revokes one. It is the low risk first step onto a production CA. The CA team grants a read right, you see every certificate the CA has issued and when each one expires, and turning issuance on later is a rights grant, a registry value and a restart, never a reinstall. Monitor mode is part of the free tier, on a production CA too. An installation watches one CA; for a second issuing CA, such as one in an operational technology zone, see several certificate authorities.

You choose it when you install. A badge on every page of the dashboard says "Monitor mode: never issues or revokes" and links to a short summary on ducksinarow.dev.

What it does

- **The certificate inventory.** Every certificate the CA has issued, synced from the CA's database, searchable on the dashboard, with its requester, template and expiry.
- **Expiry alerts.** Email or a webhook before a certificate expires, at the thresholds you set. See expiry alerts.
- **CRL watching.** A warning before a revocation list goes stale, or when the CA fails to publish one.
- **The Settings page**, including the service rights check, which marks every right only issuance or revocation needs as "Not needed in monitor mode".

What it never does

- **It never submits a certificate request.** ACME is off: every ACME URL, and the ACME administration API under `/api/acme`, answers 404, and the ACME page is hidden.
- **It never revokes a certificate.** The revoke button is hidden, and the endpoint behind it refuses.
- **It never enrols its own HTTPS certificate.** It serves one already in the machine store, or its own self signed one. See the HTTPS certificate.
- **It never writes to Active Directory.** Neither does full mode.

The service refuses each of these at several layers, ending at its CA client, which refuses both CA writes whoever asks. Treat that as defence in depth rather than as the control: the rights the CA team grants are what hold, whatever the software does. Hardening explains why.

What it needs

Requirement	Where	Why
Read on the CA	The CA's Security tab, for this server's computer account	The certificate inventory reads the CA's database

Requirement	Where	Why
Request Certificates on the CA	The CA's Security tab	Every read of the CA's own properties goes through it: Test Connection, the CA's certificates and CRL watching. Authenticated Users hold it on a default CA, so it is usually already in place
DCOM access to the CA	The CA server's Certificate Service DCOM Access group	Every call to the CA. Authenticated Users are members by default
The ADCS management tools on this server	`Install-WindowsFeature RSAT-ADCS-Mgmt`	The client components every CA call uses
Domain membership	This server	The CA authenticates this server's computer account
TCP 135 and the dynamic RPC range (49152 to 65535) to the CA	The network	DCOM to the CA
Outbound LDAP (TCP 389) to a domain controller, and HTTP (TCP 80) to the CRL distribution points	The network	CRL watching reads the lists where clients read them

Monitor mode never needs **Enroll** on a template, and never needs **Issue and Manage Certificates**. On most CAs Read is the only right to ask for. What to ask your AD and CA team has the full table and a change request ready to copy.

Choosing it at install

In the install wizard, the **Service options** screen has a checkbox: "Monitor only: watch the CA, never issue or revoke". It is unchecked by default.

Unattended, set `MONITOR_ONLY=1`, on the MSI or on the setup bundle:

```
msiexec /i Ducks-in-a-Row.msi /qn MONITOR_ONLY=1 /l*v "C:\temp\ducks-install.log"
Ducks-in-a-Row-Setup.exe /quiet /norestart MONITOR_ONLY=1 /log "C:\temp\ducks-bundle.log"
```

The installer records the choice as the registry value `Mode`, a `REG_SZ` of `monitor` or `full`, beside the data folder's location. The service reads it once at startup. Configuration describes the value.

- **An upgrade or a repair keeps the mode.** On an interactive upgrade the box is already ticked on a monitor mode server; clearing it switches the server to full mode. `MONITOR_ONLY=1` on an upgrade switches a full mode server to monitor mode. Leaving it out keeps whichever mode the server has.
- `MONITOR_ONLY` **takes the value** `1` **and nothing else.** Any other value stops setup with a message saying so.

- **A `Mode` value the service cannot use stops setup.** If someone set the value by hand to something other than `monitor` or `full`, or as a number (`REG_DWORD`), setup stops and names the value rather than overwriting it. Set it to `monitor` or `full`, or delete it for full mode, and run setup again. An uninstall always works whatever the value says.
- **Uninstalling removes the value.** Installing again without `MONITOR_ONLY=1` comes back in full mode. The data folder, and the setup it records, are kept.

The setup wizard in monitor mode

The wizard has four steps instead of six: Welcome, CA Connection, External URL and Review. There is no Templates step and no Allowed Domains step, because nothing is issued.

- **CA Connection** names the rights monitor mode needs, and the rights check marks the ones it does not as "Not needed in monitor mode".
- **External URL** is the address you open the dashboard at. Once the URL is validated, the step offers the HTTPS certificate choice described below.
- **Review** shows the CA, the URL and an Alerts card, and completing setup records no template.

When setup completes, **Set up alerts** takes you straight to the alert settings.

Alerts are the product

In monitor mode, alerts are how an expiring certificate or a stale revocation list reaches anyone, and nothing is sent until a channel is set up:

- **Email** is configured on the Settings page, the SMTP password included, which is stored protected.
- **A webhook** is configured under `Certus:Alerts:Webhook` in `appsettings.json` or `settings.json`, and is never configurable from the dashboard.

While alerting is switched off, or neither channel can deliver, every page of the dashboard shows a warning with a link to the alert settings. See expiry alerts.

The HTTPS certificate

Monitor mode never requests a certificate from your CA, its own included. The External URL step therefore offers the certificates already in this server's machine store (`LocalMachine\My`) that the service could serve, with the ones covering the URL's host first, and the choice to keep the self signed certificate the service generates. A certificate is offered only when the service could use it: valid now, issued for server authentication (or for any purpose), not a CA certificate, and holding a private key the service can actually sign with. A certificate for another name needs a confirmation. Choosing one restarts the service to serve it, and choosing **Go back to the self signed certificate** undoes an earlier choice.

The usual choice is a certificate Windows autoenrolment keeps for the server, for example from your Computer template, or one your CA team issued for the URL.

Following a renewal. The service records the certificate by thumbprint, and a renewal issues a new thumbprint. At every start, if the recorded certificate has been archived, which is what a renewal does to the certificate it replaces, or has expired, the service serves the newest certificate issued after it to the same subject from the same template, provided it is valid and holds its private key. It records the new thumbprint in `settings.json` and writes a warning in the service log saying what it did.

Three limits:

- **The choice is made at startup.** A service that runs across the old certificate's expiry keeps serving it until it restarts. Windows renews well before expiry, so any restart between the renewal and the old certificate's expiry is enough.
- **It needs the old certificate in the store.** A renewal archives it, which is enough. If someone deleted it, there is nothing to match against, and the service falls back to its self signed certificate and says so in the log.
- **It needs a template.** A certificate from a public CA, or imported from a file, names no template, so no successor can be matched to it.

In either of the last two cases, put the new certificate's thumbprint in `Certus:HttpsCertificateThumbprint` in `settings.json`, from an elevated session, and restart the service. That is also how to change the certificate after setup.

Switching to issuance

1. Ask the CA team to grant this server's computer account **Enroll** on the template ACME clients will use, and **Issue and Manage Certificates** on the CA if Ducks in a Row should revoke, the automatic revocation at finalize included. Request Certificates and Read are already in place.
2. From an elevated PowerShell session, set the mode to full and restart:

```
New-ItemProperty -Path 'HKLM:\SOFTWARE\Haruspex Systems\Ducks in a Row' -Name Mode -Value
full -PropertyType String -Force
Restart-Service DucksInARow
```

3. Open the dashboard. It sends you to the setup wizard, reopened at the Templates step with your CA already filled in. Choose the template, then the allowed domains, check the External URL, and complete setup. Until you do, ACME exposes no template and the dashboard keeps returning to the wizard.

In full mode the service renews the HTTPS certificates it enrols itself, which carry the friendly name **Ducks in a Row HTTPS** in the machine store. A certificate you chose in monitor mode was installed by someone else, so the service leaves it alone: Windows autoenrolment, or whoever issued it, keeps renewing it, and the service serves the renewed certificate from its next start, as following a renewal describes. Nothing in Ducks in a Row renews it, so it also gets the ordinary expiry alerts. The Settings page says which of the two installed the certificate in use.

To hand its renewal to the service instead, press **Renew certificate** in the HTTPS certificate section of the Settings page. The service enrols a certificate of its own from your CA and renews that one from then on. The certificate you chose stays in the machine store.

Switching back to monitor mode

1. Set the value back to `monitor` and restart:

```
New-ItemProperty -Path 'HKLM:\SOFTWARE\Haruspex Systems\Ducks in a Row' -Name Mode -Value
monitor -PropertyType String -Force
Restart-Service DucksInARow
```

2. Ask the CA team to remove **Enroll** and **Issue and Manage Certificates**, which monitor mode never uses. Removing them is what makes the switch hold whatever the software does.

The templates setup recorded are kept but not used. An ACME order the CA was holding for approval at the switch stays as it is, and the first check after a switch back to full mode finishes it.

7. Several Certificate Authorities

One installation of Ducks in a Row connects to one certificate authority. That is the free product as released, and licence and paid module says so. An estate that has been tidied up often has more than one issuing CA under an offline root: a root plus two issuing CAs is a common shape, and a utility or a manufacturer often has one issuing CA for the office network and one for the operational network. This page says what the supported arrangement is today, how the installations relate to each other, and what you do not get from it.

The short version: **one installation per issuing CA**, each on its own server, with its own data folder, database, settings and dashboard. The two never talk to each other. Everything you configure once in a single CA estate, you configure once per installation here.

One installation per issuing CA

The installer registers one service per server, `DucksInARow`, with one data folder whose location it records in the registry; see what the installer does. So an installation per issuing CA is a server per issuing CA. A virtual machine is fine, and the system requirements are the same for each.

Each installation is complete on its own, and nothing is shared:

- **Its own data folder and database.** `ducks.db`, `settings.json`, `ducks-setup.json`, the Data Protection keyring, the logs and the licence folder all live in that server's data folder, as backup and restore lists them. Back up each installation separately.
- **Its own CA connection.** Each installation's setup wizard names one CA, and that is the CA it syncs the inventory from, issues from and revokes on.
- **Its own dashboard**, at that server's external URL, with its own administrator group.

Some things are the same on both and have to be entered twice:

- **Alert recipients.** The SMTP settings on each Settings page, and the webhook block in each `settings.json`. See expiry alerts.
- **Allowed domains, templates and revocation scope.** Each installation's setup wizard records its own `ducks-setup.json`.
- **External account binding credentials.** A credential is issued by one installation and registers an account there only. See external account binding.
- **The ACME directory URL.** The URL names the server and the template, so each installation has its own set: An ACME client enrols from whichever CA its directory URL points at. A machine that needs a certificate from each CA runs two client configurations, one per URL. See connecting ACME clients.

```
https://ducks-it.corp.example.com:5001/acme/WebServer/directory
https://ducks-ot.corp.example.com:5001/acme/WebServer/directory
```

The OT pattern

The common reason for a second issuing CA is a separate operational technology zone with a CA of its own, where nothing may write to the CA without a change request. The supported shape is an installation in the

OT zone, in monitor mode, against the OT issuing CA, while the IT installation issues from the IT CA.

Monitor mode is Ducks in a Row with issuance switched off. In the words of the monitor mode page: the service watches your certificate authority and never writes to it. It keeps the certificate inventory, sends expiry alerts and watches the revocation lists, and it never submits a certificate request and never revokes one. Monitor mode is part of the free tier, on a production CA too, so the OT installation costs nothing extra.

What the OT installation needs

The rights and openings below are copied from what monitor mode needs. None of them is a right only issuance or revocation needs, and nothing reaches the IT CA.

Requirement	Where	Why
Read on the CA	The CA's Security tab, for this server's computer account	The certificate inventory reads the CA's database
Request Certificates on the CA	The CA's Security tab	Every read of the CA's own properties goes through it: Test Connection, the CA's certificates and CRL watching. Authenticated Users hold it on a default CA, so it is usually already in place
DCOM access to the CA	The CA server's Certificate Service DCOM Access group	Every call to the CA. Authenticated Users are members by default
The ADCS management tools on this server	`Install-WindowsFeature RSAT-ADCS-Mgmt`	The client components every CA call uses
Domain membership	This server	The CA authenticates this server's computer account
TCP 135 and the dynamic RPC range (49152 to 65535) to the CA	The network	DCOM to the CA
Outbound LDAP (TCP 389) to a domain controller, and HTTP (TCP 80) to the CRL distribution points	The network	CRL watching reads the lists where clients read them

Monitor mode never needs **Enroll** on a template, and never needs **Issue and Manage Certificates**. What to ask your AD and CA team puts it the same way: in monitor mode, Read on the CA, plus the Request Certificates a default CA already grants, is the whole request.

So the firewall openings for the OT installation run from the OT server to the OT CA, to the OT CA's CRL distribution points, and to a domain controller for the LDAP locations. A CRL distribution point served over https also needs TCP 443, which the network ports table lists. Nothing is opened from the OT zone to the IT CA, and nothing from the IT installation to the OT CA. Add to that only what every installation has: TCP 5001

inbound for the dashboard, and SMTP or the webhook endpoint when alerts are configured, both from the same ports table. The three outbound challenge validation rows in that table are not needed, because monitor mode never validates a challenge.

If the OT CA later needs issuance, switching to issuance is a rights grant, a registry value and a restart, never a reinstall.

The shared root's revocation list

Both issuing CAs chain to the same offline root, and each installation watches the revocation lists of every CA in the chain of the CA it is connected to, the root included. The root's CRL is read from the distribution points named on the connected CA's own certificate; where each CRL is read from describes it. Two issuing CAs signed by one root normally carry the same distribution points, so both installations read the same root CRL and each one runs its own warning ladder against it.

The root warning therefore arrives twice. At each step of the ladder, 30, 14, 7 and 1 days before expiry by default and once more when the CRL expires, each installation sends its own alert, to its own recipients, naming the root CA and the CRL number in the payload. The ladder and its thresholds are the ones revocation lists describes, shared with certificate expiry. Two alerts with the same CA name and CRL number from two servers are not a bug and not a sign that one installation is reading the wrong CRL: it is one CRL being watched from two places, which is exactly what two independent installations do. Call it the **duplicate root warning** when you brief whoever receives the alerts, so that it is expected rather than filed.

There is no setting that tells one installation to leave the root to the other. `Certus:Alerts:Crl:Enabled` watches every CA in the chain or none, so turning it off on one server would also silence that server's warnings about its own CA's CRLs. Leave it on in both. If your alert receiver deduplicates, key on the CA name and the CRL number the payload carries, not on the sending server.

The forest rule

The server must be domain joined in the CA's forest: the CA authenticates the server's computer account through Active Directory, and the service reads certificate templates from the forest's directory. What to ask your AD and CA team lists it as a requirement that always applies, and the quickstart starts with it.

So where IT and OT are separate forests, they are separate installations, one server joined to each forest, and that stays true whatever a later release adds about covering more than one CA from one installation. A licence bound to a forest follows the same line; see forest binding.

What you do not get

Two installations are two products running side by side, and nothing joins them:

- **No single inventory.** Each dashboard lists the certificates of its own CA. Finding a certificate means knowing which CA issued it, or searching both.
- **No single alert configuration.** Recipients, thresholds and the webhook are set once per installation, and a change is made twice.

- **No single dashboard.** Two external URLs, two administrator groups, two Settings pages, two licence cards.

The licence page states that the paid tier adds capabilities beside the core, such as covering more than one certificate authority. That is the whole of what is promised: no date, and no description of what the capability will look like. Nothing on this page changes when it arrives, because the one installation per CA arrangement stays supported.

Two installations and the free tier

Two installations of one organisation, each connected to one CA, are two free installations. Nothing in the licence counts organisations: the free tier is counted per installation and per CA.

8. Configuration

Every setting the service reads, where to put it, and which file wins.

Where settings live

Configuration is layered. Later sources override earlier ones:

1. `appsettings.json` in the **installation** folder. Shipped defaults. Upgrades replace this file, so do not edit it.
2. `appsettings.{Environment}.json`, if one is present.
3. `settings.json` in the **data** folder. This is the one to edit. The setup wizard writes here, and it survives upgrades.
4. Environment variables.
5. Command line arguments.

Restart the service after editing a file. The overlay is not watched for changes, and the restart is the apply step.

Edit `settings.json` as an administrator. The data folder is writable only by SYSTEM and Administrators, so open the file from an elevated session. The service ignores the file, with a Critical line in the log, if anyone other than SYSTEM or Administrators owns it or may write it. See Troubleshooting.

Keep secrets out of `appsettings.json`. The installation folder is readable by every local user, so an SMTP password, a webhook signing secret or a webhook header put there is readable by every local user too; a secret belongs in the dashboard, which stores it protected, or in `settings.json` in the data folder, which only SYSTEM and Administrators can read.

The service writes to `settings.json` too, and keeps your keys. The setup wizard, the Settings page, the alert settings card and the HTTPS certificate renewal each change only the keys they manage and write every other key back as it was. One thing they refuse: if you set a key they manage in the colon form at the top level, such as `"Certus:ExternalUrl"`, a save would set that key twice, which stops the service starting. The save is refused instead and the log names the key. Move it into the `"Certus"` section and save again.

Not everything lives in these files. The allowed domain list, the EAB enforcement mode, the revocation scope and the device attestation settings are stored separately and apply immediately, with no restart. Those are called out where they appear below. The instance mode lives in the registry instead; see Monitor mode.

A minimal `settings.json` after setup looks like this:

```
{
  "Certus": {
    "CaConnectionString": "CA-SERVER\\MyCA",
    "ExternalUrl": "https://ducks.yourdomain.local:5001"
  }
}
```

A setting that looks like it has a default may be null. Several keys ship in `appsettings.json` as an explicit JSON `null`, and the configuration binder applies that null over the value the code would otherwise start from. So the shipped file, not the code, decides the starting value for those keys. They are marked below.

Core settings

Setting	Default	Purpose
`Certus:CaConnectionString`	`null`, ships null	The CA, in `CAHOST\CA Name` form. The wizard sets it
`Certus:DatabasePath`	`ducks.db` in the data folder, ships null	The SQLite database file. It must be in the data folder, and the service refuses to start otherwise (why); to keep the data on another disk, move the whole data folder. A blank value opens a private temporary database per connection, which is why it is normalised after binding rather than trusted from the file
`Certus:ExternalUrl`	`null`, ships null	The base URL you expect clients to use. Recorded and checked at startup. When it is set, every URL the ACME server hands out takes its scheme, host and port from it, whatever host the client connected to; when it is not, ACME URLs follow the host the client connects to
`Certus:SyncIntervalMinutes`	`5`	How often the dashboard syncs certificates from the CA. Values below 1 are raised to 1
`Certus:RequestHistoryDays`	`30`	How far back the sync reaches for pending, denied and failed requests, so their detail pages can show the CA's own explanation. Issued and revoked certificates are always synced in full. `0` skips those three passes and empties the matching filters
`Certus:FreeSpaceWarningMegabytes`	`1024`	Below this much free space on the volume holding the data folder, the health endpoint reports `Degraded` and the log carries a warning (once when it drops, then daily while it stays low). Nothing is deleted automatically. `0` turns the check off. Absent from `appsettings.json`, so this is the default unless you add it

Setting	Default	Purpose
`Certus:EnableWalMode`	`true`	SQLite write ahead logging. Set `false` on a filesystem that cannot support WAL, or when a backup tool needs a single file with no `-wal` and `-shm` sidecars. Applied in both directions on every start, so changing it converts the existing database
`Certus:UseMockCa`	`false`	Development only. The service refuses to start if this is true while a CA connection string is set
`Certus:SettingsOverlayPath`	`settings.json` in the data folder	Development and tests only. The service reads the overlay from the data folder and refuses to start when this names any other file (why)

The server's own HTTPS certificate

Ducks in a Row can enrol and renew its own TLS certificate from the CA it is connected to. See quickstart for turning it on.

Setting	Default	Purpose
`Certus:HttpsCertificateThumbprint`	`null`	The certificate currently served, from `LocalMachine\My`. The wizard and the Settings page write it. When the certificate it names has been archived by a renewal, or has expired, the service serves the newest one issued after it to the same subject from the same template, and records that one here; see following a renewal

Setting	Default	Purpose
`Certus:HttpsCertificateAutoRenewalEnabled`	`true`	Renew before expiry without being asked. Only a certificate the service installed itself is renewed, which is one carrying the friendly name `Ducks in a Row HTTPS`; a certificate someone else installed, such as one chosen in monitor mode, is left to whoever issued it. While it is on, a certificate it renews is left out of the expiry alerts, because renewal owns it; while it is off, or for a certificate it leaves alone, the server's certificate gets the same alerts as every other certificate. Monitor mode turns it off
`Certus:HttpsCertificateRenewalWindowDays`	`30`	How long before expiry to renew
`Certus:HttpsCertificateRenewalCheckIntervalHours`	`24`	How often to check

The last three are deliberately absent from `appsettings.json`, so they take the values above unless you add them.

Monitor mode

Monitor mode is for a CA you want watched but not written to. The service keeps the certificate inventory, the expiry alerts, CRL watching and the alert settings, and turns off everything that writes to the CA: ACME, issuance and revocation. It is the low risk first step onto a production CA. Turning issuance on later is a rights grant, a change to this value and a restart, never a reinstall. Monitor mode has the whole picture: what it does and never does, what it needs from the CA team, choosing it at install, the setup wizard, the HTTPS certificate, and switching in both directions.

The installer writes the value, from the **Monitor only** checkbox or `MONITOR_ONLY=1`. The mode is not in any configuration file. It is a registry value beside the data folder's:

Key	Value	Type	Contents
`HKLM\SOFTWARE\Haruspex Systems\Ducks in a Row`	`Mode`	`REG_SZ`	`monitor` or `full`

- Absent or empty means full, so an installation that never chose behaves exactly as before.
- Case and surrounding spaces do not matter.

- Any other value, or a value of another type such as `REG_DWORD`, stops the service at startup with the reason in the log. It is never guessed at: reading a mistyped `monitr` as full would turn issuance back on for a CA whose team was promised none.
- Only a local administrator can change it, and a restart applies it. Neither `appsettings.json`, `settings.json`, an environment variable nor the dashboard can set it, by design. The one exception is a host run in the Development environment, which a production service never is: there `Certus:Development:Mode` overrides the value, for testing.

To set it, from an elevated PowerShell session:

```
New-ItemProperty -Path 'HKLM:\SOFTWARE\Haruspex Systems\Ducks in a Row' -Name Mode -Value
monitor -PropertyType String -Force
Restart-Service DucksInARow
```

What changes in monitor mode, in configuration terms:

- Every ACME URL answers 404, and so does the ACME administration API under `/api/acme`.
- The challenge validation worker, the worker that collects certificates the CA held for approval, and the automatic renewal of the server's own HTTPS certificate do not run, whatever `Certus:HttpsCertificateAutoRenewalEnabled` says.
- The dashboard refuses to revoke a certificate, and to enrol or renew its own HTTPS certificate, with the problem type `https://ducksinarow.dev/problems/monitor-mode`.
- The server's own HTTPS certificate gets the ordinary expiry alerts, because nothing renews it.
- `Certus:Acme:ExposeAllTemplates` and a revocation scope of `all` have no effect, and the startup log says so at Information rather than warning.

Monitor mode needs **Read** on the CA, plus the **Request Certificates** that Authenticated Users hold on a default CA. See what it needs.

ACME

Setting	Default	Purpose
`Certus:Acme:ExposeAllTemplates`	`false`	Break glass. Exposes every template the CA publishes, ignoring the wizard's selection. Leave it false; the open posture is logged as a warning at every start

Which templates are exposed is otherwise decided by the wizard and stored in `ducks-setup.json`. With no recorded selection nothing is exposed at all, which is deliberate: it fails closed.

`Certus:Acme:ExposeAllTemplates` does not widen what the dashboard may revoke. That is a separate setting; see revocation.

Challenge validation egress

The server connects out to the name being validated for HTTP-01 and TLS-ALPN-01. `device-attest-01` never connects out at all.

Setting	Default	Purpose
`...ChallengeValidation:BlockPrivateRanges`	`false`	The RFC 1918 ranges are allowed, because an internal CA usually issues for exactly those addresses. Set true if everything you validate is public
`...ChallengeValidation:BlockLoopback`	`true`	Always refuse loopback
`...ChallengeValidation:BlockLinkLocal`	`true`	Always refuse link local, including the cloud metadata address
`...ChallengeValidation:BlockUniqueLocalIpv6`	`true`	Always refuse IPv6 unique local
`...ChallengeValidation:AdditionalBlockedCidrs`	`[]`	Extra ranges to refuse
`...ChallengeValidation:AllowRedirects`	`false`	Follow redirects during HTTP-01
`...ChallengeValidation:MaxValidationAttempts`	`5`	Attempts before a challenge fails
`...ChallengeValidation:PollIntervalSeconds`	`5`	How often the validation worker sweeps

The prefix is `Certus:Acme:ChallengeValidation:`. The hardening guide explains the tradeoffs.

Orders the CA holds

Setting	Default	Purpose
`Certus:Acme:PendingIssuance:PollIntervalSeconds`	`60`	How often to ask the CA about orders it is holding for manager approval. Clamped between 5 seconds and 1 hour

A template set to hold every request for approval is an ordinary ADCS configuration, not an error. The order stays `processing` until an operator approves or denies it at the CA, and this sweep is what finishes it.

Rate limiting

Setting	Default	Purpose
`Certus:RateLimiting:Enabled`	`true`	
`Certus:RateLimiting:NewAccountLimit`	`10`	Per window, `acme-new-account`
`Certus:RateLimiting:NewOrderLimit`	`30`	Per window, `acme-new-order`

Setting	Default	Purpose
`Certus:RateLimiting:PollLimit`	`300`	Per window, `acme-poll`: authorization and order polling
`Certus:RateLimiting:GeneralLimit`	`100`	Per window, `acme-general`: everything else
`Certus:RateLimiting:WindowSeconds`	`60`	
`Certus:RateLimiting:SegmentsPerWindow`	`6`	How many steps the window releases permits in
`Certus:RateLimiting:HelpUrl`	none	Documentation URL for a `Link: rel="help"` on a refusal
`Certus:RateLimiting:MaxStoredNonces`	`10000`	The most replay nonces kept at once; the oldest are evicted when it is full. Values below 1 are raised to 1
`Certus:RateLimiting:MaxIdentifiersPerOrder`	`100`	The most identifiers one new-order request may name. More is refused as `malformed`
`Certus:RateLimiting:MaxPendingOrdersPerAccount`	`1000`	The most orders one account may hold open (pending or ready, not expired). A request from an account at the cap is refused as `rateLimited`

A non positive value while rate limiting is enabled is logged as a warning at startup.

The last three bound what the server stores rather than how often it is asked, so they apply whether or not `Enabled` is on. A nonce is stored on every ACME protocol response and lives an hour, so without the cap a client could grow the store at line rate; evicting the oldest costs a legitimate client at most one `badNonce` retry, which RFC 8555 requires every client to handle. No nonce is minted on a rate limit refusal or on an unknown ACME URL. The open order cap clears when the client finalizes an order, deactivates its authorizations, or lets it expire a week after it was placed, so that refusal carries no `Retry-After`.

Which rate limit refused a request

A refusal names the policy that produced it, so you can tell which of the four settings above to change. The name appears in three places:

- the `detail` of the ACME problem document the client receives, for example `Too many new account requests.`, which is what shows up in the client's output
- a `Retry-After` header, in whole seconds

- a Warning in the service log naming the policy, the path, the caller's address, and the setting to raise. That line is written at most once per policy per minute, and reports how many further refusals it did not log, so that a flood cannot fill the log file

Set `HelpUrl` to your own runbook to add an RFC 8555 section 6.6 `Link` header pointing at it, with the policy name as the fragment. Nothing is sent anywhere by default.

The ACME polling budget

`PollLimit` covers the two POST-as-GET endpoints a client repeats while it waits, `authz` and `order`. Their request count is set by how long validation takes rather than by how many certificates were asked for.

The default is 5 requests a second. A client polling once a second contributes 60 a minute for as long as it waits, so the default carries about five concurrent issuances polling that hard, or ten polling every two seconds. The two endpoints share the budget without doubling the cost, because RFC 8555 puts them in sequence: a client polls the authorization until the challenge settles, then the order after finalize. Raise `PollLimit` before raising `GeneralLimit` if clients are refused part way through an issuance; the refusal names which of the two it was.

Rate limits partition per source address

Every policy counts per caller address, so anything that makes many clients share one apparent address makes them share one budget. A reverse proxy, a NAT gateway, or a Kubernetes cluster egressing through one address will do this.

Behind a proxy, set `Auth:TrustedProxies` to the proxy's address. Until you do, forwarded headers are ignored entirely and every client counts as the proxy, so one fleet shares a single limit. The service says so at startup when `TrustedProxies` is empty.

The partition is the full address, for IPv6 as for IPv4. An IPv6 client that holds a whole /64 can therefore present a fresh address per request and sit in a fresh partition each time; the per address limits do not stop that, and the resource caps above (the nonce store, the identifiers per order, the open orders per account) are what bound it. Partitioning IPv6 callers by prefix is not implemented.

Permits are released in `SegmentsPerWindow` steps across the window rather than all at once, so a refused fleet recovers gradually instead of retrying in lockstep. `Retry-After` reports one step, `WindowSeconds / SegmentsPerWindow`. A caller that spends its whole budget in a single instant still waits a full window, and will be told to retry sooner than that; retrying early costs one more refusal carrying a fresh `Retry-After`, which is the cheaper error than holding back a caller that could already proceed.

Refusals are immediate. Requests are never queued, because queueing on a window this long would hold a connection open until the window replenished instead of answering, which is worse for the client and for the server than an immediate refusal the client can act on.

Alerts

See alerts for what these do. Note the shipped `appsettings.json` writes this section with its full name as a single key, `"Certus:Alerts"`, rather than nesting `Alerts` inside `Certus`. Both forms bind.

Setting	Default	Purpose
`Certus:Alerts:Enabled`	`true`	
`Certus:Alerts:CheckIntervalMinutes`	`60`	
`Certus:Alerts:ThresholdDays`	`[30, 14, 7, 1]`	Days before expiry to alert
`Certus:Alerts:Smtp`	`null`, ships null	Null means email alerts are off
`Certus:Alerts:Webhook`	`null`, ships null	Null means webhook alerts are off
`Certus:Alerts:Crl:Enabled`	`true`	Watch the revocation lists of the CAs in the chain
`Certus:Alerts:Crl:CheckIntervalMinutes`	`60`	
`Certus:Alerts:Crl:FetchTimeoutSeconds`	`15`	How long to wait for a distribution point
`Certus:Alerts:Crl:MaxCrlBytes`	`33554432`	The largest CRL to accept, 32 MB

Most of the SMTP block is editable from the Settings page. The webhook block is deliberately not, so a dashboard administrator cannot redirect alert traffic.

Data retention

How long each class of record stays in the database before the daily retention pass removes it. The backup and restore page says what each class holds and which of it is personal data. Each period is a number of days, `0` keeps that class for ever, and none of the keys is in `appsettings.json`, so these are the defaults unless you add them to `settings.json`. The pass reads them at startup; restart after a change.

Setting	Default	Purpose
`Certus:Retention:FinishedOrderDays`	`90`	How long after an ACME order reached `valid` or `invalid` its authorizations and challenges are kept, and how long an invalid order itself is kept. A valid order's row and its certificate record are kept until this many days after the certificate expired, because renewal information, `replaces` and ACME revocation look a live certificate up through them

Setting	Default	Purpose
`Certus:Retention:DeactivatedAccountDays`	`365`	How long after deactivation an ACME account keeps its contact addresses. The account row itself is deleted once that period has passed and the order rule has removed its last order
`Certus:Retention:AlertHistoryDays`	`730`	How long certificate, CRL and licence alert history rows are kept after the alert was sent
`Certus:Retention:AuditTrailDays`	`730`	How long the administrator audit trail and the refused sign in counts are kept. Never below 90: a lower value is raised to 90 and the service log says so at startup. `0` keeps the trail for ever

The CA inventory mirror is not subject to retention and has no setting: it reflects what the CA holds, and the next sync would recreate anything removed. The policy rejection table keeps its newest 1,000 rows, which is not configurable.

Authentication

The dashboard and the setup API use Windows Integrated Authentication and are limited to an administrator group.

Setting	Default	Purpose
`Auth:Mode`	`Negotiate`	The only other legal value is `Disabled`, which the service refuses to start with while a CA is configured
`Auth:AdminGroup`	`null`, ships null	The group allowed in. Null means the built in Administrators group
`Auth:RequireHttps`	`true`	HSTS and an HTTP to HTTPS redirect outside development
`Auth:TrustedProxies`	`[]`	Reverse proxy addresses whose `X-Forwarded-*` headers are trusted
`Auth:RequireKerberos`	`false`	Refuse a dashboard sign in that used NTLM with a 401 that says why. The package every sign in used is logged whether this is on or off. Register the HTTP service principal name first; see NTLM and the dashboard

Web server and logging

Setting	Default	Purpose
`Kestrel:Endpoints:Http:Url`	`http://0.0.0.0:5000`	
`Kestrel:Endpoints:Https:Url`	`https://0.0.0.0:5001`	
`Kestrel:Endpoints:Https:Certificate:Path`	empty	Empty means the self signed certificate in the data folder is served
`Kestrel:Endpoints:Https:SslProtocols`	not set	Not set means TLS 1.2 and TLS 1.3 only, whatever Windows allows. A list here, such as `["Tls13"]`, replaces that. `Kestrel:EndpointDefaults:SslProtocols` is ignored for this endpoint. Every start logs the protocols the endpoint serves, and logs a warning when this list admits anything older than TLS 1.2
`Serilog:MinimumLevel:Default`	`Information`	Logs go to `logs\ducks-<date>.log` in the data folder; a file rolls at 100 MB and 30 files are retained

What the service refuses to start with

Two combinations are refused outright, because each one silently disables a security control that the presence of a CA says you wanted:

- `Auth:Mode=Disabled` while `Certus:CaConnectionString` is set.
- `Certus:UseMockCa=true` while `Certus:CaConnectionString` is set.

So is a `Mode` registry value that is neither `monitor` nor `full`, rather than guessed at; see Monitor mode.

Everything else that looks wrong is logged as a startup warning and the service runs. Read `logs\ducks-<date>.log` after a configuration change; the warnings name the setting.

9. ADCS Configuration

What to set up on the certificate authority side. Most of this is done once, and the setup wizard handles the parts it can. If someone else administers your CA or your directory, send them what to ask your AD and CA team first.

Finding your CA connection string

The connection string is `hostname\CA Name`. The wizard finds it for you, so you rarely need to build one by hand. These are the fallbacks.

Let the wizard discover it. Open `https://your-server:5001` after installing. The wizard lists the CAs published in Active Directory and lets you pick one. It then tests the connection through the same DCOM path the service uses in production, as the service's own account, and checks what else that account may do on the CA (see the service's rights). This is the recommended route and needs nothing typed.

Read it off the CA. Open the Certification Authority console (`certsrv.msc`) on the CA server. The CA name is the node in the console tree. The connection string is the CA server's hostname, a backslash, then that name.

Ask the forest. On any domain joined machine:

```
certutil -config - -ping
```

That lists the available CAs with their connection strings already in the right form.

Use the fully qualified host name. The short form has not been verified to work.

A CA name may contain a comma or a space. Both are legal and both are handled. Quote the whole string when you pass it to a command line tool.

Certificate template permissions

The service runs as LocalSystem, so the identity the CA sees is the server's machine account, written `DOMAIN\SERVERNAME$`. That account needs **Enroll** on every template you want to expose over ACME.

Template permissions live in the Certificate Templates console, not in the Certification Authority console:

1. On the CA server, run `certtmpl.msc`. You can also get there from `certsrv.msc` by right clicking **Certificate Templates** and choosing **Manage**.
2. Right click the template and choose **Properties**.
3. Open the **Security** tab.
4. Add the Ducks in a Row server's machine account, for example `DUCKS-SERVER$`. Machine accounts do not appear in the object picker until you add **Computers** to the object types. You can add a group that holds the account instead; see granting to a group for the restart that needs.
5. Grant **Enroll**. Do not grant **Autoenroll**: nothing here autoenrols, and granting it widens what the account can do for no benefit.

6. Click OK, and repeat for each template you intend to expose.

The wizard's template step checks each published template for ACME readiness and tells you what is missing. It does not check this permission. The Review step does: it reads each chosen template's permissions as the service's account and reports Enroll as **Inferred**, which is a reading and not a proof. Only an enrolment proves it. In the wizard that is the HTTPS certificate on the External URL step, for the template it uses; otherwise it is the first ACME order.

CA permissions

The CA's own rights are on its Security tab: Certification Authority console, right click the CA, **Properties**, **Security** tab.

Request Certificates is what submitting a request needs, and it is also what the CA asks for before it answers the wizard's Test Connection, lists its templates, hands over its own certificates, or lets the service read its CRLs or collect a certificate it held for approval. Authenticated Users hold it on a default CA, so it is usually in place already. If someone removed it, the CA refuses with `0x80094011`.

Read is what lets the dashboard sync the certificate inventory. Without it enrolment still works and clients get certificates, but the dashboard stays empty, which reads like a broken install and is not one. Neither right implies the other: Request Certificates does not open the inventory, and Read alone does not let the service submit.

Issue and Manage Certificates is a separate right, and optional. Every revocation the product makes needs it: from the dashboard, by an ACME client through `revoke-cert`, and the automatic revocation of a certificate that breaks the capability ceiling. See what the CA has to allow before granting it.

Template URL mapping

Each template is its own ACME endpoint:

```
https://your-server:5001/acme/<template>/directory
```

`<template>` is the template's programmatic name, which is its Active Directory `cn`, or its display name. Display names containing spaces work; the client URL encodes them. A display name carrying an invisible character, such as a soft hyphen pasted in from a word processor, is refused with a 400 rather than silently mismatching. Use the programmatic name in that case, and see troubleshooting.

No template is reachable over ACME until the wizard records a selection. On a fresh install every template's directory URL answers 403 until then, which is deliberate.

Key algorithm

An ACME client must request the key type the template asks for, and templates differ. A template built on an RSA provider needs an RSA key; an `ECDSA_P256` template needs a key on that curve. Requesting the wrong type is refused by the CA policy module at finalize. See reading a denial below for what that refusal tells you.

The provider a template names describes the key the client generates, not the CA's own signing key. Where the CA keeps that key, in software or in an HSM, is invisible to Ducks in a Row; see CA keys in an HSM.

You do not have to work this out yourself. The wizard's template step reports the key algorithm it read from the template, and the **Client setup** snippets on the ACME page of the dashboard are generated with the right flags for that template already filled in. Copy them from there rather than from memory.

See connecting ACME clients for the per client flags.

Reading a denial

When ADCS refuses a request it records two things: its own message, and a status code giving the reason. Ducks in a Row reports both, because the message on its own is often worth very little. A template Enroll denial on a stock Windows CA answers with the bare string `Denied by Policy Module` and nothing else.

The reason appears wherever the refusal does: in the setup wizard, in the service log at `C:\ProgramData\Ducks in a Row\logs\ducks-<date>.log`, and in the problem document an ACME client receives. It reads like this:

Denied by Policy Module. 0x80094012: The permissions on the certificate template do not allow the current user to enroll for this type of certificate.

The codes you are most likely to meet:

Code	What to do
`0x80094011`	The CA's own permissions refuse the account. Grant **Request Certificates** on the Security tab of the CA in `certsrv.msc`.
`0x80094012`	The template's permissions refuse the account. Grant **Enroll** as described under certificate template permissions.
`0x80094014`	A certificate manager denied a request that was waiting for approval. Nothing is misconfigured; this was a decision.
`0x80094800`	The template is not published by this CA. Add it under **Certificate Templates** in `certsrv.msc`, and check you named the template rather than its display name.
`0x80094811`	The key is smaller than the template's minimum. The wizard's template step reports that minimum.
`0x8009480A`	The template requires enrollment agent signatures, which an ACME request never carries. Use a different template.

A code Ducks in a Row does not have a description for is still reported as a number. `certutil -error 0x80094012` names any of them.

To confirm the same values at the CA itself:

certutil -view -restrict "RequestId=109" -out "Request.DispositionMessage,Request.StatusCode"

Templates that hold requests for approval

A template configured to require manager approval answers every submission "pending" rather than issuing. That is an ordinary ADCS setting, and it works: the ACME order stays in `processing` until an operator approves or denies the request at the CA, and Ducks in a Row finishes the order when that happens.

The wizard warns when a template you select is configured this way, so the behaviour is not a surprise. Nothing needs configuring for it to work.

10. Verifying Your Installation

Seven checks, in the order worth running them. Each one says where to go next when it fails, which for most of them is a section of troubleshooting.

The installed files carry our signature

Every `.exe` and `.dll` the installer puts in the application folder is signed by Haruspex Systems B.V., so two commands tell you whether the binaries on disk are the ones we published. Point them at the folder you installed into if you changed it.

```
$installed = Get-ChildItem "C:\Program Files\Ducks in a Row" `
  -Recurse -Force -File -Include *.exe, *.dll
$installed.Count # how many were checked. Zero means the path is wrong

$installed | Get-AuthenticodeSignature | Where-Object {
    $null -eq $_.SignerCertificate -or
    $_.Status -ne 'Valid' -or
    $_.SignerCertificate.Subject -notlike 'CN=Haruspex Systems B.V.,*'
  } | Format-List Path, Status, StatusMessage
```

A count with no rows under it is the answer you want. `-Force` is part of the check and not a detail: without it a file with the hidden attribute is skipped in silence. Keep the null test first as well: the two tests after it read a certificate that an unsigned file does not have.

A row is not proof of tampering on its own, so read it before acting.

- `HashMismatch`**, or** `Valid` **for a file signed by somebody else.** The file is not the one we published. Run `Get-AuthenticodeSignature` on it by itself to see who signed it, then stop the service, keep the file and `logs\ducks-<date>.log` as they are, and write to security@haruspex.systems about that first occurrence rather than reinstalling over the evidence.
- **The same complaint on every file at once**, usually `UnknownError` or an untrusted chain. That is normally something duller: a server with no internet access, which cannot finish building or checking our certificate's chain. Compare the installer against the SHA256 checksum instead, as checking the download describes. One file alone saying it belongs in the case above instead: a chain this server cannot check would not leave the rest of them clean.

This covers the binaries and not the rest: configuration files and the dashboard's own files are installed unsigned. Every binary it does cover carries the same publisher, which is what lets one publisher rule in an App Control for Business (WDAC) or AppLocker policy allow all of them, and this check is how you confirm that before writing the rule.

The service is running

```
Get-Service DucksInARow

# Status Name DisplayName
```

```
# ------ ---- -----------
# Running DucksInARow Ducks in a Row Certificate Proxy
```

A service that is not running has almost always logged the reason. Read `logs\ducks-<date>.log` in the data folder before anything else.

Health

```
Invoke-RestMethod https://your-server:5001/health
```

State	Meaning
Healthy	The database is reachable and so is the CA
Degraded	The database is fine, the CA is not reachable
Unhealthy	The database is not reachable

`Degraded` is the interesting one: the service is up and answering, but no certificate can be issued until the CA comes back. Alerts and the dashboard keep working, and ACME clients get a 503 that tells them to retry rather than an error that makes them give up.

There is a second endpoint, `/health/live`, which reports only that the process is up and answering. Use it for a load balancer probe, where a CA outage should not take the server out of rotation. Both endpoints are anonymous, so a monitor does not need credentials.

The ACME directory answers

```
Invoke-RestMethod https://your-server:5001/acme/WebServer/directory | ConvertTo-Json
```

Substitute a template you actually enabled. A 403 here means the template is not enabled for ACME, and a 404 means the name does not match a published template. Both are covered in troubleshooting.

The firewall rules exist

```
Get-NetFirewallRule -DisplayName "Ducks in a Row*" |
  Select-Object DisplayName, Enabled, Direction, Action
```

The installer opens inbound TCP 5000 and 5001. If you changed the ports in `settings.json`, the rules still name the old ones and you need to add your own.

The dashboard loads

Open `https://your-server:5001` in a browser, signed in as a member of the administrator group.

The HTTPS endpoint uses a self signed certificate out of the box, so the browser warns on the first visit. That is expected. See quickstart for giving the server a certificate your clients already trust.

If the dashboard loads but the certificate inventory is empty, the machine account is probably missing **Read** on the CA. Enrolment works without it and the inventory does not: Request Certificates, which Authenticated Users hold on a default CA, does not open the certificate database. So an empty dashboard next to working issuance points straight at that permission. See ADCS configuration.

The CA connection works

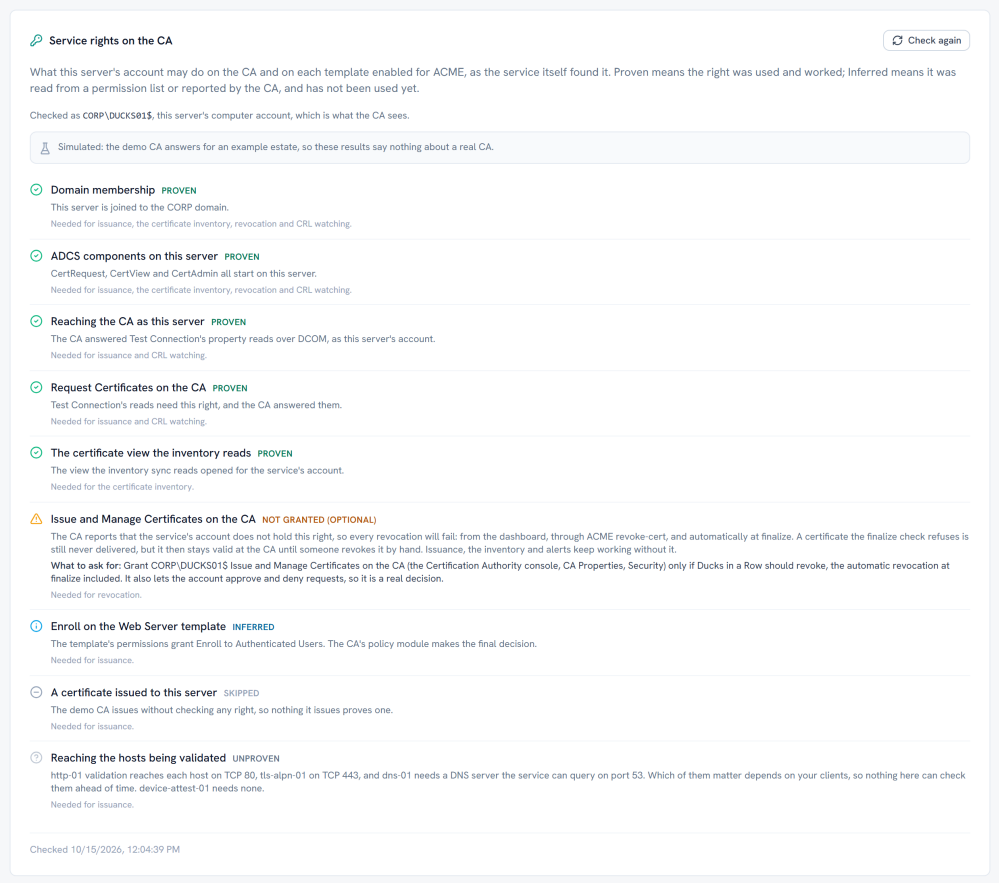
The wizard's connection step has a **Test Connection** button. It reads two properties from the CA through the same DCOM path the service uses in production, as the service's own computer account. The CA answers those reads only to an account holding **Request Certificates**, so a pass proves the network path, DCOM, and that one right.

It proves nothing about the other rights. On a default CA every domain computer holds Request Certificates through Authenticated Users, so a pass is the start of the answer, not all of it. The rights check below covers the rest.

If setup is already complete the wizard will not reopen. Use the health endpoint above, where `Degraded` is the same signal, or the rights check on the Settings page.

The service's rights

The wizard checks what the service's own account may do. The CA's rows appear as soon as Test Connection passes, and the Review step checks everything again, with one row for each template you chose. The **Service rights on the CA** card on the Settings page runs the same check against the configured CA and the templates enabled for ACME. **Check again** reruns it, for when a right has been granted or taken away since. A right granted through a group shows only after this server's Kerberos tickets renew; see granting to a group.



The Service rights on the CA card on the Settings page, showing a simulated report from the demo CA

Each row reads one of five:

Status	Meaning
Proven	The right was used and it worked, either just now or by the HTTPS certificate this CA issued to the server
Inferred	Read, not used: reported by the CA, or read from a template's permissions. Never a pass
Unproven	Nothing here could check it, or the check did not answer in time
Failed	Used and refused, or read and definitely absent. The row says what to ask for
Skipped	Not attempted, because something it depends on failed first

How each right is established:

- **Request Certificates on the CA** is proven by Test Connection itself.
- **The certificate view the inventory reads** is proven by opening it. The CA opens it to an account holding **Read**, and Request Certificates alone does not.

- **Issue and Manage Certificates** is reported by the CA, which reports the roles of any account holding Read or more. Only revocation needs this right, the automatic revocation at finalize included, so its row is marked optional.
- **Enroll on a template** is read from the template's permissions, in the order Windows evaluates them. It stays Inferred until something enrols from that template, because the CA's policy module decides with a token that can carry groups this server cannot see.

For a row that reads Failed, what to ask your AD and CA team says who usually grants each right and where.

In the wizard, only the HTTPS certificate on the External URL step proves Enroll. It proves Request Certificates and Enroll on the template it uses. The wizard offers it only when the URL answers over HTTPS with a certificate the server does not trust. Otherwise the Settings page can provision one later, or the first ACME order is the proof. If you complete setup without it, the Review step asks you to acknowledge the rows it could not prove. It never blocks completion, because you may still be collecting the approvals.

The check runs inside the service, as LocalSystem, which the CA and the directory see as `DOMAIN\SERVERNAME$`. It never checks the rights of the administrator at the browser, which would be the wrong answer. It only reads: nothing is requested, issued or revoked. On the dev host the report says **Simulated** and describes an example estate.

Issue a certificate

The real test is an end to end issuance. The quickstart has a ready to run certbot command, and the wizard's final step shows one already filled in for your server and template.

11. Connecting ACME Clients

Ducks in a Row implements RFC 8555, so any standard ACME client works. No agent or proprietary software is needed on your endpoints.

Running Kubernetes or OpenShift? cert-manager has a page of its own: Kubernetes and OpenShift.

The directory URL

Every certificate template is its own ACME endpoint:

```
https://your-server:5001/acme/<template>/directory
```

- `<template>` is the template's programmatic name (its AD `cn`) or its display name. Display names with spaces are accepted; the client URL encodes them. A display name carrying an invisible character, such as a soft hyphen left behind by a paste from a word processor, is refused with a 400 `malformed` problem document; use the programmatic name and see Troubleshooting.
- Use the HTTPS endpoint (5001) in production. Plain HTTP (5000) is for lab use only.
- Whether registration needs an external account binding credential depends on the server's enforcement mode; see External account binding below. A new install requires it, so a client needs a credential from the ACME page before it can register; an install upgraded from a version without the setting stays at Off, where clients register from their own key the first time they connect.

Trusting the server's TLS. ACME clients reject an untrusted TLS certificate on the ACME server itself. Give Ducks in a Row a certificate your clients already trust (for example, one chained to your internal CA root), or add that root to each client's trust store. The examples below note the per client switch for a lab where the server still uses its self signed certificate.

The example commands are representative. Exact flags vary by client version, so check each client's own documentation for your release.

External account binding

The ACME page of the dashboard sets one of three enforcement modes for external account binding (RFC 8555 section 7.3.4):

Mode	What a registering client needs
Off	Nothing. A presented binding is ignored. The default after an upgrade from a version without the setting.
Optional	Nothing, but a presented binding is verified and recorded.
Required (new install default)	A valid EAB credential; registration without one is refused with `externalAccountRequired`.

An administrator creates credentials on the ACME page. Each credential is a **key id** plus an **HMAC key** (base64url encoded, sized for HS256, which is what every client below signs with by default; HS384 and HS512 are also accepted). The HMAC key is shown exactly once, right after create or regenerate, so hand both values to the client operator then. The binding happens once, at account registration; later orders and renewals ride the existing account while the credential stays active (revoking a credential, or letting it expire, suspends orders from the accounts bound to it). Accounts that registered before the mode was raised to Required keep working; the ACME page lists them as Unbound.

A credential can also carry a **domain namespace**: accounts bound to it may only order certificates inside those domains, across every template. Orders outside it are refused with `rejectedIdentifier` and a message naming the credential.

The dashboard writes ready to paste setup for you: the panel shown right after create or regenerate inlines the real secret, and each credential row's **Client setup** action shows the same snippets with a placeholder for the saved secret. The dashboard covers the first six clients below; Traefik is listed here for its field names only. The flags per client:

Client	Directory URL	Key id	HMAC key
certbot	`--server`	`--eab-kid`	`--eab-hmac-key`
win-acme	`--baseuri`	`--eab-key-identifier`	`--eab-key`
acme.sh	`--server`, once with `--register-account`	`--eab-kid`	`--eab-hmac-key`
Posh-ACME	`Set-PAServer -DirectoryUrl`	`New-PAAccount -ExtAcctKID`	`New-PAAccount -ExtAcctHMACKey`
cert-manager	`spec.acme.server`	`externalAccountBinding.keyID`	`externalAccountBinding.keySecretRef`, a secret holding the base64url key; see the EAB secret
Caddy	`acme_ca`	`acme_eab` block, `key_id`	`acme_eab` block, `mac_key`
Traefik	`caServer`	`eab.kid`	`eab.hmacEncoded`

If a client with correct looking values is refused with `unauthorized`, the credential may be revoked, expired, or rotated; see troubleshooting for the causes and fixes.

Challenge types

Type	Good for	How Ducks in a Row validates it	Outbound port it uses
HTTP-01	Standard web servers	Fetches `http://<domain>/.well-known/acme-challenge/<token>`	80
DNS-01	Wildcards, hosts with no inbound HTTP	Looks up the `TXT` record at `_acme-challenge.<domain>`	53
TLS-ALPN-01	TLS only environments	Opens a TLS connection on port 443 and checks the ALPN certificate	443

The Ducks in a Row server is the party that performs validation, so the server needs outbound reachability to the domain or DNS being validated.

Key type

Template records	Ask for	Notes
An RSA provider	RSA, 2048 bits or more	The stock `Web Server ACME` template. Most clients default to an elliptic curve key, so this is the case that needs a flag
`ECDSA_P256`, `ECDSA_P384`, `ECDSA_P521`	An ECDSA key on the named curve	
`ECDH_P256`, `ECDH_P384`, `ECDH_P521`	An ECDSA key on the same curve	See below

An `ECDH_*` template is more common than it looks. The Certificate Templates console records ECDH when **Request Handling** has its purpose set to "Signature and encryption", which is a default rather than an unusual choice. A PKCS#10 certificate request cannot carry an encryption only key, so the answer is the signature key on the same curve, and the dashboard says as much when it generates the snippet.

Forcing RSA where the template is RSA:

Client	Flag
certbot	`--key-type rsa --rsa-key-size 2048`
lego	`--key-type rsa2048`
acme.sh	`--keylength 2048`

Client	Flag
dehydrated	`KEY_ALGO="rsa"`
win-acme	`--csr rsa`

cert-manager asks for RSA 2048 unless told otherwise, so an RSA template needs nothing, and an ECDSA template needs `privateKey` on every Certificate. See key type on the Kubernetes page.

CSR requirements

At finalize the server checks the CSR against the order before anything is submitted to the CA:

- The subject alternative names must be DNS names matching the order's domains exactly, wildcard marker included. Any other SAN type (an IP address, an email, a UPN, a directoryName, a PermanentIdentifier) is refused.
- The subject common name is optional. When a CN is present, it must be one of the domains on the order; a descriptive value such as `CN=My Web Server` is refused. Public ACME CAs enforce the same rule, so a client that works against Let's Encrypt needs no change.
- A CSR with no SAN at all is identified by its subject CN, which then has to match the order the same way.
- The CSR may request only a subject alternative name, a key usage, an extended key usage, basic constraints that ask for an end entity certificate (cA=false), and a subject key identifier of its own key. The clients on this page send nothing else by default. OCSP must staple (the TLS feature extension) is refused, so leave it off: `--must-staple` in certbot and lego, `--ocsp` in acme.sh, `-OCSPMustStaple` in Posh-ACME, and the OCSP must staple option in win-acme.

A CSR that breaks these rules is refused with `badCSR`, and the order stays in the ready state so the client can retry with a corrected CSR.

`badCSR` means the CSR itself, and only the CSR. A finalize on an order that is no longer ready is refused with `orderNotReady` instead, including the case where another request finalized the same order first. Rebuilding the key and the CSR cannot help there; re-read the order and act on the status it reports.

A CA that refuses the request is a different answer again. The finalize returns 500 with `serverInternal`, the detail carries the CA's own disposition message, and the order becomes invalid: the CA has decided, and no CSR will change its mind. Read the detail, fix the template or the key type, and start a new order.

That same problem document is also on the order itself, in the `error` member RFC 8555 section 7.1.3 defines, so a client that lost the finalize response or that polls rather than reads it can fetch the order and find the reason there. It reads word for word the same either way.

One case leaves the order's `error` empty on purpose: an order that went invalid because a challenge failed. The reason for that failure belongs to the authorization, so follow the order's `authorizations` URLs and read the `error` on the failed challenge, which is what section 7.1.6 asks a client to do.

If the CA cannot be reached at all, the finalize returns 503 with `serviceUnavailable` and the order is left alone. Retry the same finalize once the CA is back. Nothing is lost, and the authorizations you already completed still stand.

Renewal timing (ARI)

The directory advertises `renewalInfo`, the ACME Renewal Information extension (RFC 9773). A client that understands it asks the server when to renew each certificate instead of guessing from the expiry date, with a plain unauthenticated GET of `{renewalInfo}/{certID}`, where the identifier is built from the certificate's Authority Key Identifier and serial number exactly as the RFC describes. No account or signature is needed.

The response is a `suggestedWindow` sitting at roughly two thirds of the certificate's lifetime, about 2% of that lifetime wide, with a per certificate offset derived from the serial number. The offset is what keeps a fleet issued in one batch from renewing in one batch, and it is deterministic, so the window a client sees never moves between polls. The `Retry-After` header says how often to ask again (six hours).

Two behaviors are worth knowing about:

- A **revoked** certificate answers with a window entirely in the past, which an ARI aware client reads as "renew immediately". This is how a revocation reaches your fleet without anyone touching the clients, and it works however the certificate was revoked: through ACME, from the dashboard, or directly at the CA with certutil or certsrv.msc.
- When renewing, a conforming client adds a `replaces` member to its new-order request naming the certificate it is replacing. The server verifies it (same account, at least one shared identifier) and reflects it on the order. A second live order naming the same certificate is refused with HTTP 409 and `urn:ietf:params:acme:error:alreadyReplaced`, which stops duplicate renewal loops; if that happens, the earlier order is the one to complete. An earlier order that was abandoned stops blocking once it passes its own expiry, so a crashed renewal attempt never locks a certificate out.

Clients that read ARI include certbot from 4.1.0, lego (checked by default since 4.20.2), Caddy from 2.8.0, and simple-acme, which follows it unless `ScheduledTask.RenewalDisableServerSchedule` is set. cert-manager reads it only from 1.21, and only with its alpha `ACMEUseARI` feature gate switched on; see renewal and ARI. Clients that do not simply ignore the directory member and renew on their own schedule, as before.

certbot

```
# HTTP-01, certbot answers the challenge itself on port 80
certbot certonly --standalone \
  --server https://your-server:5001/acme/WebServer/directory \
  --email you@example.com \
  -d host.corp.example.com \
  --key-type rsa --rsa-key-size 2048
```

Match the key type to the template. Your client must ask for the key type the template wants, and templates differ. Ask for the wrong one and the CA policy module refuses at finalize with `Denied by Policy Module`, which reads like a permissions problem and is not one. The wizard's template step reports the key algorithm it read from the template, and

the **Client setup** snippets on the dashboard's ACME page come with the right flags already filled in for that template. That is the reliable answer for your CA; the table under key type is the general shape.

For a lab server with an untrusted TLS certificate, set `REQUESTS_CA_BUNDLE` to your CA root PEM so certbot trusts the endpoint, or install that root into the system trust store.

win-acme (Windows)

Run `wacs.exe` and, in the menu, set the ACME server to the template directory URL, then create a certificate. Unattended example:

```
wacs.exe --source iis --siteid 1 ^
  --baseuri https://your-server:5001/acme/WebServer/directory
```

win-acme uses the Windows certificate trust store, so import your CA root on the machine running it if the server certificate is not already trusted.

Caddy

```
{
  acme_ca https://your-server:5001/acme/WebServer/directory
  # For an internal CA, point Caddy at the root so it trusts the ACME endpoint
  acme_ca_root /etc/ssl/certs/corp-root.pem
  email you@example.com
}

host.corp.example.com {
  respond "Hello from Caddy"
}
```

Traefik

```
certificatesResolvers:
  ducks:
    acme:
      caServer: https://your-server:5001/acme/WebServer/directory
      email: you@example.com
      storage: /etc/traefik/acme.json
      httpChallenge:
        entryPoint: web
```

Traefik uses the host or container trust store. Add your CA root there if the ACME endpoint certificate is not already trusted.

Posh-ACME (PowerShell)

```
Import-Module Posh-ACME

# Point Posh-ACME at the template directory.
```

```
# Add -SkipCertificateCheck only for a lab server with untrusted TLS.
Set-PAServer -DirectoryUrl https://your-server:5001/acme/WebServer/directory

New-PAAccount -Contact you@example.com -AcceptTOS

New-PACertificate -Domain host.corp.example.com -Plugin WebRoot `
  -PluginArgs @{ WebRootPath = 'C:\inetpub\wwwroot' }
```

cert-manager (Kubernetes)

```
apiVersion: cert-manager.io/v1
kind: Issuer
metadata:
  name: ducks-in-a-row
spec:
  acme:
    server: https://your-server:5001/acme/WebServer/directory
    caBundle: <base64 of your CA root, PEM>
    email: you@example.com
    privateKeySecretRef:
      name: ducks-account-key
    solvers:
      - http01:
          ingress:
            ingressClassName: nginx
```

cert-manager trusts only public roots unless `caBundle` names yours, and it asks for an RSA 2048 key unless a Certificate says otherwise. Issuer or ClusterIssuer, the EAB secret, the path an HTTP-01 check takes through your ingress, and what is known about OpenShift are on the Kubernetes and OpenShift page.

Picking a template

Any template that was enabled in the setup wizard is reachable at `/acme/<template>/directory`. Match the template to the certificate purpose, for example a Web Server template for TLS server certificates. A template the CA publishes but that was not enabled in the wizard answers `403 not enabled for ACME`; if a directory URL returns `404 unknown certificate template`, check the name against the templates shown in the setup wizard.

Which URLs check the template

Five URLs check the template name: the directory, `new-account`, `new-order`, `issuer-cert` and `renewalInfo`. Each refuses a template that is unknown or not enabled for ACME, as described above. The template a certificate is issued from is fixed when the order is created.

The other URLs carry a template name but do not check it: the order and its `finalize`, an authorization (`authz`), a challenge, the certificate download, the account and its list of orders, `key-change`, `revoke-cert`, which finds the certificate from the certificate itself, and `new-nonce`, whose nonces are accepted on every template's URLs. A signed request is authorized by the key that signed it: the account key, or for

`revoke-cert` either the account key or the certificate's own key. An order is always finalized against the template it was created under, whatever name the `finalize` URL carries.

So a request through another template's URL, or through the URL of a template that is no longer offered over ACME, is not refused for the name. One thing still follows the name in the URL: the certificate download links to the issuing CA certificate at `issuer-cert` under the same template name, and that URL does check it, so a client that walks the chain should download through the URL of an enabled template. The account URLs in a response, such as the `Location` header of `key-change`, also carry the template name of the URL the request was sent to; they name the same account.

This is deliberate. RFC 8555 does not require these URLs to match the directory a client started from, and a client revoking a certificate after a key compromise must never be held up by a template setting. See Revocation.

For `key-change`, the inner payload's `account` must be the same URL as the `kid` the outer request is signed with, as RFC 8555 section 7.3.5 requires, whatever template name the `kid` or the `key-change` URL carries.

12. Kubernetes and OpenShift

Workloads in a Kubernetes cluster get their certificates from cert-manager, and often from a CA the cluster keeps for itself, which nothing outside the cluster trusts. cert-manager's built in ACME issuer can ask Ducks in a Row instead. The certificates then chain to the ADCS root your organisation already trusts. The cluster needs no plugin, holds no Active Directory account and no rights on the CA, and one scoped credential can decide which names it may order.

This page covers cert-manager on Kubernetes, and what is known about OpenShift. For other clients, see connecting ACME clients.

What has been run. Our regular test cycle runs cert-manager against a real ADCS certificate authority: a namespaced Issuer, a Certificate with an RSA key, the Ingress annotations, HTTP-01 through ingress-nginx, and DNS-01. Not yet run against Ducks in a Row: trusting the server over HTTPS with `caBundle`, external account binding, a ClusterIssuer, an ECDSA template, renewal and ARI, a challenge that fails, and anything on OpenShift. Sections that rely on those say so, and what they describe is read from the server's code and from cert-manager's own code and documentation.

Before you start

On the Ducks in a Row side:

- The template is enabled for ACME in the setup wizard. One that is not answers 403, "not enabled for ACME".
- If the allowed domain list is on, the cluster's host names are on it.
- The external account binding mode suits you, and a credential exists if the mode is Required or you want the cluster's orders limited to its own domains. See external account binding.
- The server's External URL is the name the cluster will use to reach it. See pointing at a template.
- The server has enrolled its own HTTPS certificate from your CA. See giving clients a TLS certificate they trust.
- The server can reach the cluster's ingress on TCP 80, and its DNS resolves the certificate names to that ingress. See HTTP-01 through your ingress.

On the cluster side:

- cert-manager 1.11 or later, the first release with `caBundle`. Use a release cert-manager still supports.
- An ingress controller that answers plain HTTP on port 80.

Issuer or ClusterIssuer

Each one is an ACME account on the server. They differ in who can use them and where their secrets live.

	Issuer	ClusterIssuer
Issues for	Certificates in its own namespace	Certificates in every namespace

	Issuer	ClusterIssuer
Reads its secrets from	Its own namespace	The cluster resource namespace: `cert-manager`, unless the controller's `--cluster-resource-namespace` flag changed it
Accounts on the server	One per Issuer	One for the cluster

Not yet run against Ducks in a Row: a ClusterIssuer. Nothing on the server tells the two apart; both register an account the same way.

Choose by who owns the host names. If each team owns its own names, give each team's namespace its own Issuer and each team its own credential, with a domain namespace listing that team's domains. A team can then order only its own names, and revoking one team's credential leaves the others alone. If one platform team owns every name, a single ClusterIssuer with one credential is simpler.

The word namespace means two unrelated things here. A Kubernetes namespace is where an Issuer and its secrets live. A domain namespace belongs to an EAB credential and lists the DNS domains its accounts may order; see domain namespaces. The pattern above maps one of each to one team.

Start from the dashboard manifest

The dashboard writes the manifest for you. On the **ACME** page, open **Credentials**, create a credential, and pick **cert-manager** in the setup panel. The panel shown right after create inlines the real HMAC key. Later, a credential row's **Client setup** shows the same manifest with a placeholder where the key goes, because the key is shown only once.

The manifest holds three things:

- the `kubectl create secret` line that stores the HMAC key,
- an `Issuer` pointed at the template's directory, carrying the credential's key id and a `caBundle` placeholder,
- a sample `Certificate` whose `privateKey` already matches the template.

Before you apply it:

1. Replace the `caBundle` placeholder; see trusting the ACME endpoint. Left in place, it makes `kubectl apply` refuse the manifest.
2. Set the email, and set `ingressClassName` to your ingress controller's class.
3. Pick the namespace, and give the secret line and `kubectl apply` the same `-n`.
4. For a ClusterIssuer, change the Issuer's kind and the Certificate's `issuerRef` kind to `ClusterIssuer`, and create the secret in the cluster resource namespace instead.

The Issuer it produces looks like this:

```
apiVersion: cert-manager.io/v1
kind: Issuer
metadata:
```

```
  name: ducks-in-a-row
  namespace: team-a
spec:
  acme:
    server: https://ducks.corp.example.com:5001/acme/WebServer/directory
    caBundle: <base64 of your CA root, PEM>
    email: you@example.com
    privateKeySecretRef:
      name: ducks-account-key
    externalAccountBinding:
      keyID: <key id from the dashboard>
      keySecretRef:
        name: ducks-eab
        key: secret
    solvers:
      - http01:
          ingress:
            ingressClassName: nginx
```

Leave out `externalAccountBinding` if the enforcement mode is Off and you are not using a credential.

The EAB secret

Not yet run against Ducks in a Row: external account binding with cert-manager.

Store the HMAC key exactly as the dashboard shows it:

```
kubectl -n team-a create secret generic ducks-eab --from-literal=secret='<HMAC key>'
```

- The key is already base64url encoded: 43 characters, no padding. Store it exactly as shown, and do not encode it again, even though cert-manager's own documentation shows an encoding step for keys that arrive raw. cert-manager decodes the stored value and uses the result as the HMAC key as it stands, so a key encoded twice, or stored with a trailing newline (as when a file written by `echo` is loaded with `--from-file`), gives the wrong key. The server then refuses the registration with 403 `unauthorized` and `External account binding signature verification failed.` A value that is not base64url at all fails earlier, inside cert-manager, with `failed to decode external account binding key data`. Use `--from-literal`, as above.
- The binding happens once, when cert-manager registers its account. From then on the account key, stored in the secret `privateKeySecretRef` names, is the cluster's identity. Keep it, and back it up with the cluster's other secrets.
- One credential can bind any number of accounts. A rebuilt cluster, a second cluster or an extra Issuer registers a new account against the same credential, and the ACME page lists every account bound to it.
- Regenerating a credential keeps its key id and every account already bound to it. Only a new registration needs the new key, but update the Kubernetes secret anyway so the next one works.

- An account stays bound to the credential it registered with. Pointing an existing Issuer at a different key id changes nothing on the server, and if the old credential is revoked, that account's orders are refused with `unauthorized`. To move an Issuer to a new credential, give `privateKeySecretRef` a new name as well, so cert-manager creates a new account key and registers it with the new binding.
- Revoking a credential, or letting it expire, suspends orders from every account bound to it; see revoking, expiring, and the difference.

Pointing at a template

`spec.acme.server` is the template's directory URL:

```
https://ducks.corp.example.com:5001/acme/<template>/directory
```

- Use the host name in the server's External URL, exactly. The certificate the server enrols for itself names that host and no other, so cert-manager refuses any other name for it. And once the External URL is set, every URL the server hands back, for new accounts, orders and certificates, is built from it, so the cluster has to resolve and reach that name whatever the directory URL says. The dashboard's manifest already uses it.
- Use HTTPS on port 5001. Plain HTTP on port 5000 is for lab use only, and by default it redirects to HTTPS.
- Every template is its own directory, so each template needs its own issuer. Name it by its programmatic name, its AD `cn`, to keep spaces out of the URL.

Trusting the ACME endpoint

Not yet run against Ducks in a Row: `caBundle`. Our test cycle reaches the server over plain HTTP.

Once the server has enrolled its own certificate, that certificate comes from your ADCS CA, and cert-manager trusts only the public roots in its container, so it has to be told about yours. Without that the Issuer never becomes ready, and reports `x509: certificate signed by unknown authority`. A server still serving its self signed certificate is no use here: that certificate names only `localhost` and the server's short machine name, not the External URL host, so enrol one first.

Put your root certificate in `caBundle`, as PEM, base64 encoded on one line:

```
base64 -w0 corp-root.pem

[Convert]::ToBase64String([IO.File]::ReadAllBytes('corp-root.pem'))
```

To get the PEM, export the root on any domain joined Windows machine: in `certlm.msc`, open **Trusted Root Certification Authorities**, then **Certificates**, right click your root, choose **All Tasks**, **Export**, and pick **Base-64 encoded X.509**. With an issuing CA under an offline root, put the issuing CA's certificate in the bundle after the root, so the chain verifies even when the server does not send it.

- Never set `skipTLSVerify`. It turns off verification of the server entirely, and cert-manager's own documentation marks it insecure.

- cert-manager is written in Go, which is stricter than Windows about two things older ADCS estates run into. It refuses a certificate signed with SHA-1 anywhere below the root, so a CA that still signs with SHA-1 has to move to SHA-256 first. And it never takes a host name from a certificate's common name, only from its subject alternative names. The certificate the server enrols for itself carries the External URL host as a DNS name, so it passes; a certificate you install by hand has to as well.

HTTP-01 through your ingress

Ducks in a Row validates a challenge itself, from the Windows server it runs on. A challenge goes like this: cert-manager starts a small solver pod, routes `/.well-known/acme-challenge/<token>` to it through your ingress controller, checks that it can fetch the token itself, then tells the server it is ready. The server then fetches `http://<name>/.well-known/acme-challenge/<token>` on port 80.

What that asks of your network:

- The server must reach the ingress on TCP 80; see network ports. In a segmented network this is the one firewall rule to ask for, from the server to the ingress addresses.
- The server resolves the name with its own DNS servers and connects to the first address it gets back. On the server, the name has to resolve to the ingress, not to an address only the cluster can reach.
- By default the server does not follow redirects (`AllowRedirects` is off). If your ingress sends plain HTTP to HTTPS for the whole host, exempt `/.well-known/acme-challenge/` from that, or the challenge fails with the redirect's status, for example `server returned HTTP 308`.
- By default a name that resolves to a loopback, link local or IPv6 unique local address is refused, and so is a name with any such address among its answers. If `BlockPrivateRanges` or `AdditionalBlockedCidrs` is set in challenge validation egress, make sure the ingress addresses are still allowed.

cert-manager's own check, the one before it tells the server it is ready, runs from inside the cluster through the cluster's DNS. If the cluster cannot resolve your internal zone, the challenge sits pending and the server is never asked; our test cycle hit exactly this. Make the zone resolvable in the cluster, for example by forwarding it from CoreDNS to your DNS servers. From cert-manager 1.21 you can instead set `waitInsteadOfSelfCheck` on the solver, which skips the check and waits a fixed time.

HTTP-01 cannot prove a wildcard name, so a wildcard order is offered DNS-01 only. The server looks up `_acme-challenge.<name>` with its own DNS servers, so cert-manager's DNS-01 solver has to write into a zone those servers answer from. Setting one up for an internal zone is outside this page; cert-manager's documentation lists the providers, and our test cycle has run its RFC 2136 solver. The server also offers TLS-ALPN-01, but cert-manager has no solver for it.

Requesting a certificate

A `Certificate` asks for one:

```
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
```

```
  name: app
  namespace: team-a
spec:
  secretName: app-tls
  dnsNames:
    - app.corp.example.com
  issuerRef:
    name: ducks-in-a-row
    kind: Issuer
  privateKey:
    algorithm: RSA
    size: 2048
```

Or annotate an Ingress, and cert-manager creates the Certificate from the Ingress's `tls` section:

```
metadata:
  annotations:
    cert-manager.io/issuer: ducks-in-a-row
    cert-manager.io/private-key-algorithm: RSA
    cert-manager.io/private-key-size: "2048"
```

For a ClusterIssuer the annotation is `cert-manager.io/cluster-issuer`, and a Certificate names it with `kind: ClusterIssuer` in its `issuerRef`.

What the server accepts is set out under CSR requirements. For a Certificate that means:

- DNS names only. The server refuses IP address identifiers and every other kind of subject alternative name, so leave `ipAddresses`, `uris` and `emailAddresses` unset.
- `commonName` is optional. If you set it, it has to be one of the `dnsNames`.
- `duration` changes nothing. The template sets the certificate's lifetime. cert-manager sends a requested lifetime only when the issuer sets `enableDurationFeature`, and the server then accepts the request and still issues for the template's lifetime.

Key type

The template decides the key type, and cert-manager defaults to RSA: with no `privateKey` it asks for RSA 2048. So an RSA template needs nothing, and an ECDSA template needs the setting on every Certificate, or the annotations on every Ingress.

Template records	privateKey
RSA, minimum 2048	Nothing, or `algorithm: RSA` and `size: 2048`
RSA, minimum above 2048	`algorithm: RSA` and `size: 4096`. cert-manager accepts only 2048, 4096 and 8192, so a 3072 minimum needs 4096
`ECDSA_P256`	`algorithm: ECDSA` and `size: 256`

Template records	privateKey
`ECDSA_P384`	`algorithm: ECDSA` and `size: 384`
`ECDSA_P521`	`algorithm: ECDSA` and `size: 521`
`ECDH_P256`, `ECDH_P384`, `ECDH_P521`	The ECDSA key on the same curve; see key type

Not yet run against Ducks in a Row: an ECDSA template. Our test cycle issues with an RSA key.

The dashboard's manifest fills this in for the template you pick, and the setup wizard shows the algorithm it read from the template.

A key the template does not accept is refused by the CA's policy module, not by Ducks in a Row. The finalize answers `serverInternal` carrying the CA's own message, typically `Denied by Policy Module` with the explanation that the public key does not meet the template's minimum size, and the order becomes invalid. cert-manager then waits before trying again, starting at an hour and doubling up to 32 hours. Fix the key settings, then run `cmctl renew <certificate>` to retry at once.

Trusting issued certificates in pods

The Secret cert-manager writes holds `tls.crt` and `tls.key`. The server returns the leaf, the issuing CA and the root, and cert-manager keeps what it downloads, so `tls.crt` carries that whole chain. The ACME issuer never writes `ca.crt`, so an application that insists on one has to be given the root another way.

Anything that connects to these workloads has to trust your ADCS root. Windows machines in the domain already do. Inside the cluster you distribute the root yourself:

- **A ConfigMap.** Put the root PEM in a ConfigMap in each namespace that needs it and mount it beside the certificate, for example with a projected volume that presents `tls.crt`, `tls.key` and the root in one directory. It needs no extra component, but keeping the copies current is up to you.
- **trust-manager.** cert-manager's companion project copies one bundle into every namespace, or into the namespaces a label selects. Its `Bundle` API is still `v1alpha1`, so follow its documentation for the release you install.

Renewal and ARI

cert-manager renews at two thirds of the certificate's actual lifetime unless you set `renewBefore` or `renewBeforePercentage`, and the lifetime is whatever the template gave the certificate. Since cert-manager 1.18, every renewal generates a new private key, because `rotationPolicy` defaults to `Always`.

The server advertises ACME Renewal Information (ARI), which lets it tell a client when to renew; see renewal timing. Its suggested window sits at about the same two thirds point, so on an ordinary day ARI changes nothing for cert-manager. It matters after a revocation: a revoked certificate's window moves into the past, and a client that reads ARI renews at once.

cert-manager reads ARI only from release 1.21, and only with the `ACMEUseARI` feature gate switched on for the controller. The gate is alpha and off by default, so out of the box cert-manager renews on its own schedule, and a revocation does not reach the cluster until the next scheduled renewal. The gate goes in the controller's feature gates, for example this Helm value:

```
config:
  featureGates:
    ACMEUseARI: true
```

Not yet run against Ducks in a Row: renewal, and ARI with cert-manager.

The server limits requests per source address, and a cluster usually reaches it through one address, so every Issuer in it shares one budget. A burst of new certificates can be refused with `rateLimited`; see rate limits partition per source address.

OpenShift

Nothing on this page has been run on OpenShift, and nothing about OpenShift itself has been verified against Ducks in a Row. The Issuer and Certificate above are standard cert-manager resources, and Red Hat ships cert-manager as the cert-manager Operator for Red Hat OpenShift; see Red Hat's documentation for the operator. Two points are documented upstream and have not been run here:

- **HTTP-01 through the built in router.** Since cert-manager 1.18, the solver's temporary Ingress uses `pathType: Exact`. OpenShift converts an Ingress into a Route for its router and skips a path of that type, so the challenge URL is never served and the challenge fails with a 503. cert-manager's compatibility notes give the fix, switching off the `ACMEHTTP01IngressPathTypeExact` feature gate, and say the Red Hat operator has had that gate off by default since its 1.18.0 release. Documented upstream, not run here.
- **Routes.** cert-manager itself does not act on OpenShift Routes. The separate openshift-routes project issues certificates for Routes from any cert-manager issuer, and warns that it is not designed for clusters shared between tenants. Documented upstream, not run here.

Troubleshooting

Symptom	Likely cause	What to do
The Issuer never becomes ready: `x509: certificate signed by unknown authority`	cert-manager does not trust your root	Set `caBundle`; see trusting the ACME endpoint
`x509: certificate relies on legacy Common Name field`, or a SHA-1 complaint	The server's certificate names its host only in the common name, or the chain is signed with SHA-1	Let the server enrol its own certificate, and move the CA to SHA-256
`externalAccountRequired`	The mode is Required and the Issuer has no binding	Add `externalAccountBinding` and the secret

Symptom	Likely cause	What to do
`failed to decode external account binding key data`	The secret is not base64url, for example standard base64 with +, / or = in it	Store the key exactly as the dashboard shows it
403 `unauthorized`: `External account binding signature verification failed.`	The key cert-manager holds is not the one issued: encoded a second time, stored with a trailing newline, or from before a regenerate	Recreate the secret from the key as shown, with `--from-literal`; see the EAB secret
403 `unauthorized` naming a credential	The credential is revoked, expired, or cannot be decrypted on this server	See troubleshooting
`rejectedIdentifier`	A name is outside the allowed domain list or the credential's domain namespace	Add the domain, or order within the namespace
403 "not enabled for ACME"	The template is not enabled for ACME	Enable it in the setup wizard, or point the Issuer at one that is
`serverInternal` with `Denied by Policy Module`	The key does not suit the template	Match `privateKey` to the template; see key type
A challenge stays pending and the server log shows no attempt	cert-manager's own check cannot fetch the token, often because the cluster cannot resolve the name	Make the zone resolvable in the cluster, or use `waitInsteadOfSelfCheck`
The Challenge fails soon after cert-manager accepted it, and the order fails with it	The server's HTTP-01 attempt failed	Read the reason on the Challenge or in the server log, fix it, then `cmctl renew`; see below
`rateLimited`	Many certificates at once through one source address	Retry later, or see rate limits

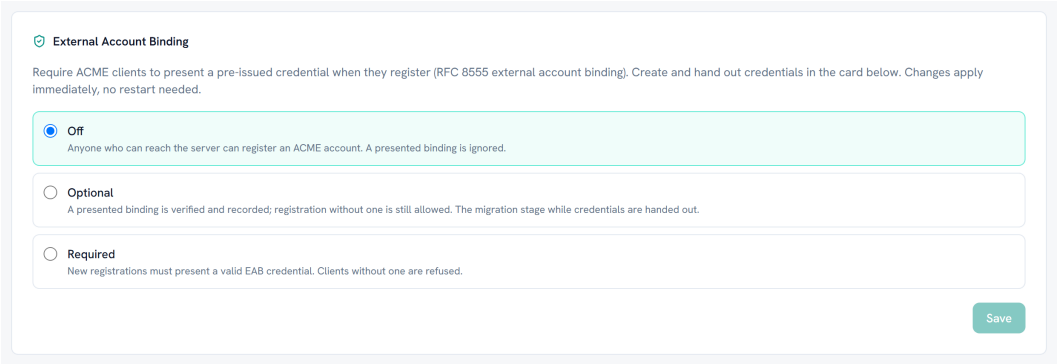
A failed HTTP-01 check fails the order. When the server's HTTP-01 attempt fails, the server marks that challenge invalid and, as RFC 8555 section 7.1.6 requires, the authorization and the order invalid with it, even though the other challenges it offered for the same name, DNS-01 and TLS-ALPN-01, were never answered. cert-manager waits on the authorization, not the challenge, so it sees the failure as soon as the attempt ends. The reason travels on the failed challenge, and the server's log carries it too, in a warning of the form `Challenge <id> (http-01) for <name> failed: <reason>`, where the reason is something like a refused connection or `server returned HTTP 308`. A failed order cannot be retried, so once the cause is

fixed, `cmctl renew` starts a new one. The cert-manager side of this is read from its code and has not been run.

13. External Account Binding

External account binding (RFC 8555 section 7.3.4) decides **who may register an ACME account** with your server. This page is the administrator's half. For the client flags, see connecting ACME clients.

Administration lives on the **ACME** tab of the dashboard.



The External Account Binding card on the ACME page, showing the three enforcement modes with Off selected

Enforcement modes

Mode	What a registering client needs
Off	Nothing. A presented binding is ignored. The default for an install upgraded from a version without this setting
Optional	Nothing, but a presented binding is verified and recorded
Required	A valid credential. Registration without one is refused with `externalAccountRequired`. The default for a new install

A new install requires binding. The setup wizard's Client Access step arrives with it switched on, and switching it off there is an explicit choice with a one line warning. Leave it on: in an Active Directory domain an ordinary user can create the DNS records a challenge asks for, so without binding a passed challenge proves little about who is asking (see hardening). The cost is one step before the first certificate, issuing a credential, which the quickstart walks through.

The mode applies immediately when you change it. There is no restart, and an upgrade from a version without this setting reads as Off, so nothing changes underneath an existing install. Re-running the setup wizard on a configured server keeps the mode it already has, and shows it on the Client Access step.

Raising the mode to Required does not lock out accounts that already exist. Accounts registered before the change keep working, and the ACME page lists them as Unbound so you can see which they are. If you want them gone, deactivate them deliberately; nothing does it for you.

Issuing a credential

A credential is a **key id** and an **HMAC key**. The key id is 32 hexadecimal characters. The HMAC key is sized for HS256, which is what every common client signs with by default, and HS384 and HS512 are accepted too.

The HMAC key is shown exactly twice in its life: when you create it, and when you regenerate it. It is not in any list, any error body, or any log line, and the API will not return it again. Hand both values to the client operator at the moment you create them.

The dashboard writes ready to paste client setup for you. The panel shown right after create or regenerate has the real secret inlined; each credential row's **Client setup** action shows the same snippets later with a placeholder where the secret goes.

Revoking, expiring, and the difference

Revocation is terminal. A revoked credential refuses edit, regenerate and owner change alike. There is no un-revoke. If you need the holder back, issue a new credential.

Expiry is not terminal. An expired credential is suspended, not dead. Move the expiry date forward, or clear it, and it works again.

Both suspend the accounts bound to them, and they do it at two points: when a new order is created, and at finalize. A suspended account gets 403 `unauthorized`.

The order itself stays in the `ready` state when this happens. That is deliberate: if you un-suspend the credential, the client can retry the same order rather than starting over.

Domain namespaces

A credential can carry a domain namespace, which limits what the accounts bound to it may order, across every template.

A namespace **narrows within the global allowed domain list, it never widens past it.** The global list is the ceiling and is checked first, so putting a domain in a namespace that the global list does not permit does not make it orderable. If you want a credential to reach a new domain, add it to the global list first.

A credential with no namespace is unrestricted, subject to the global list like any other.

An order outside the namespace is refused with `rejectedIdentifier` and a message naming the credential, and the refusal is recorded on the dashboard activity feed labelled as an EAB refusal, so it is distinguishable from a global allowed domain refusal.

Linking a credential to a person or a machine

A credential can record which Active Directory principal it was issued to. Users, computers, groups and service accounts can all be picked, including group managed service accounts, which are the usual identity for an automated ACME client.

This link is a record, not a control. Nothing checks it at issuance time. It exists so that six months later you can answer "who has this credential" without consulting a spreadsheet.

Moving the data folder breaks the secrets, on purpose

Credential secrets are encrypted with the machine's Data Protection keyring, which lives in `keys\` inside the data folder and is tied to the machine.

Restore the data folder onto a different machine and the secrets can no longer be decrypted. Verification then **fails closed**: clients get 403 `unauthorized` and the log says to regenerate.

This is the intended behaviour, not a defect. It means a copy of your data folder is not a copy of your credentials.

Recovery is straightforward, because everything except the secret survives the move. The key id, name, expiry, namespace and owner are all stored in the clear:

1. Open the ACME page on the new machine. Your credentials are all listed.
2. Regenerate each one. **The key id stays the same**, so only the HMAC key has to be redistributed.
3. Give each client operator the new HMAC key.

See backup and restore for how to avoid the situation.

EAB and device attestation do not mix

The Apple ACME payload has nowhere to put EAB credentials, so a device using `device-attest-01` cannot present one.

The consequence is concrete: **with enforcement set to Required, Apple devices cannot register at all.** That is not a bug and it is not fixable at this end.

Separate templates do not get round this. The enforcement mode is one setting for the whole server, not one per template, and an ACME account works on every template's directory, so Required refuses a new device on every template alike. An account a device registered before the mode was raised keeps working, but any new registration is refused, a device enrolling again included. If you use device attestation, keep the mode at Off or Optional; the device attestation allowlist, not EAB, is the gate for device orders. See device attestation.

14. Device Attestation

Device attestation lets a hardware backed device prove its identity to the CA before it gets a certificate. Ducks in a Row implements the ACME `device-attest-01` challenge from draft-ietf-acme-device-attest-08, which extends RFC 8555 with two device identifier types and one attestation challenge.

In v1 the only supported attestation format is `apple`, produced by Apple Managed Device Attestation (MDA). An Apple device managed by your MDM proves, with a key that never leaves its Secure Enclave, that it is the specific device named in the order. Ducks in a Row verifies that proof against Apple's own attestation root before it asks ADCS to issue.

The feature is **off by default and fails closed.** Until an administrator creates a device attestation profile on a template, that template does not offer `device-attest-01` at all, and an order for a device identifier is refused as if the feature did not exist. Nothing you do elsewhere in the product turns this on by accident.

How it works

A device order carries a **permanent-identifier** rather than a `dns` name. Its value is the device serial number (or the UDID), compared octet for octet.

1. The MDM pushes an ACME certificate payload to the device, pointing it at one of your template directories.
2. The device orders a certificate for its own permanent-identifier. If the template has a profile and the identifier is allowed, the order gets a single `device-attest-01` challenge; otherwise the order is refused (see Gate modes).
3. The device builds an attestation object with its Secure Enclave key and posts it to the challenge. Ducks in a Row verifies it: the attestation chains to the Apple root, the freshness nonce matches this challenge, and the attested serial equals the order identifier.
4. At finalize the server binds the certificate request to the attestation: the CSR public key must be the exact key that was attested, and the CSR identity must match the order under the profile's binding mode.
5. The request goes to ADCS on the profile's template, unchanged from any other ACME issuance.

The security model is a three way binding: a trusted attestation authority (Apple) vouches for the device, the attested key equals the key in the CSR, and the attested device identifier equals the order identifier. All three must hold.

Enabling it

Everything is configured on the **ACME** tab of the dashboard, in the **Device attestation** card. No wizard step and no configuration file entry is involved, so a setup wizard re-run never changes it.

1. Confirm the template you will use meets the ADCS requirements below and is published and ACME enabled.

2. Create a **profile** on that template. Pick a gate mode and a CSR identifier binding; the defaults (allowlist, cn-or-san) are the safe starting point.
3. Add the devices you will issue to. Each **allowlist entry** is one serial number (or UDID). An allowlist entry is required in the default gate mode: an empty allowlist issues to nobody.
4. Point the MDM payload at the template's directory URL and deploy it to those devices.

Every change applies immediately to in flight orders. There is no restart and no cache to wait on.

Gate modes

A profile decides which devices reaching the template may enrol.

Mode	Behaviour
Allowlist (default)	Only devices whose serial or UDID is on the profile's allowlist may order. A device not on the list is refused. This is the fail closed default.
Open	Any device that passes attestation may order, with no allowlist check. This is an observation mode for a controlled fleet, not a setting to leave on in production.

Why allowlist is the default. Any genuine Apple device that can reach the template directory can attest its own real serial number. Without an allowlist, `open` mode would issue to any managed device that finds the URL. The allowlist is what limits issuance to the devices you decided on.

A **disabled** profile behaves exactly like no profile: the template stops offering `device-attest-01` and device orders are refused invisibly. Disabling is the clean way to pause a template without deleting its allowlist.

Invisibly means the refusal names no device concept and reads exactly like the refusal for an identifier type the server does not know, whatever the client sends: a well formed serial, an empty value, an oversize one, and a device order mixed with a `dns` one all get the same answer. What that hides is whether **this template** takes device orders. It does not hide that the product implements the draft, which the version and this page already say.

CSR identifier binding

The Apple ACME payload cannot put the device identifier where the draft's strict mode expects it. Apple CSRs carry only RFC822, DNS, and URI subject alternative names, never the RFC 4043 PermanentIdentifier `otherName`. Deployed practice is to put the serial in the subject common name instead. The binding mode chooses how strict the server is about where the identifier appears in the CSR.

Mode	The CSR must carry the identifier as
cn-or-san (default)	the subject common name **or** a PermanentIdentifier SAN. This is the mode Apple clients work with.

Mode	The CSR must carry the identifier as
san-required	a PermanentIdentifier SAN only. Strict draft mode. Apple CSRs cannot satisfy this; use it only for clients that emit the `otherName`.
none	the CSR need not carry the identifier at all (privacy mode). The attestation still binds the device; the certificate simply does not restate the serial.

Two checks always run and are not configurable: the CSR public key must equal the attested key, and any `dns` SAN on a device CSR is refused. A wrong or unexpected value anywhere always refuses rather than issuing something loosely matched.

The Phase 0 lab probe confirmed ADCS passes a PermanentIdentifier `otherName` from the client CSR into the issued certificate untouched on an enrollee-supplies-subject template, with no CA wide flags. So `san-required` is honest on ADCS for a client that can produce the `otherName`; Apple clients stay on `cn-or-san` because their CSRs cannot. **Do not** enable the CA wide `EDITF_ATTRIBUTESUBJECTALTNAME2` flag to work around this; it is a well known CA security hazard and is never required here.

ADCS template requirements

The template a profile points at is issued through the same ADCS path as any other ACME certificate, so it must:

- **Allow the enrollee to supply the subject.** The device identity lives in the CSR (the CN or the PermanentIdentifier SAN). A template that builds the subject from Active Directory instead will not carry the device identifier.
- **Carry the Client Authentication EKU** (`1.3.6.1.5.5.7.3.2`) if the certificate is for device authentication, plus any other EKU your use needs.
- **Be published on the CA and ACME enabled** in Ducks in a Row, the same as a template used for `dns` issuance.
- **Match the device key.** Apple hardware bound keys are ECDSA on the Secure Enclave (P-256). The template's key requirements must accept that key, or ADCS refuses at finalize with `Denied by Policy Module`.

The Apple MDM payload

Managed Device Attestation is driven by an ACME certificate payload (`com.apple.security.acme`) that your MDM delivers to the device. The settings that matter for attestation are:

Payload setting	Value
Directory URL	the template's directory, `https://<server>/acme/<template>/directory`
Client Identifier	the device serial number (what you put on the allowlist)

Payload setting	Value
Attest	`true` (request an attestation)
Hardware Bound	`true` (generate the key in the Secure Enclave)
Key Type	`ECSECPrimeRandom` (the Secure Enclave key type)
Key Size	`256`

Use your MDM's own interface or Apple's configuration profile reference for the exact key names and capitalisation; MDM vendors surface these differently. `Attest` and `Hardware Bound` must both be on, or the device produces an ordinary key with no attestation and the challenge cannot be satisfied.

The `Client Identifier` the payload sets becomes the order's permanent-identifier and must match an allowlist entry exactly.

External account binding does not apply

EAB (external account binding) cannot gate Apple device orders, because **the Apple ACME payload has no place for EAB credentials.** A device managed by your MDM cannot present an HMAC key id.

This has a concrete consequence: if EAB enforcement is set to **Required**, Apple devices cannot register an account and cannot enrol at all. That is expected, not a bug. A new install starts at Required, so lower it to Optional on the ACME page before you turn device attestation on. The device attestation allowlist, not EAB, is the gate for device orders. The mode applies to the whole server rather than to one template, so putting devices on a template of their own does not help: keep EAB at Off or Optional wherever device attestation is in use. An account a device registered before the mode was raised keeps working, but any new registration is refused, a device enrolling again included.

Renewal and Apple rate limits

Apple rate limits fresh attestations per device. A device that attests too often in a short window is told to wait, so treat each attestation as a scarce resource:

- **Prefer longer certificate lifetimes.** Aim for 90 days or more so renewals are infrequent. A short lifetime that forces daily reattestation will hit Apple's limit.
- **The server never retries a spent attestation.** An attestation object is consumed once. Attestation failures are terminal for that challenge, not transient, so there is no server side retry loop burning through the device's budget.
- **The gate runs before the attestation.** A device that is not on the allowlist is refused at newOrder, before it spends an attestation, so a misconfigured fleet does not exhaust its budget against a closed gate.

Trust anchors and Apple root rotation

The Apple Enterprise Attestation Root CA ships embedded in Ducks in a Row, pinned by SHA-256 fingerprint and verified at load. It is shown on the trust anchor panel as a read only built in anchor. Verification fails closed if the embedded copy ever fails its pin.

Custom trust anchors are **additive**. You can add an anchor (by pasting its PEM on the trust anchor panel), but a custom anchor can never shadow, disable, or override the pinned Apple root. Adding an anchor makes the server trust attestations that chain to it, so only an administrator can add or remove one, and a certificate that equals a built in root is refused rather than stored as a second copy.

Two things custom anchors are for:

- **A test or synthetic attestation authority**, so a lab rig can drive the flow without a real Apple device.
- **Bridging an Apple root rotation.** When Apple rotates its attestation root:
  1. Obtain the new root and add it as a custom trust anchor. Devices attesting under either root now verify.
  2. Update to the Ducks in a Row release that pins the new root as the built in anchor.
  3. Remove the custom anchor once every device has moved to the new root.

Where refusals show up

A device order refused by the gate is recorded and surfaced on the dashboard activity feed as **blocked by the device attestation policy**, distinct from a domain allow list or EAB namespace refusal so you know to look at the device attestation card. Successful `device-attest-01` validations are counted in the Validation Methods widget alongside the `dns` challenge types.

15. Revocation

Ducks in a Row can revoke a certificate from the dashboard, and an ACME client can revoke its own. What the dashboard may touch is deliberately narrow, and it is narrowed by two independent things: a ceiling you cannot raise, and a scope you choose.

The ceiling you cannot raise

Before any setting is consulted, a certificate must clear a capability ceiling. It is revocable from the dashboard only when all of these hold:

- It has an extended key usage extension, and that extension is **not empty**.
- Every purpose in it is server authentication or client authentication, and nothing else.
- Its key usage does not include certificate signing or CRL signing.
- It is not a CA certificate.

A certificate whose capability cannot be determined at all, because there is no stored copy of it and no parsed detail, is refused rather than allowed. The ceiling fails closed.

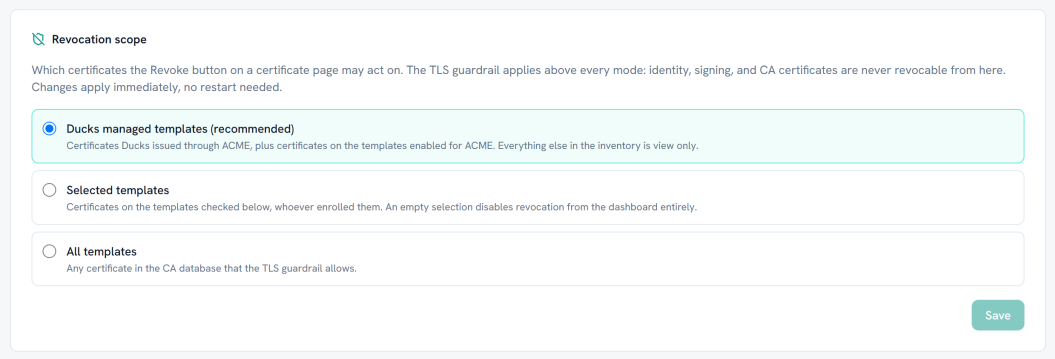
No administrator setting widens this. There is no configuration key, no break glass flag, and no dashboard control that lets the ceiling through. It is a subset rule, not a blacklist: a purpose nobody anticipated is outside the allowed set by construction, so it is refused without anyone having to think of it in advance.

The reason is blunt. A tool that proxies certificate requests should not be able to revoke a domain controller certificate, a code signing certificate, or the CA's own certificate. Those are exactly the revocations that take an organisation down, and none of them is a TLS certificate.

A certificate with no EKU at all is refused. An empty or absent EKU extension means valid for every purpose, which is the opposite of a narrow TLS certificate. It reads as unlimited, so the ceiling treats it that way.

The scope you choose

Underneath the ceiling, `revocationScope` decides which of the remaining certificates the dashboard offers to revoke. Set it on the Settings page.



The Revocation scope card on the Settings page, showing the three scope modes with Ducks managed selected

Mode	What the dashboard may revoke
`ducks-managed`	The default. Certificates Ducks in a Row issued, plus anything on a template you enabled for ACME
`custom`	Only certificates on the templates you list
`all`	Anything that clears the ceiling

Ducks managed is a union of two tests, and either one is enough: the certificate has an ACME record bridging it to a request this server made, or its template is one you enabled for ACME. The first covers certificates this server obtained. The second covers certificates on the same templates that something else obtained, which is usually what an administrator means by "ours".

`custom` with an empty template list disables dashboard revocation entirely. That is a legitimate configuration, and the startup log says so plainly rather than leaving you to wonder.

`all` is logged as a warning at every start. It is not forbidden, but it is worth being deliberate about.

Template names are matched in both forms. A template can be recorded under its programmatic name or its display name, so the match tries the stored form first and then maps display name to programmatic name through the list the CA publishes. If the CA is unreachable, only the raw match is available, so the scope narrows rather than widens. That is the safe direction.

`Certus:Acme:ExposeAllTemplates` does **not** widen revocation. The scope reads the recorded template selection directly rather than going through the ACME exposure policy, precisely so that an ACME break glass stays an ACME break glass and does not quietly become a revocation one.

What the CA has to allow

Every revocation the product makes goes to the CA as the server's machine account, so all three kinds need that account to hold **Issue and Manage Certificates** on the CA: revocation from the dashboard, an ACME client revoking its own certificate through `revoke-cert`, and the one automatic revocation described below. That is a higher privilege than the **Read** the inventory needs, and than the **Request Certificates** on the CA and **Enroll** on the template that issuance needs.

Granting it is a real decision, because it also lets the account approve and deny requests the CA holds. Without it, issuance, the inventory and alerts keep working, and all three kinds of revocation fail with a permission error naming the right that is missing. That includes a client revoking a certificate whose key it believes is compromised. Grant it if you want any of the three. Either way, the ceiling and the scope are defence in depth on top of the CA's own permissions, not a replacement for them. What to ask your AD and CA team lists it with the other rights.

ACME clients revoking their own certificates

An ACME client can revoke a certificate it holds through the standard `revoke-cert` endpoint, authorised the way RFC 8555 section 7.6 requires: either from the account that ordered it, or by proving possession of the certificate's own private key.

This path is not subject to the scope modes. A client revoking its own certificate is responding to something it knows about, usually a key compromise, and that must not wait on a dashboard setting an administrator chose months earlier. It only ever reaches certificates Ducks in a Row issued, so the blast radius is already bounded. It does need the same CA right as the dashboard; see what the CA has to allow.

The template name in the `revoke-cert` URL is not checked either. A client can revoke through the URL of any template: the one the certificate was issued from, another one, or one that is no longer offered over ACME. What authorizes the request is the account key or the certificate's own key, and the certificate sent has to be byte for byte the one this server issued. Checking the name would add nothing to that, and it would let a template setting stand between a compromised key and its revocation. The order, authorization, challenge, certificate, account and key-change URLs treat the name the same way, while `renewalInfo` does check it; see which URLs check the template.

The one automatic revocation

There is exactly one place where the product revokes a certificate without being asked. At finalize, the certificate the CA returns is checked against the capability ceiling before it is handed to the client. A leaf that breaches the ceiling is revoked immediately, with reason code 5, and the order fails.

The revocation needs **Issue and Manage Certificates** on the CA, like every other. Without it the revocation fails and the certificate stays valid at the CA: the order still fails and the client still never receives it, and the service log names the certificate's serial number and CA request id and says to revoke it by hand.

That is the guarantee behind the advisory check at new order time: a template's recorded metadata is unverified and might be wrong, but the certificate itself is checked every time and cannot be wrong. If a template is misconfigured so badly that it mints something outside the ceiling, the client never receives it.

Before either check, the certificate must carry the public key from the order's own CSR. One that does not is not the order's certificate at all, so it is neither delivered nor revoked: the order fails, and the service log names the CA request id. The usual cause is an install pointed at a different CA while an order was waiting at the old one for a manager's approval. Every CA numbers its requests from 1, so the new CA's request of the same number belongs to someone else, and revoking it would break their service.

Nothing else revokes on its own. In particular, an order that is abandoned or expires while the CA is holding it does not revoke anything: it logs the CA request id at warning level and leaves the decision to an operator. Automatic revocation of something a human might still want is the more dangerous mistake.

Where refusals show up

Every refusal, whether by the ceiling or by the scope, is recorded on the dashboard activity feed with the reason. If a certificate you expected to be revocable is not offered, that feed says which of the two stopped it.

16. Expiry Alerts

Ducks in a Row watches every certificate in its inventory and tells you before one expires, by email or by webhook.

Thresholds

By default an alert fires 30, 14, 7 and 1 days before expiry. Each certificate gets at most one alert per threshold, so a certificate does not nag you hourly for a month. The monitor sweeps every 60 minutes.

Both are configurable: `Certus:Alerts:ThresholdDays` and `Certus:Alerts:CheckIntervalMinutes`. See configuration.

Revocation lists

Ducks in a Row also watches the certificate revocation lists of every CA in the chain of the connected CA, including the ones published by a CA above it.

This matters more than it sounds. A certificate expiring breaks one service. A CRL expiring breaks revocation checking for **everything that CA ever signed**, at once, on every client that checks revocation: domain controllers, VPN and wireless authentication, and the certificate authority's own service. An offline root's CRL is the dangerous one, because it expires on a schedule of its own, nothing automatic replaces it, and the root is switched off and easy to forget.

Where each CRL is read from

- The connected CA's own CRLs come from the CA itself, over the same connection everything else uses, and from the locations the CA tells its clients to read them.
- The CRL covering the CA's own certificate, which on a two tier estate is the offline root's, is read from the distribution points named on that certificate. Both `ldap://` and `http://` locations are read.

Every copy is checked against the certificate of the CA that should have signed it, and one that does not verify is ignored rather than trusted.

Each copy is watched separately, and that is the point. A root CRL renewal that is published to the directory but never copied to the web server leaves the web server serving a CRL that expires, and every client that reads the HTTP location fails while the domain looks healthy. Ducks in a Row warns about the stale copy, names the location, and says that a newer CRL already exists elsewhere.

When it warns

Which rule applies depends on who replaces the CRL:

The CRL is	What you get
Replaced by a CA on a timer (the connected CA's own, and any short lived one)	One warning when the CA misses its own scheduled publication, and one when the CRL actually expires

The CRL is	What you get
Published by hand (an offline root or policy CA, recognised by a CRL that outlives your widest threshold)	The same ladder as certificate expiry: 30, 14, 7 and 1 days by default, then one when it expires

The ladder deliberately does not run against a CRL a CA replaces on a timer. A Windows CA publishes a weekly CRL that is valid for about seven and a half days and replaces it with about twelve hours to spare, so the ordinary thresholds would fire several times a week on a CA doing exactly what it should.

A warning fires once per CRL. Publishing a new one resets the ladder, so a root CRL renewed in good time goes quiet again on its own.

When a CRL cannot be read

A distribution point that cannot be reached, or a CA that is down, does not silence anything: the warnings continue against the last copy that was read, and say when that was. An unreachable location is not evidence that the CRL behind it was renewed.

The **Revocation lists** card on the Settings page shows every CRL being watched, each place it is published, and what was last read from each.

Configuration

```
{
  "Certus:Alerts": {
    "Crl": {
      "Enabled": true,
      "CheckIntervalMinutes": 60,
      "FetchTimeoutSeconds": 15,
      "MaxCrlBytes": 33554432
    }
  }
}
```

The warning thresholds are the ones above, shared with certificate expiry: a site that wants sixty days of notice wants it for both.

Licence

With the paid module installed and a licence stored, Ducks in a Row also tells you before the licence runs out, through the same email and webhook channels. A lapse never touches a certificate: it switches paid features off, and every licence alert ends by saying so. See licence and paid module.

When it warns

The paid module decides which warning is due, and Ducks in a Row sends it.

Warning	When	Paid features
Expiring	30, 14, 7 and 1 days before the licence ends, counting only those inside the licence's warning period	On
Grace	When the licence ends, if it has a grace period	On until the grace period ends
Lapsed	When the grace period ends, or when the licence ends if it has none, as a trial does	Off

- **Each warning is sent once per licence.** A renewed licence carries a new licence id, so its warnings start again from the top. Restoring the previous licence repeats nothing.
- **Only the warning due now is sent.** A server that was switched off at 30 days and starts again with 10 left receives the 14 day warning, not a backlog.
- **Every configured channel receives it, in full mode and in monitor mode.** `Certus:Alerts:Enabled` switches the certificate checks only, and there is no setting that turns licence alerts off.
- **With no channel configured nothing is recorded**, so a channel you add later still receives the warning that is due.
- **Some licences send nothing.** A licence bound to another forest switches nothing on here, so it sends no warnings, and the banner says why. Neither does a licence the paid module cannot use, or one that has not started yet.
- **The paid module has to be running.** If it has failed or stopped responding, nothing is sent until it runs again, and the Licence card says so.

The check runs every 5 minutes, and the paid module looks at the licence at least once an hour, so a warning arrives within about an hour of its moment.

An email is marked NOTICE at 30 and 14 days and WARNING after that. It is never CRITICAL, which is kept for alerts about certificates.

A webhook payload looks like this:

```
{
  "event": "licence.expiring",
  "timestamp": "2026-12-24T09:00:00Z",
  "licenceId": "0192a3b4-c5d6-7e8f-9a0b-1c2d3e4f5a6b",
  "tier": "standard",
  "stage": "days7",
  "validUntil": "2027-01-01T00:00:00Z",
  "graceUntil": "2027-01-31T00:00:00Z",
  "daysRemaining": 6
}
```

- `stage` is one of `days30`, `days14`, `days7`, `days1`, `expired` and `grace_ended`.
- `daysRemaining` counts whole days to the end of the licence, or during grace to the end of grace. It is left out once the licence has lapsed.

- `tier` is left out when the paid module reports something that is not a tier name.

The licensee's name is never part of a licence alert, in the email or in the payload. The licence id is the reference to quote when you ask about a renewal.

Email

The whole SMTP transport can be set from the **Alerts** card on the Settings page: relay host, port, transport security, username and password, sender address and name, and the recipient list. That is the recommended route. It needs no file editing, and the password is stored encrypted in the data folder rather than sitting in a configuration file in the clear.

The file form remains available for scripted or unattended setup:

```
{
  "Certus:Alerts": {
    "Enabled": true,
    "CheckIntervalMinutes": 60,
    "ThresholdDays": [30, 14, 7, 1],
    "Smtp": {
      "Host": "smtp.yourdomain.local",
      "Port": 587,
      "TlsMode": "starttls",
      "Username": "ducks@yourdomain.local",
      "Password": "your-smtp-password",
      "FromAddress": "ducks@yourdomain.local",
      "FromName": "Ducks in a Row",
      "Recipients": ["admin@yourdomain.local"]
    }
  }
}
```

`TlsMode` accepts `none`, `starttls` or `implicit`. Leave it out and the port decides: 465 means implicit TLS, anything else negotiates mandatory STARTTLS.

A null `Smtp` block means email alerts are off. That is the shipped default.

Saving from the dashboard changes who owns the setting. The Alerts card writes into the settings file in the data folder, which outranks `appsettings.json`. After that, editing those keys in `appsettings.json` has no effect. The card lists the keys it now owns, so the situation is visible rather than puzzling. An environment variable or command line argument still outranks both, and where one is in use the card disables that field and says why instead of accepting an edit that could never take effect.

A password saved from the dashboard is protected with the machine's Data Protection keyring. It never appears in any file in readable form and is never returned by the API. Restoring the data folder onto a different machine makes it undecryptable by design, so save it again on the new machine. See backup and restore.

Webhooks

A webhook alert is an HTTP POST carrying a JSON payload, signed with HMAC-SHA256 in the `X-Certus-Signature` header so the receiver can verify it came from you.

```
{
  "Certus:Alerts": {
    "Webhook": {
      "Url": "https://hooks.example.com/services/YOUR/WEBHOOK/URL",
      "Secret": "your-hmac-secret",
      "Headers": {
        "X-Custom-Header": "value"
      }
    }
  }
}
```

A null `Webhook` block means webhook alerts are off, which is the shipped default.

Every alert carries an `event` name, so a receiver can route on it:

Event	Sent when
`certificate.expiring`	One or more inventory certificates crossed a threshold
`crl.expiring`	A revocation list published by hand crossed a threshold
`crl.overdue`	A CA did not replace its own revocation list on schedule
`crl.expired`	A revocation list is past its next update
`server.certificate.renewal_failed`	Automatic renewal of this server's own certificate failed
`licence.expiring`	The licence reached its 30, 14, 7 or 1 day warning
`licence.grace`	The licence ended and its grace period began
`licence.lapsed`	The licence lapsed: its grace period ended, or it had none
`test.alert`	The test button on the Alerts card

A CRL payload names the issuing CA, whether it is a base or a delta CRL, its number, when it expires, and every location it is served from. When a newer CRL is published somewhere else it carries that number too, which is what tells a receiver that a renewal reached one location and not another.

A server certificate payload carries the outcome, the template, the days left on the certificate the server is still serving, and the reason in `detail`. Ducks in a Row never adds the service's computer account or its domain to that reason, which otherwise repeats the certificate authority's own words. The alert email does name them, because it goes to your own recipients and that is the account to grant Enroll to; a webhook often goes to a service outside your network.

The webhook block is deliberately not editable from the dashboard. It is always read from the file and never written by the Settings page. An alert destination is a place your certificate inventory gets described to, so changing it is a file edit and a service restart, not a form field.

A saved change needs a restart

Alerting reads its settings once, when the service starts. A change saved on the Settings page is stored but not in force until you restart, and the card says so rather than pretending otherwise. It offers a restart button for exactly this.

Alert history

The Alerts card shows whether monitoring is on, how often it checks, the thresholds in force, who is on the recipient list, whether a webhook is configured, and the most recent alerts including any that failed to send. A failed send is worth looking at: an alert that could not be delivered is indistinguishable, from the outside, from a certificate that never needed one. Each alert says which channels delivered it and which failed, with each failed channel's error, so a webhook failure beside a delivered email reads as "Partly sent". A failed alert recorded by an earlier version says only that at least one channel failed.

The same data is available from the API at `GET /api/alerts/history` and `GET /api/alerts/summary`.

Licence alerts are not listed on the Alerts card. The service log records each one, with the licence id and the channels it went to.

Testing it

The Alerts card has a test send. Use it after any change to the transport: a relay that refuses the service's credentials will refuse them at 3am on the day a certificate expires just as readily as it does during a test, and the test is the cheaper way to find out.

17. Licence and Paid Module

Ducks in a Row is free to use. The paid tier is switched on by a licence file, which a separately installed paid module checks. The Licence card on the Settings page is where you install the file and see what the paid module makes of it.

Certificate issuance and renewal never depend on the licence or on the paid module. A missing, expired or damaged licence, or a paid module that fails to start, changes nothing about the certificates Ducks in a Row issues and renews. Every warning the card and the banner show ends by saying so.

The free tier is the product as released. One installation connects to one certificate authority, and an estate with more than one issuing CA runs one installation per CA, as several certificate authorities describes. The certificate inventory, the dashboard, expiry alerts, CRL watching and monitor mode never depend on the licence either, and nothing a release shipped on the free tier moves behind the licence later. The paid tier adds capabilities beside the core, such as covering more than one certificate authority; ducksinarow.dev lists the tiers.

What you need

- **The paid module.** It comes with its own installer, which installs whatever the state of your licence; see security updates and your licence. Until it is installed, the card reads "Free tier. No paid module is installed."
- **A licence file** from Haruspex Systems. It is a small text file naming your organisation, your tier, how many CAs it covers, and its dates.

The order does not matter. A licence installed before the paid module is kept, and checked as soon as the module runs.

Before it loads the paid module, the service verifies that every file of the module carries a valid, timestamped Authenticode signature by Haruspex Systems B.V., and a module whose signature it cannot verify is not loaded; the Licence card then names the file.

Security updates and your licence

Every release of the paid module can be downloaded and installed whatever the state of your licence: valid, expiring, lapsed, or none at all. That includes its security releases, which are free of charge, as every security fix for Ducks in a Row is. The licence only switches paid features on. It never decides what you may install, so a lapsed licence changes nothing about which release of the module you can run, and the module's installer does not ask for one. Where the module's releases are published is announced with its first release.

The paid module is a product of its own, with its own support period and its own security policy. Its support period is published with its first release. It is not the 31 December 2031 date that applies to Ducks in a Row itself, which SECURITY.md states; until the module's own date is published, do not read that date as covering the module.

Each release of the module is built against a module contract that a release of Ducks in a Row carries. A module whose contract does not fit the installed release does not load, as described below, and the installer warns before an upgrade of Ducks in a Row that would cause it. So a security release of the module may need a newer release of Ducks in a Row, and that upgrade is free as well.

Installing a licence

1. Open **Settings**. The Licence card comes straight after the About block.
2. Choose **Upload a licence**, or **Replace the licence** when one is already installed. Drop the file on the card, choose it, or paste its text.
3. The card asks the paid module to check the file and shows what it found: who it is licensed to, the tier, how many CAs it covers, its dates, the forests it is bound to, and the forest this server is in. Nothing is installed yet.
4. Choose **Activate**.

If the paid module judges the new file worse than the licence installed now (a lower stage, an earlier end date, fewer features or fewer CAs), the card shows the two side by side and asks you to confirm before installing it. The licence it replaces is kept, so **Restore previous licence** can undo the change.

A file the paid module cannot use is not offered for activation. The card says why, in plain words.

With no paid module running there is nothing to check the file, so the button reads **Store it** instead. The file is kept and checked once the module runs.

Pasting the text

Pasted text is used exactly as pasted. A licence file is one long line, and mail programs often wrap long lines, so a pasted licence that went through a mail program can arrive changed. The card then refuses it, says a mail program may have wrapped a long line, and suggests choosing the file instead: save the attachment and choose it.

What the card says

State	What the card says	Paid features
No paid module, no licence	Free tier. No paid module is installed.	Off
No paid module, licence stored	A licence is stored but not checked, because the paid module is not installed.	Off
Paid module, no licence	The paid module is installed. Upload your licence file.	Off
Valid	Licensed to (name) (reference). (Tier), up to (n) CAs. Valid until (date).	On

State	What the card says	Paid features
Expiring	Your licence expires on (date), in (n) days.	On
Grace	Your licence expired on (date). Paid features switch off on (date).	On
Lapsed	Your licence lapsed on (date). Paid features are off.	Off
Not yet valid	Your licence starts on (date).	Off
Bound to another forest	This licence is for (forests). This server is in (forest).	Off
Not usable	This licence file cannot be used, and why.	Off
Paid module not running	What happened to the module, and what to do. See below.	Off

Dates are shown in UTC, written year first, for example 2027-10-28. Day counts come from the paid module on the server. The dashboard never counts days itself, and reads the status again every hour, so a page left open overnight shows the server's count rather than a stale one.

The card also warns when this server's clock looks more than a day behind the date the licence was issued, and when the paid module does not know the licence's tier, which a newer paid module fixes.

Warning, grace and lapse

A licence carries its own warning and grace periods, 30 days each unless it says otherwise.

- **Expiring.** From the start of the warning period until the end date. Paid features stay on, and an amber banner shows the days left.
- **Grace.** From the end date until the grace period ends. Paid features stay on, and the banner turns red.
- **Lapsed.** After the grace period. Paid features are off, and a grey banner says so. You can dismiss it for the rest of the browser session.

A lapse deletes nothing and reconfigures nothing. Whatever the paid features hold is kept, and a renewed licence switches everything back on as it was.

With an email or webhook channel configured, Ducks in a Row also sends an alert at each warning inside the warning period, when the licence ends, and when it lapses. See licence alerts.

The banner

The banner runs across the top of every page, and links to the Licence card. It appears only when a licence is stored, so an installation on the free tier never sees one.

- **Amber:** the licence is expiring, has not started yet, or is stored while no paid module is installed.
- **Red:** the licence is in grace, cannot be used, or is bound to another forest; or the paid module is not running; or the licence folder cannot be trusted.

- **Grey:** the licence has lapsed. Only this one can be dismissed.

Removing and restoring

Remove the licence asks first, then switches paid features off. The licence it removed is kept as the previous one.

Restore previous licence puts the previous licence back. It undoes a removal or a replacement. One previous licence is kept.

When the paid module is not running

The card says what happened, and shows the service's own reason beneath it.

- **It failed to start, stopped responding, or was set aside after failing to start twice.** Choose **Retry the paid module**. The service restarts to try again, and the dashboard and ACME clients lose their connection for a few seconds. If the service cannot restart itself, the card says so: restart it by hand with `Restart-Service DucksInARow`.
- **It was built for another release.** The card names the contract version the module was built for and the one this release needs. Install the paid module built for this release, then restart the service. Retry is not offered, because a restart alone cannot change it. Setup warns before an upgrade that would cause this; see installation.
- **It was not loaded, because non-administrators can change its folder.** Repair the paid module's installation, then restart the service. Retry is not offered here either.
- **Its signature could not be verified.** The card names the file: it is not signed, was signed by someone other than Haruspex Systems, or was changed after it was signed. Reinstall the paid module from a release Haruspex Systems published, then choose **Retry the paid module**.

The paid module is looked for only when the service starts, so after installing, repairing or removing it, restart the service with `Restart-Service DucksInARow` if its installer did not.

- **More than one module folder is present.** Only one paid module may be installed. Remove the extra folder from the `modules` folder in the program folder, then restart the service.

Forest binding

A licence can be bound to one or more Active Directory forests. A bound licence switches paid features on only on a server joined to one of them; on a server in another forest, or one not joined to a domain, it switches nothing on. A licence with no binding works on any server, and that is the default.

Once the paid module is installed, the card shows the forest this server is in, so you can quote it when you buy a bound licence. The paid module reads it from the server itself and asks no domain controller, so a domain controller outage does not change the result.

Installing a licence without the dashboard

An administrator can copy a licence file to `licence.json` in the `licence` folder of the data folder, by default `C:\ProgramData\Ducks in a Row\licence\licence.json`, for example from a script or by group policy. The service picks it up within about a minute.

This way skips the downgrade check and keeps no previous licence, so prefer the card when you can. The service creates the `licence` folder the first time a licence is installed from the card.

Do not create the `licence` folder by hand. The service refuses a licence folder that anyone other than SYSTEM and Administrators owns or can write to, and a folder you create may be owned by your own account.

The licence folder cannot be trusted

If the card says the licence folder cannot be trusted, someone other than SYSTEM and Administrators owns it or can write to it. The service then reads no licence from it, and never repairs it, because resetting the permissions on a folder someone else prepared would itself be unsafe.

1. Find the folder the service log names. By default it is `C:\ProgramData\Ducks in a Row\licence`.
2. Delete it.
3. Install the licence again from the card. The service creates the folder with the right permissions.

Backup and restore

The `licence` folder is part of the data folder, so it travels with every backup. An unbound licence works on any server it is restored to; a bound one works on a server in the same forest. See backup and restore.

18. Hardening

Ducks in a Row ships with defaults that favour first issuance on an internal network. This page collects the optional lockdowns and explains the reasoning behind each default, so an open posture is an informed choice rather than an accident.

The data folder

The installer restricts the data folder, `C:\ProgramData\Ducks in a Row` by default, to SYSTEM and Administrators, and lets Users only read and run the installation folder. Keep it that way. Whoever can write `settings.json` can name the group the dashboard admits, and whoever can write `ducks-setup.json` sets the ACME policy. The service refuses either file if anyone else may write it, which is a backstop, not a reason to widen the folder.

The lock also protects what has no such backstop: the database, which holds the ACME accounts, orders and EAB credentials, and the `keys` folder, which protects the EAB secrets and the saved SMTP password. That is why the database must stay in the data folder. The service refuses to start when `Certus:DatabasePath` points anywhere else, and re-applies the lock to the data folder on every start. To keep the data on another disk, choose that disk for the whole data folder at install.

The service account

The service runs as LocalSystem, which the CA and the directory see as the server's own computer account. A virtual service account (`NT SERVICE\DucksInARow`) would present the same computer account on the network and hold far fewer rights on the server, so it was evaluated on a test lab for release 1.1. It was not adopted:

- The Data Protection keyring is protected with DPAPI under the account the service runs as. Under another account the existing keys cannot be read, so every stored EAB secret and the saved SMTP password are lost on an upgrade unless the keyring is migrated.
- Renewing the server's HTTPS certificate writes to the machine's personal certificate store and restarts the service, and neither is open to a virtual account without widening the store's permissions and the service's own.
- The CA rights an attacker would inherit are the computer account's either way. What limits them is granting that account only the rights you need, as the CA rights below describe.

Do not change the account the service logs on as, in Services or with `sc.exe config`. Measured on that lab: the service stops starting and writes nothing to its log, because it can no longer apply the data folder's permissions; with those widened by hand, the stored EAB secrets can no longer be decrypted, HTTPS on port 5001 drops every connection when it serves a certificate from the machine store, until that certificate's private key admits the new account, and a certificate renewal fails after the CA has already issued the certificate. Changing back to LocalSystem does not undo it either. Every file the other account wrote is owned by that account, so the service then refuses a `settings.json` or `ducks-setup.json` written in between and runs as if it were absent, and any secret saved in between cannot be decrypted. That direction

was not measured.

What limits a compromise of the service instead: the locked data and installation folders above, the publisher check on a paid module before it is loaded, the administrator audit trail, and monitor mode on a CA you do not yet want it to write to.

The administrator audit trail

Every change an administrator makes through the dashboard or its API is recorded: who made it, when, what it touched, and where a setting changed, the value before and after. The Audit page shows the trail, newest first, with a filter by administrator. The record is the Windows identity the dashboard authenticated, so it is only as good as the sign in: keep the dashboard on Windows Integrated Authentication and the admin group small, and read NTLM and the dashboard sign in below.

Three things make the trail hard to silence. It has no setting that turns it off. Nothing an administrator can reach from the dashboard edits or deletes a row; the only thing that removes one is the daily retention pass, whose period is `Certus:Retention:AuditTrailDays` (two years by default, and a value below 90 days is raised to 90, so a short period cannot be used to hide a change). And every row is also written to the service log at Warning level, which a raised minimum level cannot drop, so the log file and the database are two records of the same action.

A change to a settings file is recorded before the file is written. If the write is then refused, a second row says so, so a change that did not land is never mistaken for one that did. A change to the database, such as erasing an account or revoking a credential, is recorded in the same transaction as the change, so the row and the change commit or roll back together.

The trail never carries a secret. A credential's MAC secret, an SMTP password and a licence file's bytes are described by what happened to them (created, regenerated, cleared) and never by their value.

The same page counts refused sign ins: requests to the API that offered credentials and were refused (401), and requests from a signed in account that is not in the admin group (403), per source address and per hour, so a password guessing attempt against the dashboard shows as a growing count against one address. A browser's first unauthenticated request, which is how Windows authentication always starts, is not counted, and nor is an administrator's own request that lacked the anti forgery header. The counts are kept for the same period as the trail.

NTLM and the dashboard sign in

The dashboard signs administrators in with Windows Integrated Authentication (Negotiate), which settles on Kerberos when it can and falls back to NTLM when it cannot. The two are not equal. A Kerberos ticket is issued for the name of the server the browser asked for and is useless to any other host. An NTLM exchange is not tied to a host at all, so an attacker who can get an administrator's browser to authenticate to them can relay that exchange to the dashboard and act as the administrator. The anti forgery header does not stop a relay, because the relayed request carries it. This is the attack class behind ESC8 against ADCS web enrolment, aimed one hop earlier at the tool that manages the CA.

Channel binding (Extended Protection for Authentication) is the usual answer and the service cannot offer it on this release. The web server is Kestrel, and its Negotiate handler on .NET 10 has no setting that requires a channel binding token and never passes one to the exchange; only HTTP.sys and IIS do. .NET 11 adds automatic channel binding to Kestrel, and the service will pick that up when it moves to that runtime. Until then the defence is to stop accepting NTLM, in three steps.

1. Make Kerberos work. The browser asks for a Kerberos ticket for `HTTP/<the host in the URL>`. The service runs as LocalSystem, so that name must be registered on the server's computer account. If administrators open the dashboard by the server's own name, the computer account's built in `HOST` name already covers it and nothing needs registering. If they use an alias, such as the external URL `https://certus.example.com`, register it from an elevated prompt on a domain member:

setspn -S HTTP/certus.example.com SERVER01$

where `SERVER01` is the server's computer name. `-S` refuses to add a name that another account already holds, which is the one thing that silently forces NTLM for everyone. An IP address in the URL always falls back to NTLM, because no ticket can be issued for one. Browsers also have to be allowed to hand out a ticket: Edge and Chrome do so for sites in the Local intranet zone or named in the `AuthServerAllowlist` policy; Firefox needs the host in `network.negotiate-auth.trusted-uris`.

2. Confirm which package each sign in uses. The service log names it, once per administrator and package since the last start, at Information:

Dashboard sign in by CORP\alice used Kerberos

A line that says `NTLM` means that administrator's browser could not get a ticket: the wrong name in the URL, a missing service principal name, a browser outside the intranet zone, or a workstation that is not domain joined. Fix those first, and watch the log until every administrator shows `Kerberos`.

3. Refuse NTLM. Set `Auth:RequireKerberos` to `true` in the settings overlay and restart the service. A sign in that used NTLM is then answered with a 401 and a problem document that says why, logged at Warning, and counted on the Audit page with the other refused sign ins. The setting ships off, because turning it on before step 1 locks every administrator out, and the service cannot yet tell you whether your name is registered.

Windows can also refuse NTLM for the whole server, through the Group Policy setting `Network security: Restrict NTLM: Incoming NTLM traffic` under Computer Configuration, Windows Settings, Security Settings, Local Policies, Security Options, set to `Deny all accounts`. Audit first with the matching `Audit Incoming NTLM Traffic` setting and read the NTLM operational log. Be clear about what it does: it applies to every service on that server, not just the dashboard, so file shares, remote management and anything else that a client reaches by IP address or without a registered name will stop authenticating too. `Auth:RequireKerberos` is the narrower switch and is the one to reach for first.

Challenge validation egress

For HTTP-01 and TLS-ALPN-01 challenges, the server itself is the party that connects out: it fetches `http://{host}/.well-known/acme-challenge/{token}` on port 80, or opens a TLS connection on port 443 and inspects the ALPN certificate. DNS-01 and device-attest-01 never connect to the validation target,

so nothing in this section affects them.

Because the validator runs next to the CA, those outbound connections are worth fencing. Out of the box:

Target	Range	Default
Loopback	`127.0.0.0/8`, `::1`	blocked
Link local, including the cloud metadata address `169.254.169.254`	`169.254.0.0/16`, `fe80::/10`	blocked
IPv6 unique local	`fc00::/7`	blocked
The literal name `localhost`		blocked
RFC 1918 private ranges	`10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`	**allowed**

Every address a name resolves to is vetted at connect time, so a record that flips between a public and a private address (DNS rebinding) does not slip through, and IPv4 addresses wrapped in IPv6 form are unwrapped before the check. Redirects are not followed by default, so a challenge server cannot bounce the validator to an internal address either.

Why the private ranges are open by default. An internal CA usually validates hosts that live on exactly `10.0.0.0/8`, `172.16.0.0/12`, and `192.168.0.0/16`; blocking them by default would break the product's primary use case on day one. The cost of the open posture: an ACME client with an account on your server can point the validator at internal hosts on ports 80 and 443 and learn from the timing whether something answers there. Validation only succeeds on the exact expected response, so nothing is read back, but it is an internal reachability oracle. Once nothing you validate lives on a private address, turn the flag below on and that oracle is gone.

Blocking the private ranges

If every name your CA validates resolves to a public address, set one flag in `settings.json` in the data folder (`C:\ProgramData\Ducks in a Row`):

```
"Certus:Acme:ChallengeValidation": {
  "BlockPrivateRanges": true
}
```

Or as an environment variable: `Certus__Acme__ChallengeValidation__BlockPrivateRanges=true`. Restart the service after either; the egress settings are read at startup.

With the flag on:

- An order that names a private IP address directly is refused at order time with `rejectedIdentifier`.
- A name that resolves to a private address fails HTTP-01 and TLS-ALPN-01 validation as a hard policy rejection. The challenge is marked invalid and is not retried, because nothing transient is wrong.

Blocking more ranges

`AdditionalBlockedCidrs` fences off anything else, in `address/prefix` form. The checks are additive: an address that any rule blocks is blocked. For example, to also cover the carrier grade NAT range:

```
"Certus:Acme:ChallengeValidation": {
  "BlockPrivateRanges": true,
  "AdditionalBlockedCidrs": ["100.64.0.0/10"]
}
```

An invalid CIDR entry stops the service at startup rather than silently weakening the fence; the reason is in the log file under `C:\ProgramData\Ducks in a Row\logs`.

Settings reference

All keys live in the `Certus:Acme:ChallengeValidation` section.

Setting	Default	Purpose
`AllowRedirects`	`false`	Whether validation follows HTTP redirects. Leave off so a challenge server cannot redirect the validator to an internal address
`BlockLoopback`	`true`	Refuse loopback validation targets
`BlockLinkLocal`	`true`	Refuse link local targets, including the cloud metadata address
`BlockUniqueLocalIpv6`	`true`	Refuse IPv6 unique local targets
`BlockPrivateRanges`	`false`	Refuse the RFC 1918 private ranges. See above
`AdditionalBlockedCidrs`	`[]`	Extra `address/prefix` ranges to refuse

This screen applies to the validator's own outbound connections only. It is not a host firewall, and it does not constrain what the CA may issue; the allowed domain list does that.

Active Directory integrated DNS and DNS-01

On a new install, external account binding is required and the allowed domain list holds the server's own Active Directory domain. Of the two, binding is the one that matters in an Active Directory domain, and this section says why.

An Active Directory integrated zone with secure dynamic updates, the default, lets **Authenticated Users create a record for any name that does not exist yet**. That is how a new computer registers its own address, and it is not limited to computers: any ordinary domain account can do it, through a dynamic update or straight through LDAP. The account that creates a record owns it and can change it afterwards. It cannot touch a record somebody else already owns.

A DNS-01 challenge asks for a TXT record at `_acme-challenge.<name>`, which is a name of its own in the zone. For most hosts nobody has ever created it, so an ordinary domain user can create it and pass the challenge for an existing host, `web.corp.example.com` included. For a name that does not exist at all, the

same user can create its address record and pass HTTP-01 or TLS-ALPN-01 too. So in such a zone a passed challenge proves only that the requester holds a domain account, not that they run the host.

Binding closes this; the allowed domain list does not. With binding required, an ACME account can only be registered with a credential you handed out, so a user without one never reaches the challenge. The allowed domain list cannot help here, because the forest's own suffix is exactly the list entry and exactly where that user can write. It keeps issuance away from names outside your organization; it does not decide who inside it may ask. Leave binding required, and give a credential a domain namespace when its holder should only order names under one subdomain.

A credential holder can still do the same thing with their own credential, so hand credentials to clients and teams you would trust with a certificate for any name their namespace allows.

The wizard fills the list with one suffix: the primary DNS suffix of the server it runs on. In a forest with several domains, a server in a child domain gets that child's suffix and nothing else, so add the other domains yourself if their clients should be served.

Removing the create right from Authenticated Users on the zone also closes the gap, but it stops new computers from registering their own records, so treat it as a change to your DNS design rather than to Ducks in a Row, and test it first.

The allowed domain list

The allowed domain list on the Settings page is the name restriction: with it switched on, an ACME order may only name the listed domains and their subdomains. On a new install the setup wizard switches it on with the Active Directory domain the server is joined to, and leaves it off in a workgroup, where there is no domain to suggest. It fails closed. If the restriction is on and the list holds no usable entry, which can only happen when `ducks-setup.json` in the data folder was edited by hand, every order is refused until an entry is added or the restriction is switched off, and the service log says so at Error. The same applies when the file cannot be read before the service has loaded it once. A read failure after a good load keeps the last good list. Switching the restriction off is the only way to issue for any name; an empty list is never that.

Staying on a current release

Ducks in a Row never contacts the internet, so it cannot check whether a newer release exists. A fix for a vulnerability reaches you through the security notices list, the GitHub security advisories and the release notes, and the upgrade itself is yours to schedule under your own change control.

What the product can do is show how old the installed release is. The About block on the Settings page shows the release's date beside its version, with the date of the commit it was built from rather than the time of the build, so every build of one release shows the same date. Once that date is older than a threshold, the block warns that a newer release may exist and links the releases page. The same block names the .NET and ASP.NET Core runtime versions the service runs on, because patches to those arrive through Windows Update, not through the Ducks in a Row installer: the bundle installs a runtime only when none is present, and never updates one.

Setting	Default	Purpose
`Certus:ReleaseAgeWarningDays`	`180`	How many days old the installed release may be before the Settings page warns that a newer one may exist. `0` turns the warning off, and so does any negative value; the date and the runtime versions stay shown. Set it in `settings.json` in the data folder, as for any other setting

Nothing here sends anything: the warning is computed from the date inside the installed binaries and the server's clock, and the links are ordinary links that your browser follows only when you click them.

Hardening the CA behind Ducks in a Row

Ducks in a Row is a proxy. It decides *who* may ask for a certificate and *for which names*, then hands the request to Active Directory Certificate Services, which decides what to issue. A hardened proxy in front of an unhardened CA still leaves you exposed, because anything with a domain account can talk to the CA directly and never touch Ducks in a Row at all.

Apply the July 2026 update

CVE-2026-54121, published as "CertiGhost", is a critical elevation of privilege in AD CS with a public proof of concept. A low privilege domain account with network reach to the CA can drive certificate enrollment into impersonating a domain controller, which leads to full domain compromise. Microsoft shipped the fix on 14 July 2026.

Install the July 2026 security update on every CA in the forest. Nothing in this product substitutes for that patch.

What Ducks in a Row does and does not do here. The attack rides in ADCS request attributes: `cdc` points the CA at an attacker controlled host and `rmd` names the domain controller to impersonate. Those attributes travel in the request attribute string, which is a separate parameter of the CA call from the CSR. Ducks in a Row sends exactly one request attribute, `CertificateTemplate`, and builds it in a single place that refuses any template name carrying a control character, a line separator, or a Unicode formatting character, so an ACME client cannot append a second attribute through the attribute string. That closes the attribute string as a route to this bug. It does not fix the bug, it does not stop anyone reaching the CA by any other path, and it says nothing about what a client can put in the CSR, which the next section covers.

What the CSR can and cannot carry

The attribute string is not the only thing a client sends. The CSR is the other, and a Microsoft CA reads more of it than a reading of the PKCS#10 standard suggests. Measured on a lab CA in October 2026: on an enrollee supplies subject template, ADCS honoured a subject alternative name carried under the Microsoft certificate extensions attribute (`szOID_CERT_EXTENSIONS`, `1.3.6.1.4.1.311.2.1.14`) rather than the standard extension request OID, and, on a schema version 1 template on a CA without the November 2024 update, it copied requested Application Policies into the issued certificate (ESC15, CVE-2024-49019). Either is a way to put a name or a capability in a certificate that the ACME identifier check never saw.

Ducks in a Row refuses both at finalize, before anything reaches the CA. A CSR may carry exactly one standard extension request attribute with a single value, and that value may request only a subject alternative name, a key usage, an extended key usage, basic constraints that ask for an end entity certificate (cA=false and no path length, as Posh-ACME sends), or a non critical subject key identifier that is the SHA-1 of the request's own public key; any other attribute or requested extension, including a request for a CA certificate, is refused as `badCSR`. After issuance, a second guard compares the certificate the CA actually returned against the order: a DNS name or a subject common name the order did not authorize, a subject alternative name that is not a DNS name, or an Application Policy outside server and client authentication is refused, and the certificate is revoked. This is defence in depth, not a substitute for patching the CA and restricting who may enrol; a CA that honours a dangerous requested extension is still misconfigured for every client that is not this product.

Reduce the blast radius

Worth doing regardless of patch state, and all of it is CA and directory configuration rather than product settings:

Step	Why
Set `ms-DS-MachineAccountQuota` to `0`	The default of 10 lets any domain user create machine accounts, which several AD CS escalation paths need. Grant machine join to a delegated group instead
Enable CA auditing with `AuditFilter` set to `127`	Records every CA event, including the request attributes on each submission, so an attempt is visible afterwards. `certutil -setreg CA\AuditFilter 127`, then restart the CA service
Review Enroll and Autoenroll on every published template	Enroll rights held by broad groups such as Domain Users or Authenticated Users are what turn a template flaw into a domain wide one
Prefer templates that build the subject from Active Directory	An enrollee supplies subject template trusts the requester for the name. Ducks in a Row validates names before submitting, but a template that does not need that trust is a smaller target
Keep the CA off general purpose networks	The precondition for CertiGhost is network reach to the CA with a domain account. Restricting who can reach the enrollment endpoints shrinks the population that can try

Monitor mode: least privilege on a production CA

If you want Ducks in a Row to watch a CA before you trust it to issue from that CA, run it in monitor mode. The service then never submits a request and never revokes, and the CA team is asked for Read on the CA, plus Request Certificates, which Authenticated Users already hold on a default CA. It needs no Enroll on any template and no Issue and Manage Certificates.

Monitor mode refuses every write in the product at several layers: the ACME routes, the background workers, the dashboard endpoints, and finally the CA client itself. Treat that as defence in depth, not as the

control. The service runs as LocalSystem, as does any paid module it loads, and anything running under that account could call the CA directly. The rights you grant on the CA are the control that holds whatever the software does, so grant only the ones monitor mode needs.

Confirming your own exposure

The service account matters as much as the CA configuration. Ducks in a Row submits every request as its own computer account, so that account's Enroll rights are the ceiling on what any ACME client can obtain through it. Grant Enroll only on the templates you intend to expose over ACME, and check the enabled template list on the setup wizard against that grant.

19. Backup and Restore

Everything worth backing up is in one folder. One thing in it does not survive being moved to another machine, and knowing which is the whole of this page.

What to back up

The data folder, by default `C:\ProgramData\Ducks in a Row\`. If you chose a different one at install time it is recorded in the registry under `HKLM\SOFTWARE\Haruspex Systems\Ducks in a Row`, value `DataDirectory`.

Item	What it holds	Survives a move to another machine
`ducks.db`	ACME accounts, orders, certificate inventory, alert history, EAB credential records, device attestation profiles; see what it keeps, and for how long	Yes
`settings.json`	CA connection string, external URL, alert transport	Yes
`ducks-setup.json`	Enabled templates, allowed domains, EAB enforcement mode, revocation scope	Yes
`logs\`	Serilog daily files, 30 retained	Yes
`keys\`	The Data Protection keyring	**No**, and not to another service account on the same machine
`licence\`	The installed licence file and the previous one	Yes; a licence bound to a forest works only in that forest
`modules\`	The paid module's load record and its own state	Yes
`ducks-selfsigned.bin`	The fallback HTTPS certificate, encrypted with the keyring	Regenerated when it is absent or cannot be read

The application folder under Program Files does not need backing up. Reinstalling replaces it, and it holds nothing you configured.

Taking a backup

Stop the service first, so the database is quiescent and the write ahead log has been folded back in:

```
Stop-Service DucksInARow
Copy-Item -Recurse "C:\ProgramData\Ducks in a Row" "D:\Backups\ducks-$(Get-Date -Format yyyyMMdd)"
```

```
Start-Service DucksInARow
```

If you cannot stop the service, back up the whole folder including the `ducks.db-wal` and `ducks.db-shm` files. Copying `ducks.db` on its own while the service is running gives you a database missing every change still in the write ahead log.

Stopping the service is the simpler answer, and it is quick.

A backup tool that cannot cope with the sidecar files can be accommodated by setting `Certus:EnableWalMode` to `false`, which converts the database to rollback journal mode on the next start and leaves a single file. See configuration.

Restoring onto the same machine

Copy the folder back and start the service. Everything works, including the encrypted secrets, because the Data Protection keyring came back with it and the machine and the service account it is tied to have not changed. The keyring is protected with DPAPI under the account the service runs as, not under the machine's own scope, so the same machine with the service running as a different account counts as a different machine for the secrets: see the next section.

Copy it back as an administrator, so the restored files are owned by the Administrators group. The service ignores `settings.json` and `ducks-setup.json` if anyone else owns them or may write them; if the log says it has, see Troubleshooting.

Restoring onto a different machine

Everything works **except anything that was encrypted**, because the keyring is tied to the machine and the service account that created it. On the new machine:

- **EAB credential secrets** cannot be verified. Clients get 403 `unauthorized` and the log tells you to regenerate.
- **An SMTP password saved from the dashboard** cannot be read, so email alerts do not send.
- **The fallback HTTPS certificate** cannot be read either. The service notices at start, generates a new one and logs it, so a browser that had accepted the old one asks again. A server with a CA issued certificate configured is unaffected.

This is by design. It means a copy of your data folder is not, by itself, a copy of your credentials, and someone who obtains the backup does not thereby obtain the secrets in it.

Recovering takes two steps, and neither loses history:

1. On the ACME page, regenerate each EAB credential. The key id, name, expiry, namespace and owner are all stored in the clear and come across unchanged, so only the HMAC key needs redistributing. See external account binding.
2. On the Settings page, enter the SMTP password again and save.

Everything else, including every issued certificate, every ACME account and the whole alert history, is intact.

Do not try to move the keyring. Copying `keys\` to the new machine does not help, because the files in it are themselves protected with DPAPI under the original service account on the original machine. Regenerating is the supported recovery, and it is quicker than any alternative.

A backup taken from 1.0.0 or earlier

Releases up to 1.0.0 kept the fallback HTTPS certificate as `ducks-selfsigned.pfx`, a PFX under a password that was the same in every copy of the product. Anyone holding such a backup holds that private key. The service now deletes that file when it finds one and keeps the replacement, `ducks-selfsigned.bin`, encrypted with the keyring. Nothing is needed from you beyond accepting the new certificate once in your browser; if the old key is a concern, the backup is the only place it still exists, so remove the file from old backups, or re-encrypt them. A server that serves a CA issued certificate never used the file for anything.

What a backup does not protect

The certificates themselves live in the certificate authority's database, not in this one. Ducks in a Row keeps an inventory of them, not the authority.

So restoring a backup does not reissue, revoke or otherwise change a single certificate. It restores the record of them. Backing up the CA is a separate job and a more important one; if you have not got a CA backup, that is the gap worth closing first.

What the database keeps, and for how long

`ducks.db` is the one place the product keeps records of its own, and some of them are personal data: the contact address on an ACME account, the serial number of a device, the name of the person a credential was handed to, the requester of a certificate in the CA. Your organisation is the controller of that data. This section says what is stored, which of it is personal, how long each class is kept by default, how to change the periods, and how to erase one account's personal data. It is also the statement of the data's security properties the intended use page refers to.

Class	What it holds	Personal data in it	Kept
ACME accounts	The account's public key, status, contact list, and which EAB credential it is bound to	The contact addresses	While the account is valid. A deactivated account loses its contacts one year after deactivation, and its row is deleted once its last order has gone

Class	What it holds	Personal data in it	Kept
ACME orders, authorizations and challenges	The identifiers an order asked for, the stored CSR, the challenge tokens and validation results, and for a device order the attested facts (serial number, UDID) and the attestation object	The identifier values and attested facts of a device order. Hostnames are the names of servers	The authorizations and challenges go 90 days after the order finished (reached `valid` or `invalid`). An invalid order goes with them. A valid order's own row stays with its certificate record, below
Issued certificate records	The certificate Ducks in a Row issued, its serial number, the CA request id, and the revocation stamp	Whatever the certificate names: a device certificate carries the device serial in its subject, as the CA's own copy does	90 days after the certificate expired. Renewal information (ARI), `replaces` on a renewal order and ACME revocation all find a live certificate through this record, so it outlives its order
EAB credentials	The key id, a name, the MAC secret in protected form, and an optional linked Active Directory principal (SID, name and type)	The principal link	Never deleted, because bound accounts refer to them. The principal link is cleared by an account erasure
The CA inventory mirror	Every certificate and request the CA holds: subject, alternative names, template, requester, dates, status, and the certificate itself	Requester names, the subjects of user certificates, and the user certificates themselves	Not subject to retention. It mirrors the CA's database, which keeps the same records, and the next sync would recreate any row removed. It includes user certificates because the inventory exists to show every certificate the CA issued, which is what the dashboard and the expiry alerts are for
Alert history	Which alert was sent about which certificate, CRL or licence, when, to which channels, and whether every channel accepted it	None beyond the subject of the certificate concerned	Two years after the alert was sent

Class	What it holds	Personal data in it	Kept
Administrator audit trail	Every change made through the dashboard or its API: who made it, when, what it touched, and the value before and after. Also the count of refused dashboard sign ins per source address and hour	The administrator's Windows identity, and whatever a change named: an account id, a device serial on an allowlist entry, the owner of a credential. Never a secret or a password	Two years after the action, and never less than 90 days. See the hardening page
Policy rejections	The identifiers an order was refused, the account, the client address, and the stage of the refusal	The client address	The newest 1,000 rows; older ones are removed as new ones arrive
Device attestation profiles, allowlist and trust anchors	Administrator configuration: which templates attest, which device serials may enrol, which roots are trusted	An allowlist entry is a device serial or UDID	Until an administrator removes them
Log files	The service log	Account ids, hostnames, device serials, and the Windows identity of an administrator in the lines that record an administrative action	30 daily files, in `logs\`

The daily pass

Once a day, five minutes after the service starts and every 24 hours after that, a retention pass removes what has run out its period, in batches of 500 rows so that ACME traffic and the dashboard never wait on it, and writes one line to the service log with the counts for each class. It runs in monitor mode too: it writes to this database, never to the CA.

A finished order has no recorded finishing instant, so the pass measures from the facts it has, each a bound from above on the real one: a valid order finished when its certificate was stored, and an invalid order finished no later than its own expiry, seven days after it was created. Measuring from an upper bound never removes anything early; an invalid order may be kept up to a week longer than the period says.

Two consequences are worth knowing. A valid order whose authorizations have gone still answers a client that fetches it, with an empty authorizations list; clients do not revisit an order once they hold its certificate. And an alert history row is also what stops the same warning being sent twice, so a condition that is still true two years after its alert, such as a CRL that was never republished or a lapsed licence that was never

replaced, is warned about once more when its row goes.

Changing the periods

Each period is a setting in `settings.json`, in days, and `0` keeps that class for ever. The settings and their defaults are on the configuration page. The pass reads them at startup, so restart the service after a change. The CA inventory mirror has no setting because it is not subject to retention.

Erasing one account

An administrator can remove one ACME account's personal data while keeping the records the CA audit needs. Two steps, both through the API, and both need the dashboard administrator role:

1. **Deactivate the account.** Erasure refuses a valid account, because its client is still using the contacts and identifiers, and deactivation already cancels the account's open orders. The Deactivate button on the ACME tab does this too.
2. **Erase it.** `POST /api/acme/accounts/{id}/erase`, where `{id}` is the row id the account list shows. From PowerShell, as an administrator of the dashboard:

```
Invoke-RestMethod -Method Post -UseDefaultCredentials -Headers @{ 'X-Certus-Csrf' = '1' }
-Uri https://your-server:5001/api/acme/accounts/42/erase
```

What goes: the account's contact addresses; the Active Directory principal link on the credential the account is bound to; and the identifiers its orders carried, meaning every identifier value in each order, the stored CSR, each authorization's identifier value, attested key and attested facts, and each challenge's attestation object.

What stays: the account row with its public key and the instant of erasure; every order row with its id, status, template, CA request id and timestamps; and every certificate record, the issued certificate included. The certificate names what it was issued to, exactly as the CA's own copy does, and it is what renewal information, `replaces` and ACME revocation still need while it is live. The identifier list keeps each identifier's type with an empty value, so the record still says a device order was a device order.

One EAB credential may bind many accounts, and its principal link names the person the credential was handed to. Erasing one account clears that link for every account bound to the same credential.

The erasure is recorded in the administrator audit trail, in the same database transaction as the erasure itself, with the administrator's Windows identity and the account id, and written to the service log at Warning level, so a raised log level does not drop it. A second erase request for the same account answers with the first erasure's instant, changes nothing and writes no second row. The audit row names the account by its id, which the erasure keeps, and never the data it removed.

20. Troubleshooting

Where to look first:

- **Logs:** `logs\ducks-*.log` in the data folder (default `C:\ProgramData\Ducks in a Row`; daily files, 30 day retention).
- **Windows Event Log:** Application channel, source `DucksInARow.Service`.
- **Health:** `Invoke-RestMethod https://your-server:5001/health` returns `Healthy`, `Degraded`, or `Unhealthy`.
- **Service:** `Get-Service DucksInARow`.

Health result	Meaning
Healthy	Database is accessible and the CA connection works
Degraded	Database works but the CA is not reachable
Unhealthy	Database is not accessible

CA operations answer 503, or the dashboard sends me to the setup wizard

No CA is configured yet. The service starts unconfigured on a fresh install: it serves the setup wizard and answers 503 for every CA operation until the wizard connects a CA. Complete the wizard at `http://localhost:5000/setup`; it discovers the CAs in Active Directory, tests the connection, checks the service's rights on the CA, writes the configuration to `settings.json` in the data folder, and restarts the service.

If the wizard says setup is already complete but the dashboard still reports the CA unavailable, check `settings.json` in the data folder for `Certus:CaConnectionString`, then restart:

```
Restart-Service DucksInARow
```

DISP_E_MEMBERNOTFOUND (0x80020003) in the logs

The ADCS COM classes are not registered on the Ducks in a Row server. Install the ADCS Remote Administration Tools and restart the service. This feature is required even though this server is not the CA:

```
Install-WindowsFeature RSAT-ADCS-Mgmt
Restart-Service DucksInARow
```

Health is Degraded (CA not reachable)

- Confirm `Certus:CaConnectionString` (in `settings.json` in the data folder) is in `CAHOST\CA Name` form.

- Confirm the server is domain joined and can reach the CA.
- Confirm DCOM and RPC are allowed to the CA: TCP 135 plus the dynamic range 49152 to 65535.
- Test from the Ducks in a Row server:

```
certutil -config "CA01.corp.example.com\Corp Issuing CA" -ping
```

The dashboard is empty

- The machine account needs **Read** permission on the CA (Certification Authority console, CA properties, Security tab). Enrolment can work without it, but the inventory sync cannot read the CA database. The log reports "CA view access denied" in that case.
- The first sync may not have run yet. Sync runs every `Certus:SyncIntervalMinutes` (default 5). Restart the service to force one.

The rights check shows Failed or Unproven

The wizard and the **Service rights on the CA** card on the Settings page check what the service's own account may do. Verifying explains the five statuses, and what to ask your AD and CA team says who usually grants each right and where. The rows that come up most:

- **Reaching the CA as this server: Failed, "the CA answered and refused".** The account lacks **Request Certificates** on the CA. Authenticated Users hold it on a default CA, so someone has taken it away. A CA that refuses remote requests altogether (the `IF_NOREMOTEICERTREQUEST` interface flag) answers the same way. The template list, the CA certificate downloads and CRL watching read through the same interface, so they fail too. Also check that the CA server's **Certificate Service DCOM Access** group still admits the account. It holds Authenticated Users on a default CA, and a CA server that refuses the account at DCOM does so before its own permissions are consulted.
- **A row still reads Failed after the right was granted to a group.** The CA sees a computer's new group membership only after its Kerberos tickets renew. Restart the server, or run `klist -li 0x3e7 purge` and restart the service, then **Check again**. See granting to a group.
- **The certificate view the inventory reads: Failed.** Grant **Read** on the CA. Request Certificates does not open the view. If the row says the CA reports Read and still refused the view, the CA may refuse remote administration altogether (the `IF_NOREMOTEICERTADMIN` interface flag).
- **Issue and Manage Certificates: Not granted (optional).** Only revocation needs it, but every kind of revocation: from the dashboard, through `revoke-cert`, and the automatic revocation at finalize. Everything else works without it; see revocation before granting it.
- **Issue and Manage Certificates: Unproven, "the CA would not report the service's roles".** The CA reports roles only to an account holding Read or more. Grant Read, then **Check again**.
- **Enroll on a template: Failed.** Either a deny entry refuses it, or no entry grants it to the account or to any group it belongs to. See certificate template permissions. A grant to a group from another domain of the forest is not visible from the service's server, so a row that says "no grant" can be wrong in a multi domain forest. The first enrolment settles it.

- **Enroll on a template: Inferred.** Not a problem. The permissions grant it, and nothing has enrolled from that template yet.
- **Any row: Unproven, "did not answer within N seconds".** The CA or a domain controller was slow or unreachable. Check again, and read the service log.

A certificate request fails with "unknown certificate template"

The template name in the directory URL must match a template the CA publishes, either by its programmatic name (AD `cn`) or its display name. Names are matched case insensitively. Check the exact names in the setup wizard's template list. Also confirm the template is published to Active Directory and the machine account has **Enroll** on it.

The display name must not carry a control character, a Unicode line separator, or a formatting character. See the next section if the client reports a 400 rather than a 404.

The directory URL answers 400: the URL contains a formatting character

The client reports an error like:

```
urn:ietf:params:acme:error:malformed
The request URL contains a formatting character (U+00AD) at position 9, so it
was refused before routing.
```

Cause. The template's display name carries an invisible character. U+00AD SOFT HYPHEN is the usual one: a word processor inserts them at hyphenation points, and a display name is routinely typed by pasting. U+200B ZERO WIDTH SPACE and the bidirectional overrides do the same thing. These characters are invisible in the Certificate Templates console, on the dashboard, and in the client's own configuration file alike, so the name looks correct everywhere you would go to check it.

The server refuses any URL carrying one before routing, because such a character can forge or disguise a line in the service log, which is the record hardening.md tells you to read after a suspected enrollment attack. Nothing about the template itself is wrong: it still issues normally.

Two fixes, either one works.

- Address the template by its **programmatic name** (its AD `cn`), which never carries these characters in practice. Use `/acme/WebServer/directory` rather than `/acme/Web Server/directory`. This needs no change on the CA.
- Retype the **Template display name** on the **General** tab of the template in the Certificate Templates console. Retype it rather than paste it, or the same character comes back. No republish is needed; the template list is re-read every five minutes.

Where this is reported. The setup wizard's template step flags the selected template under "ACME readiness", naming the code point and the position. The service log carries one warning near startup for any affected template that is exposed over ACME. Neither ever prints the name itself, for the same reason the URL is refused: writing the name would put those characters into the log.

If the programmatic name is the one carrying it, the template cannot be used at all, not over ACME and not from the settings page, because Ducks in a Row will not build an ADCS request attribute string from such a name. The setup wizard hides such a template and counts it in its hidden templates note. A programmatic name is fixed when a template is created, so the fix is to duplicate the template with a clean name and publish the copy.

If the template OID is the one carrying it, nothing is refused. The wizard notes it on the template's "ACME readiness" list, and the startup warning counts it when the template is exposed over ACME, but the template issues certificates and is addressed exactly as any other, because nothing in Ducks in a Row routes on the OID or builds a request from it. It is reported because the wizard prints the OID on every template row to identify the template, and the value you see there is not the value the CA stored: the character is invisible, so the OID looks correct on that row, in the Certificate Templates console, and anywhere else it is shown. The row itself is safe to read. Each of the three values on it is isolated, so a bidirectional override inside one can no longer reorder the text around it. The OID is set when a template is created, like the programmatic name, so there is no field to retype: either ignore the note, which costs nothing, or duplicate the template if you would rather the row read as it is stored.

The directory URL answers 403: template "is not enabled for ACME"

The template exists on the CA but is not exposed over ACME. Exposure fails closed: only templates enabled in the setup wizard are served. Common causes:

- **The template was not selected in the wizard.** Re-run the wizard, or check which template it recorded, and address that template in the directory URL.
- **The wizard has not recorded a template selection yet.** A fresh install exposes nothing over ACME until the wizard's template step records a choice; every template answers 403 before that.
- **The wizard status file is unreadable.** If `ducks-setup.json` in the data folder is corrupt, the service log carries an Error naming the file, and no templates are exposed until the file is fixed or the wizard is re-run.

Setting `Certus:Acme:ExposeAllTemplates` to `true` exposes every CA published template regardless of the wizard's selection. That is a break glass override, named in a startup warning; leave it off in normal operation.

An ACME order is rejected: domain is not in the allowed domain list

The client reports an error like:

```
urn:ietf:params:acme:error:rejectedIdentifier
This server's domain policy does not allow issuance for: web.other.local.
An administrator can change the allowed domains on the Settings page.
```

The allowed domain restriction is on and the requested name falls outside every allowed domain. On the dashboard, open **Settings, Allowed Domains** and add the domain (subdomains of an entry are covered automatically), or turn the restriction off. Changes apply immediately, no restart needed. Each rejected order also appears on the dashboard activity feed as **Rejected**.

Every ACME order is rejected: the allowed domain list is empty or unreadable

Every order gets the `rejectedIdentifier` error above, whatever name it asks for, and the service log carries one of these at Error:

```
Domain restriction is enabled in C:\ProgramData\Ducks in a Row\ducks-setup.json
but no usable entries were found; every ACME order is refused until an allowed
domain is added or the restriction is switched off

Could not read C:\ProgramData\Ducks in a Row\ducks-setup.json; every ACME order
is refused until the file is fixed or the setup wizard is re-run
```

The restriction is on but there is nothing to allow. The wizard and the Settings page refuse to save that state, so the file was edited by hand or damaged. The restriction fails closed on purpose: an operator who switched it on asked for a closed posture. To recover, open **Settings, Allowed Domains** and add at least one domain, or switch the restriction off. For the second message, fix or restore the file (see Backup and restore) or re-run the setup wizard. Orders issue again on the next request; no restart is needed.

An ACME order is rejected: outside the credential's domain namespace

The client reports an error like:

```
urn:ietf:params:acme:error:rejectedIdentifier
The external account credential 'web team' this account registered with does
not allow issuance for: other.example. An administrator can change the
credential's domain namespace on the ACME page.
```

The account is bound to an EAB credential whose domain namespace does not cover the requested name. On the dashboard, open **ACME**, find the credential, and use **Edit** to widen its namespace (or empty the list to remove the restriction). Changes apply to the next order immediately. When the Settings allowed domain list is also on, it stays the ceiling: a namespace entry outside it does not allow issuance there. These rejections appear on the activity feed labeled as blocked by the credential's namespace, so they are distinguishable from allowed domain list refusals.

Registration is refused: externalAccountRequired

```
urn:ietf:params:acme:error:externalAccountRequired
```

The enforcement mode on the ACME page is **Required** and the client tried to register without an external account binding. Create a credential on the ACME page (or use an existing one), hand its key id and HMAC key to the client, and pass them with the client's EAB flags; the credential row's **Client setup** action shows the exact command for each supported client, and Connecting ACME clients lists the flags. Accounts that registered before the mode became Required are not affected; only new registrations need a credential.

Registration or ordering fails with 403 unauthorized naming a credential

The message names the cause:

- **Unknown external account key identifier**: the kid was mistyped, or the credential was deleted from a different install than the client points at. Check the key id against the ACME page.
- **The credential has been revoked**: revocation is terminal and also suspends orders from accounts already bound to it. Create a new credential and re register the client with it.
- **The credential expired on ...**: extend or clear the expiry with the credential's **Edit** action to put it and its accounts back in use, or create a new credential.
- **Signature verification failed**: the client holds a stale secret, most often after a **Regenerate** (every old copy stops verifying the moment the secret rotates). Paste the current secret into the client; if it was never saved, regenerate again and use the new value.
- **The credential is unusable on this server**: see the next section.

EAB secrets stopped verifying after moving or restoring the data folder

The MAC secrets are encrypted at rest with a Data Protection keyring stored in the `keys` folder in the data folder. The keyring is protected with DPAPI under the account the service runs as, on that machine: not the machine's own DPAPI scope, so it decrypts only for that account, there. Restoring the data folder onto a different machine, reinstalling Windows, or running the service as a different account means the keyring can no longer be decrypted, so every stored EAB secret becomes unreadable. Verification fails closed: clients get 403 unauthorized and the log says the stored secret could not be decrypted.

The credentials themselves survive: the key id, name, expiry, and domain namespace are plain columns. The recovery is **Regenerate** on each credential from the ACME page, which issues a new secret on the same key id; hand the new secret to the client and nothing else changes. Already registered accounts are unaffected (the secret is only used at registration).

The dashboard will not let me revoke a certificate

Two independent things can stop it, and the dashboard activity feed says which.

Cause: the capability ceiling. A certificate is revocable from the dashboard only when its extended key usage is present, non empty, and limited to server authentication and client authentication, its key usage carries no certificate or CRL signing, and it is not a CA certificate. A certificate whose capability cannot be read at all is refused as well.

No setting raises this. It is what stops the product from being able to revoke a domain controller certificate or the CA's own certificate, so the fix is not to widen it. Revoke that certificate from the Certification Authority console instead.

Cause: the revocation scope. Underneath the ceiling, the `revocationScope` setting decides which certificates are in reach. The default, ducks-managed, covers certificates Ducks in a Row obtained plus anything on a template you enabled for ACME. If the certificate is on some other template, widen the scope on the Settings page, or revoke it at the CA.

A `custom` scope with an empty template list disables dashboard revocation completely. The startup log says so; check there before assuming something is broken.

Cause: the CA has not granted the right. Revocation needs the server's machine account to hold **Issue and Manage Certificates** on the CA, which is separate from the Read that the inventory needs. The error names the missing right. Without it an ACME client cannot revoke through `revoke-cert` either. See revocation.

An ACME client revoking its own certificate is not subject to the scope modes, so when the scope is what stops the dashboard, a client can still revoke through `revoke-cert`.

A device order is refused, or the device challenge never appears

`device-attest-01` fails closed, and the two refusals look different on purpose.

Cause: no profile, or a disabled one. A template with no device attestation profile does not offer `device-attest-01` at all, and a device order is refused as `unsupportedIdentifier`, exactly as if the feature did not exist. That invisibility is deliberate: it means an install that does not use device attestation gives nothing away about it. Create and enable a profile for the template on the ACME page.

Cause: the device is not on the allowlist. This one is visible. The order is refused with `rejectedIdentifier` and a subproblem naming the device, and the refusal appears on the activity feed labelled as a device attestation policy refusal. Add the device's permanent identifier to the profile's allowlist. An empty allowlist issues to nobody.

Cause: the attestation did not verify. The only supported format is `apple`, and the leaf must chain to the pinned Apple Enterprise Attestation root or to a trust anchor you added. A custom anchor is additive and can never replace the built in root.

Cause: the CSR does not match. The key in the certificate request must be the key the attestation vouches for, with no exception and no setting. A device CSR carrying any DNS subject alternative name is always refused.

The gate is re-checked when the order is created, when the challenge is validated, and at finalize, so disabling a profile mid flight stops an order that was already moving. See device attestation.

An ACME client reports a TLS or certificate trust error

The HTTPS endpoint uses a self signed certificate by default (kept in the data folder as `ducks-selfsigned.bin`, encrypted with the service's keyring). ACME clients reject untrusted TLS on the ACME server. Either:

- give Ducks in a Row a certificate your clients trust (Kestrel HTTPS certificate in `appsettings.json`), or
- add that certificate's root to each client's trust store, or
- for a quick lab test, use the per client switch to skip the check (see Connecting ACME clients).

The dashboard says "Server certificate renewed. Restart to apply it"

Ducks in a Row renews the certificate it serves for its own HTTPS endpoint before it expires. The renewal deliberately does not restart the service: a production service must not bounce itself unannounced, and every ACME client connected at that moment would lose its connection.

So the new certificate is enrolled, installed into `LocalMachine\My`, and recorded in the settings overlay, and the service keeps serving the previous certificate until it restarts. Renewal runs a full window ahead of expiry, so there is no hurry, but the notice stays until you apply it and grows more urgent as the certificate actually in use runs down.

Apply it either way:

- press **Apply now** in the banner, or **Restart now to apply it** in the HTTPS certificate section of the Settings page, or
- restart the service yourself:

```
Restart-Service DucksInARow
```

The superseded certificate stays in the machine store until the restart has happened, because removing it would delete the private key the running service is still using. It is cleaned up automatically on the next renewal check.

The dashboard says automatic renewal of the server certificate failed

Nothing was changed: the service is still serving its current certificate, and the attempt is retried on the next check (daily by default). The Settings page carries the reason the certificate authority gave. The usual ones:

- **denied** — the computer account lost Enroll permission on the template. Grant it on the Security tab of that template.
- **pending** — someone turned on "CA certificate manager approval" for the template, so the CA is holding the request for a person to approve. The server's own certificate is renewed on a schedule and does not wait around for that, so either approve the queued request in the Certification Authority console, or turn the setting off on the Issuance Requirements tab and deny the queued request. ACME issuance on that template is not stopped by this; see the section below for what it does instead.
- **sanMismatch** — the template stopped honouring the requested subject, so the issued certificate no longer covers the external URL host. Set the Subject Name tab back to "Supply in the request". The certificate the CA issued was discarded rather than applied.
- **failed** — usually the CA being unreachable; see the CA connectivity entries above.

If email or webhook alerts are configured, one alert is sent per failed attempt. While automatic renewal is on, the server's own certificate is deliberately excluded from the ordinary 30, 14, 7, and 1 day expiry alerts, because the product renews it itself; this failure alert is the only thing those channels say about it. With automatic renewal off, in monitor mode, or for a certificate the service did not install itself (one without the friendly name `Ducks in a Row HTTPS`, such as a certificate chosen in monitor mode), nothing in Ducks in a Row renews it, so it gets the ordinary alerts like every other certificate.

To turn automatic renewal off, or to change the window, set `Certus:HttpsCertificateAutoRenewalEnabled` or `Certus:HttpsCertificateRenewalWindowDays` in

`settings.json` in the data folder (see Configuration). The default is on, 30 days, capped at a third of the certificate's own validity so a short lived template does not renew on every check.

HTTPS went back to the self signed certificate after a renewal

The browser warns about the certificate again, and the log names the self signed certificate in "HTTPS certificate for this boot". The certificate the service was pointed at by thumbprint has been renewed, and the service could not find its successor at startup. It follows a renewal when the old certificate is still in the machine store, archived, and the new one has the same subject and template; see monitor mode. The Warning line above "HTTPS certificate for this boot" says which condition failed:

- **No certificate has the configured thumbprint**: the old certificate was deleted from the store, so there is nothing to match against.
- **It names no certificate template**: a certificate from a public CA, or one imported from a file.
- **No valid certificate issued after it ... carries its private key**: the renewal has not happened yet, or its key is missing.

Put the new certificate's thumbprint in `Certus:HttpsCertificateThumbprint` in `settings.json`, from an elevated session, and restart the service.

An ACME client waits a long time, and the order stays "processing"

The template has "CA certificate manager approval" turned on, on the Issuance Requirements tab. Every request against such a template is held by the CA until a person approves it in the Certification Authority console, under **Pending Requests**. The setup wizard's readiness checklist flags this when you pick the template, and it is a legitimate way to run a CA, so nothing stops you using it.

What happens meanwhile:

- The order sits in `processing`, which is the correct ACME status for an issuance the server has accepted and not yet completed.
- The service re-checks every held request once a minute. When you approve one, the certificate is collected and the order goes to `valid` on the next check, with no action from the client beyond continuing to poll.
- If you deny the request, the order goes to `invalid` and the client is told the CA refused it.
- If nobody decides either way before the order expires, seven days after it was created, the order goes to `invalid` and the log names the CA request id. **Approving that request afterwards issues a certificate that no longer has an order behind it**, so deny it instead, or revoke what it issues.

Most ACME clients give up waiting long before a person gets to the console. That is a client timeout, not a failure of the order: the order completes regardless, and the client collects the certificate the next time it runs. Clients that renew on a timer handle this by themselves. If you want issuance to complete inside a single client run, the template has to issue without approval.

To change how often held requests are re-checked, set `Certus:Acme:PendingIssuance:PollIntervalSeconds` in `appsettings.json`. The default is 60, and

values are clamped to between 5 seconds and an hour.

An ACME client gets "connection refused"

- The service is running: `Get-Service DucksInARow`.
- The firewall rules exist:

```
Get-NetFirewallRule -DisplayName "Ducks in a Row Certificate Proxy*"
```

- The client reaches the server on the right port (5001 for HTTPS, 5000 for HTTP).

The service ignores a configuration file

The log has a Critical line saying that `settings.json` or `ducks-setup.json` "is being ignored because it cannot be trusted". Then either the dashboard sends you to the setup wizard, or every ACME request is refused because no template is enabled.

The service reads these two files only when no one but SYSTEM and Administrators could have written them:

- the file must be owned by SYSTEM or the Administrators group;
- no one else may have write access to it.

The log line says which rule the file broke. A file anyone else could change is treated as if it were not there, because the dashboard's administrator group and the ACME policy live in these files.

A file in this state was changed by someone other than an administrator, or its permissions were widened by hand. **Check what it contains before you trust it again.** Then either:

- delete it and run the setup wizard again, which writes a fresh one; or
- in an elevated PowerShell, give it back to Administrators, reset its permissions to the folder's, and restart the service:

```
$file = "C:\ProgramData\Ducks in a Row\settings.json"
icacls $file /setowner "*S-1-5-32-544"
icacls $file /reset
Restart-Service DucksInARow
```

Saving from the setup wizard or the Settings page also replaces an untrusted `settings.json` with a fresh file. Because the service does not trust the old file, the new one keeps only what that page saves, so add anything else you had in it again. A save over a file the service trusts keeps every key it does not manage.

The Licence card says the paid module is not running

The card names what happened and shows the service's own reason beneath it: the module failed to start, stopped responding, was set aside after failing to start twice, was built for another release, or sits in a folder that non-administrators can change. Certificate issuance and renewal are not affected in any of these. What to do for each is in licence and paid module.

The Licence card says the licence folder cannot be trusted

Someone other than SYSTEM and Administrators owns the `licence` folder in the data folder, or can write to it, so the service reads no licence from it and never repairs it. Delete the folder the service log names, then install the licence again from the card, which recreates the folder with the right permissions. See licence and paid module.

The service will not start

- Check the log files and the Application event log.
- Validate `appsettings.json` for JSON errors (a trailing comma or missing quote will stop startup).
- Confirm ports 5000 and 5001 are not already in use.
- Check the registry value `Mode` under `HKLM\SOFTWARE\Haruspex Systems\Ducks in a Row`. Anything other than a `REG_SZ` of `monitor` or `full` stops the service at startup, and the log says "The instance mode ... cannot be used". See monitor mode.
- A Fatal line naming `Certus:DatabasePath` or `Certus:SettingsOverlayPath` means one of them points outside the data folder. See a file is outside the data folder.
- If it stopped by itself and came back, or stopped after coming back twice, it ended unexpectedly and Windows' recovery restarted it. The installer sets two restarts; see if the service stops. The System event log records each one, from the Service Control Manager.
- Run the executable directly from an elevated prompt to see the error:

```
& "C:\Program Files\Ducks in a Row\DucksInARow.Service.exe"
```

That last step matters more than it looks. A service that crashes before its logger starts hides its own error: Windows reports a start timeout, which reads like a slow service, and the event log reports a crash in `coreclr.dll`, which reads like a product bug. Only the standalone run prints the real reason.

The installer fails with Error 1920

The full text is "Verify that you have sufficient privileges to start system services", and it almost never means that. The installer is already elevated. The message appears because the service started and then exited before it could signal Windows, so this is the same problem as the section above: read `logs\ducks-*.log` in the data folder first.

A blank or missing log means the crash happened before the logger started.

On Windows Server 2022 the usual cause is an outdated servicing level. The .NET 10 runtime requires Control-flow Enforcement Technology (CET, shadow stacks) on Server 2022, and an installation that has not been patched since early 2022 does not provide it. The process faults inside `coreclr.dll` with exception code `0x80131506`, and running the executable directly prints the actual message:

```
Fatal error. Your Windows doesn't fully support CET. Please install all
available Windows updates.
```

Build 20348.587 is confirmed too old. Check yours with `winver`. Install current Windows updates and retry. Server 2019 predates the CET requirement and is unaffected; Server 2025 ships with it.

The other common cause of a pre-logger crash is a blank `DatabasePath`, which yields the connection string `Data Source=` and opens a private temporary database per connection, so the schema migration fails on a table that cannot exist. Check `settings.json` in the data folder.

An upgrade can also stop here when the database was moved out of the data folder. The log then names `Certus:DatabasePath`; see the next section.

The service will not start: a file is outside the data folder

The log ends with a Fatal line naming `Certus:DatabasePath` or `Certus:SettingsOverlayPath`. The service keeps its database and its settings file in the data folder, `C:\ProgramData\Ducks in a Row` unless you chose another one at install, and refuses to start when configuration puts either of them anywhere else.

The data folder is the one folder the service locks to SYSTEM and Administrators. The database does not move alone: `ducks-setup.json` and the `keys` folder, which protects the EAB secrets and the saved SMTP password, always sit beside it. In a folder that other accounts can write, all three are exposed, so the service no longer starts with the database anywhere else.

Bring the database back into the data folder

1. Stop the service. A clean stop writes `ducks.db-wal` back into `ducks.db` and removes it; if `ducks.db-wal` or `ducks.db-shm` is still there, copy it with the database.
2. Check who owns the files before you copy them. In an elevated PowerShell, set `$old` to the folder that holds the database: Every file should be owned by `NT AUTHORITY\SYSTEM` or `BUILTIN\Administrators`. A file owned by anyone else was created by an account that could write that folder, so do not carry it over:

```
$old = "D:\DucksData"
Get-ChildItem $old -Recurse -File |
    Select-Object FullName, @{ Name = 'Owner'; Expression = { (Get-Acl $_.FullName).Owner }
}
```

   - a file in `keys`: leave it behind. After the move, **Regenerate** every EAB credential on the ACME page and enter the SMTP password again.
   - `ducks.db` or `ducks-setup.json`: do not trust what it holds. Restore it from a backup you trust, or leave it behind; the service then starts with an empty database, syncs the certificate inventory from the CA again, and the setup wizard runs.
3. Copy the files into the data folder as an administrator. Copying, rather than moving, gives each copy the data folder's owner and permissions:

```
$data = "C:\ProgramData\Ducks in a Row"
$files = 'ducks.db', 'ducks.db-wal', 'ducks.db-shm', 'ducks-setup.json' |
    ForEach-Object { Join-Path $old $_ } | Where-Object { Test-Path $_ }
Copy-Item $files $data -Force
New-Item -ItemType Directory -Force "$data\keys" | Out-Null
```

```
Copy-Item "$old\keys\*" "$data\keys" -Force
```

4. Remove `Certus:DatabasePath` wherever it is set: `settings.json` in the data folder, `appsettings.json` in the installation folder, a `Certus__DatabasePath` environment variable, or the service's command line.
5. Start the service. Once the dashboard shows your certificates and ACME accounts, delete the old folder.

To keep the data on another disk, move the whole data folder instead. Choose it at install with `DATAFOLDER` (see installation), or for an existing install: stop the service, copy the entire data folder to the new place as an administrator, set the `DataDirectory` value under `HKLM\SOFTWARE\Haruspex Systems\Ducks in a Row` to the new folder, and start the service. The service locks the new folder to SYSTEM and Administrators on every start, replacing any other permissions on it. It must be on a local disk.

Certus:SettingsOverlayPath

The service reads `settings.json` from the data folder and nowhere else, so any other value of `Certus:SettingsOverlayPath` sent the dashboard's saves to a file the service never read. Remove the key. If the file it named holds settings you want, check its owner as above, copy them into `settings.json` in the data folder as an administrator, and restart the service.

Setup stops: the registry value Mode cannot be used

Setup ends with a message naming the registry value `Mode` under `HKLM\SOFTWARE\Haruspex Systems\Ducks in a Row` and saying Ducks in a Row cannot use it. Someone set the value by hand to something other than `monitor` or `full`, or as a number (`REG_DWORD`, which the message shows with a leading `#`). Setup refuses rather than overwrite it, because overwriting it with `full` would quietly turn on issuance for a server someone meant to keep read only.

Set it from an elevated PowerShell session, then run setup again:

```
New-ItemProperty -Path 'HKLM:\SOFTWARE\Haruspex Systems\Ducks in a Row' -Name Mode -Value monitor -PropertyType String -Force
```

Use `full` for issuance, or delete the value for full mode. An uninstall is never blocked by the value.

Setup stops: the paid module was built for another release

An unattended upgrade ends with a message saying the paid module installed on this server was built for one module contract and this release carries another. After that upgrade the installed paid module would not load, so its features would stay off until the module built for the new release is installed. Setup refuses rather than switch them off without anyone choosing it. Certificate issuance and renewal are not affected either way.

There are two ways forward:

- Get the paid module built for the new release first, then upgrade and install it straight after.

- Or upgrade anyway, saying so:

```
msiexec /i Ducks-in-a-Row.msi /qn ALLOW_INCOMPATIBLE_MODULE=1
Ducks-in-a-Row-Setup.exe /quiet /norestart ALLOW_INCOMPATIBLE_MODULE=1
```

The interactive wizard asks instead, on its first page. The setup bundle's `/passive` runs without the wizard, so it refuses too. Through the setup bundle, the message is in the MSI's own log, which Burn writes beside the bundle's log with `DucksInARowMsi` in its name. An uninstall is never blocked by this check.

Database schema mismatch

The service refuses to start and the log ends with "The existing database was created by an earlier version and its schema does not match this version".

The database predates the introduction of schema migrations and cannot be upgraded automatically. Reset it (adjust the path if you chose a different data folder at install time):

```
Stop-Service DucksInARow
Get-ChildItem "C:\ProgramData\Ducks in a Row\ducks.db*" | Rename-Item -NewName { $_.Name +
'.old' }
Start-Service DucksInARow
```

The service recreates the database at the current schema on start. The synced certificate inventory repopulates from the CA on the next sync. ACME accounts, orders, challenge state, and alert history are lost; ACME clients register again automatically on their next run.

Databases created by any version that shipped with migrations upgrade in place automatically; this reset is only ever needed for databases from before that point.

The database journal mode does not match EnableWalMode

The database runs in one of two journal modes. `wal` (write ahead logging) is the default and performs better under concurrent reads and writes, at the cost of two sidecar files, `ducks.db-wal` and `ducks.db-shm`, sitting beside the database while the service runs. `delete` is SQLite's plain rollback journal, and it works on filesystems that WAL cannot use. It keeps no permanent sidecar: a `ducks.db-journal` appears only for the moment a write is in progress and is removed when that write commits, so seeing one briefly is normal.

`Certus:EnableWalMode` chooses between them. Journal mode is stored in the database file rather than in configuration, so the setting and the file can disagree. Every start reports which mode the database is actually in:

```
SQLite journal mode is delete
SQLite journal mode changed from wal to delete
SQLite journal mode is wal after a request for delete; the change was refused ...
```

Read that line rather than inferring the mode from the setting. The first form means nothing needed doing. The second means the setting was applied. The third means it was not.

When the change is refused

Changing journal mode needs the database file exclusively, and the service can only take it at startup, before it begins serving. Anything else holding the file open at that moment refuses the change: a backup agent, an on access scanner, a replication service, a copy of the database open in a SQLite browser, or a second copy of Ducks in a Row pointed at the same data folder.

The service starts normally either way and retries on every start, so a refusal is not urgent. To clear it, stop whatever holds the file and restart the service, then confirm the logged mode.

The data folder must be on a local disk. A network share is not supported: the service locks the data folder to SYSTEM and Administrators on every start, and on a share those are the file server's own accounts, not this server's.

Cannot reach the dashboard, or get a 401

The dashboard and setup API use Windows Integrated Authentication and are limited to the administrator group. Sign in as a member of the built in Administrators group, or set `Auth:AdminGroup` to the Windows or Active Directory group you want to allow, then restart the service.

A script or tool cannot connect over HTTPS: "Could not create SSL/TLS secure channel"

The HTTPS endpoint (port 5001 by default) accepts TLS 1.2 and TLS 1.3 only. A client that offers nothing newer than TLS 1.0 or TLS 1.1 is refused during the handshake, before any request reaches the service, so the service log shows nothing. What the client reports, measured on Windows Server 2019:

- Windows PowerShell 5.1 (`Invoke-WebRequest`, `Invoke-RestMethod`) on a machine that offers TLS 1.0 only: `The request was aborted: Could not create SSL/TLS secure channel.`
- A .NET Framework `SslStream` whose `AuthenticateAsClient` is given only a host name: `The function requested is not supported`. On Windows Server 2019 that call offers TLS 1.0 alone, even though Windows PowerShell's own web requests on the same machine offer TLS 1.2.

Fix it on the client:

- Use PowerShell 7, which takes its protocols from Windows.
- In Windows PowerShell 5.1, add TLS 1.2 before the request:
  `[Net.ServicePointManager]::SecurityProtocol = [Net.ServicePointManager]::SecurityProtocol -bor [Net.SecurityProtocolType]::Tls12`
- For an `SslStream`, name the protocol: `$ssl.AuthenticateAsClient($hostName, $null, [Security.Authentication.SslProtocols]::Tls12, $false)`

When a client cannot be changed, an administrator can name a wider list in `Kestrel:Endpoints:Https:SslProtocols` (see Configuration). That opens the endpoint to those protocols for every client, and the service logs a warning about it on every start.

Still stuck

Ask in GitHub Discussions or open an issue. For a suspected security problem, follow SECURITY.md instead.

21. Uninstalling

Remove Ducks in a Row through Add or remove programs, or from the command line:

```
msiexec /x Ducks-in-a-Row.msi
```

If you installed with the setup bundle, uninstall the bundle rather than the MSI, so the entry in Add or remove programs is cleaned up too.

What is removed

- Every application file from `C:\Program Files\Ducks in a Row\`, and the folder itself.
- The `modules` folder in it, when it is empty. If the paid module is installed, the folder stays until the paid module's own installer removes it.
- The `DucksInARow` service registration. The service is stopped first, which frees ports 5000 and 5001: the service holds them only while it runs, and there is no HTTP.sys binding or URL reservation to remove.
- The registry key `HKLM\Software\Haruspex Systems\Ducks in a Row` with every value in it: the install and data folder locations, the instance mode, the module contract version and the two installer markers.
- The firewall rules the installer created.
- The Start Menu shortcut and its folder.
- The entry in Add or remove programs.

What is kept

- The SQLite database, `ducks.db` in the data folder.
- The log files in `logs\`.
- `settings.json` and `ducks-setup.json`.
- The Data Protection keyring in `keys\`.
- The self signed certificate the service generated, `ducks-selfsigned.bin`, encrypted with the keyring.
- The `licence` folder, when a licence was installed, and the `modules` folder the paid module keeps its state in, when one was installed.
- The data folder itself, by default `C:\ProgramData\Ducks in a Row\`, which holds all of the above.

Keeping the data folder is deliberate. It means an uninstall and reinstall, or an upgrade that goes through one, does not lose your CA configuration, your ACME accounts or your alert history.

Three more things stay, outside the data folder:

- **The HTTPS certificate in the machine store.** When the setup wizard, the Renew button or the automatic renewal enrolled the server's certificate from your CA, that certificate and its private key stay in `LocalMachine\My`, with the friendly name `Ducks in a Row HTTPS`. A certificate somebody else installed and you chose in monitor mode was never ours to remove.

- **The event log source** `DucksInARow`, a registry key under `HKLM\SYSTEM\CurrentControlSet\Services\EventLog\Application`, which the service registers the first time it runs.
- **The rights granted to the server's computer account** on the CA and on the certificate templates. They live on the CA and in Active Directory, and nothing on this server can remove them.

The ASP.NET Core runtime is never removed, even if the setup bundle installed it. Other applications on the server may be using it.

This list was measured on a lab server, not read from the installer's source: a fresh install, the setup wizard completed and the server's certificate enrolled from the CA, then an uninstall and an inventory of the registry, the file system, the certificate stores, HTTP.sys, the firewall and the CA.

The -wal and -shm files are gone, and that is correct

While the service is running you will see `ducks.db-wal` and `ducks.db-shm` beside the database. After an uninstall they are gone.

That is a good sign, not a loss. The database uses write ahead logging, and SQLite writes that log back into `ducks.db` and removes both files when the last connection closes, which is exactly what happens when the uninstaller stops the service. Their absence means every committed change reached `ducks.db`, and that `ducks.db` on its own is now a complete database you can copy or restore.

Removing everything

Uninstalling, then deleting the data folder, returns the server to the state it was in before Ducks in a Row was installed, apart from the runtime and the three items below. Delete the folder by hand after uninstalling:

```
Remove-Item -Recurse -Force "C:\ProgramData\Ducks in a Row"
```

Take a copy first if you might reinstall. The `keys\` folder in there is the Data Protection keyring, and it is the only thing that can decrypt stored EAB secrets and a dashboard saved SMTP password. Deleting it is not recoverable by reinstalling. See backup and restore.

Deleting the folder takes the database, the logs, the configuration files, the keyring, the self signed certificate, the `licence` folder and the paid module's state with it. Then, from an elevated Windows PowerShell session:

1. **Remove the HTTPS certificate** the service enrolled, with its private key. Only certificates carrying the service's friendly name are matched, so a certificate somebody else installed is left alone: `-DeleteKey` removes the private key with the certificate; without it the key container stays on disk. The same certificate can be removed in `certlm.msc`, under Personal. If you want it to stop working as well as to disappear, revoke it at the CA before you delete it; it stays valid at the CA until it expires otherwise.

```
Get-ChildItem Cert:\LocalMachine\My |
  Where-Object FriendlyName -eq 'Ducks in a Row HTTPS' |
  Remove-Item -DeleteKey
```

2. **Remove the event log source.** `Remove-EventLog` exists in Windows PowerShell 5.1, not in PowerShell 7:

```
Remove-EventLog -Source DucksInARow
```

3. **Ask the CA team to take back the rights** the server's computer account was granted: Enroll on each exposed template, Read and Request Certificates on the CA, and Issue and Manage Certificates if revocation was enabled. What to ask your AD and CA team lists each right and where it was granted, so the same page serves as the list to revoke. If the server itself is being decommissioned, disabling or deleting its computer account removes them all at once.

The ASP.NET Core runtime stays, as above. Remove it through Add or remove programs only if nothing else on the server uses it.

Certificates already issued are not affected

Uninstalling removes the proxy, not the certificates it obtained. Every certificate already issued stays valid until it expires, and stays in the CA's own database. Nothing is revoked.

That cuts both ways: if you are decommissioning the server and want the certificates it obtained to stop working, revoke them **before** you uninstall, while the dashboard can still reach them. See revocation.

22. Glossary

Short, plain explanations of the terms used across these guides.

ACME

A standard protocol (RFC 8555) for requesting and renewing certificates automatically. It is the same protocol Let's Encrypt uses on the public internet. Ducks in a Row speaks ACME on your internal network.

ACME client

A program that requests certificates over ACME, such as certbot, win-acme, Caddy, Traefik, or Posh-ACME. It runs on the machine that needs a certificate.

ADCS (Active Directory Certificate Services)

The Microsoft certificate service built into Windows Server. It runs your internal certificate authority. Ducks in a Row sits in front of ADCS and lets ACME clients enrol from it.

CA (Certificate Authority)

The system that issues and signs certificates. In this product the CA is your ADCS server.

Certificate template

A profile on the CA that decides what a certificate is for and what it may contain, for example a "Web Server" template for TLS server certificates. Ducks in a Row exposes each template as its own ACME endpoint.

ACME directory

The starting URL an ACME client connects to. Ducks in a Row gives each template its own directory URL in the form `https://your-server:5001/acme/<template>/directory`.

Challenge

The check an ACME server runs to confirm you control the name you are asking a certificate for. Ducks in a Row supports three types:

- **HTTP-01:** the server fetches a token over HTTP on port 80.
- **DNS-01:** the server looks up a `TXT` record in DNS. Good for wildcards.
- **TLS-ALPN-01:** the server checks a certificate over a TLS connection on port 443.

RSAT (Remote Server Administration Tools)

Optional Windows features for managing server roles from another machine. Ducks in a Row needs the ADCS part (`RSAT-ADCS-Mgmt`) because it registers the COM components used to talk to the CA. Install it with `Install-WindowsFeature RSAT-ADCS-Mgmt`.

DCOM and RPC

The Windows technologies Ducks in a Row uses to call the CA over the network. They need TCP port 135 plus a range of dynamic ports (49152 to 65535) open to the CA. The CA server also admits an ordinary account only through its **Certificate Service DCOM Access** group, which holds Authenticated Users on a default CA. See what to ask your AD and CA team.

Machine account

The Active Directory account of the Windows computer itself, also called its computer account, written with a trailing `$` (for example `DUCKS-SERVER$`). Ducks in a Row acts as this account, so the CA permissions are granted to the machine account, not to a person.

Self signed certificate

A certificate a server generates for itself, with no trusted authority behind it. Ducks in a Row uses one for HTTPS until you give it a certificate your clients already trust. Browsers and ACME clients warn about self signed certificates.